Workgroup:	Network Working Group
Internet-Draft:	draft-ietf-rpp-data-objects-02 (editorial)
Published:	2 July 2026
Intended Status:	Standards Track
Expires:	3 January 2027
Authors:	P. Kowalik	M. Wullink
---	---	---
	DENIC	*SIDN Labs*

RESTful Provisioning Protocol (RPP) Data Objects

Abstract

This document defines data objects for the RESTful Provisioning Protocol (RPP) and sets up the IANA RPP Data Object Registry to describe and catalogue them. Specifically, it details the logical structure, constraints, and protocol operations (including their inputs, outputs, and business logic) for foundational resources: domain names, contacts, and hosts. In accordance with the RPP architecture [I-D.ietf-rpp-architecture], these definitions focus entirely on the semantics, remaining independent of any specific data representation or media type (e.g., JSON or XML).

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of BCP 78 and BCP 79.

Internet-Drafts are working documents of the Internet Engineering Task Force (IETF). Note that other groups may also distribute working documents as Internet-Drafts. The list of current Internet-Drafts is at https://datatracker.ietf.org/drafts/current/.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as "work in progress."

This Internet-Draft will expire on 3 January 2027.

Copyright Notice

Copyright (c) 2026 IETF Trust and the persons identified as the document authors. All rights reserved.

This document is subject to BCP 78 and the IETF Trust's Legal Provisions Relating to IETF Documents (https://trustee.ietf.org/license-info) in effect on the date of publication of this document. Please review these documents carefully, as they describe your rights and restrictions with respect to this document. Code Components extracted from this document must include Revised BSD License text as described in Section 4.e of the Trust Legal Provisions and are provided without warranty as described in the Revised BSD License.

Table of Contents

1. Introduction 10
1.1. Conventions and Terminology 10
2. Resource Definition Principles 11
2.1. Primitive Data Types 11
2.1.1. String 12
2.1.2. Integer 12
2.1.3. Boolean 12
2.1.4. Decimal 12
2.1.5. Date 12
2.1.6. Timestamp 12
2.1.7. URL 12
2.1.8. Binary 12
2.1.9. Dictionary 12
2.1.10. Object 13
2.1.11. Fully Qualified Domain Name (FQDN) 13
2.2. Data Element Abstraction 13
2.3. Extensibility 13
2.3.1. Standardised and Private Extensions 13
2.3.2. Extension Points 14
2.4. Data Element Semantics 14
2.5. Inheritance 15
2.6. Reserved Property Names 16
2.7. Operations 16
2.7.1. Authorisation 16
2.7.2. Operation Input and Output 17
2.7.3. Include Elements 17
2.7.4. Uniform interface 18
2.7.4.1. Create 18

2.7.4.2. Read 18
2.7.4.3. Query 19
2.7.4.4. Update 19
2.7.4.5. Delete 19
2.7.5. Offline Review 19
2.7.6. Operations beyond uniform interface 20
2.7.7. Transfer Operations 20
2.7.7.1. Transfer Messages 20
2.7.8. Restore Operations 22
2.7.9. Renew Operations 22
2.8. EPP Compatibility Profile 22
3. Internationalized Domain Names (IDN) 23
4. Profiles 23
4.1. Inheritance 23
5. Messages 24
6. External Data Types 24
6.1. Referencing External Data Types 24
6.2. Versioning and Compatibility 25
6.3. JSContact 25
7. Common Data Types 25
7.1. Identifier 25
7.2. Client Identifier 25
7.3. Phone Number 26
8. Associations 26
8.1. Aggregation 26
8.2. Composition 27
8.3. Labelled Aggregation 28
8.4. Dictionary Aggregation 28
8.5. Labelled Composition 29
8.6. Dictionary Composition 29

9. Component Objects 30
9.1. Period Object 30
9.2. Provisioning Metadata Object 31
9.3. Domain Name Status Object 32
9.3.1. Status Values 33
9.3.2. Allowed Transitions 35
9.3.3. Pending Status Removal 37
9.3.4. Status Exclusions 37
9.3.5. Redemption Grace Period 38
9.3.5.1. Allowed Transitions 39
9.4. Host Status Object 41
9.4.1. Status Values 41
9.4.2. Allowed Transitions 43
9.4.3. Pending Status Removal 45
9.4.4. Status Exclusions 45
9.5. Contact Status Object 45
9.5.1. Status Values 46
9.5.2. Allowed Transitions 47
9.5.3. Pending Status Removal 49
9.5.4. Status Exclusions 49
9.6. Organisation Role Status Object 49
9.6.1. Status Values 50
9.6.2. Allowed Transitions 51
9.6.3. Status Exclusions 51
9.7. Organisation Status Object 52
9.7.1. Status Values 52
9.7.2. Allowed Transitions 54
9.7.3. Pending Status Removal 56
9.7.4. Status Exclusions 56

9.8. User Status Object 56
9.8.1. Status Values 57
9.8.2. Allowed Transitions 58
9.8.3. Pending Status Removal 59
9.8.4. Status Exclusions 59
9.9. DNS Resource Record Object 59
9.9.1. RDATA Structures in EPP Profile 60
9.10. DNS Operational Controls Object 61
9.11. DNS Data Object 62
9.12. Authorisation Information Object 63
9.13. Disclose Object 64
9.14. Restore Report Object 64
9.15. Organisation Role Object 66
9.16. Processes Object 66
9.17. Base Process Object 67
9.18. Messages 68
9.18.1. Base Message Object 68
9.18.2. Review Response Object 69
9.18.3. Transfer Request Message Object 70
9.18.4. Transfer Outcome Message Object 71
9.18.5. Expiration Deletion Message Object 72
9.18.6. Auto-Renewal Message Object 73
9.18.7. Service Notice Message Object 73
9.19. Discovery 74
9.19.1. Version Object 74
9.19.2. Environment Object 74
9.19.3. Extension Object 75
9.19.4. IDN LGR Object 76
9.19.5. Notice Object 77

10. Process Objects 77
10.1. Transfer Process Object 78
10.1.1. Operations 79
10.1.1.1. Create (Transfer Request) 79
10.1.1.2. Read (Transfer Query) 80
10.1.1.3. Delete (Transfer Cancel) 80
10.1.1.4. Approve 81
10.1.1.5. Reject 81
10.2. Restore Process Object 82
10.2.1. Operations 83
10.2.1.1. Create (Restore Request) 83
10.2.1.2. Read (Restore Query) 84
10.2.1.3. Update (Restore Report) 84
10.3. Renew Process Object 85
10.3.1. Operations 86
10.3.1.1. Create (Renew Request) 86
10.4. Create Process Object 86
10.4.1. Operations 87
10.4.1.1. Create 87
10.4.1.2. Read 87
11. Domain Name Data Object 87
11.1. Object Description 87
11.2. Data Elements 88
11.3. Operations 92
11.3.1. Create Operation 92
11.3.2. Read Operation 92
11.3.3. Update Operation 93
11.3.4. Delete Operation 93

11.4. Processes 94
11.4.1. Domain Create Process Object 94
11.4.1.1. Operations 94
11.4.2. Domain Transfer Process Object 95
11.4.2.1. Operations 96
12. Contact Data Object 96
12.1. Object Description 96
12.2. Data Elements 97
12.3. Operations 99
12.3.1. Create Operation 99
12.3.2. Read Operation 99
12.3.3. Update Operation 100
12.3.4. Delete Operation 100
12.4. Processes 100
13. Host Data Object 101
13.1. Object Description 101
13.2. Data Elements 101
13.3. Operations 102
13.3.1. Create Operation 102
13.3.2. Read Operation 103
13.3.3. Update Operation 103
13.3.4. Delete Operation 103
13.4. Processes 104
14. Organisation Data Object 104
14.1. Object Description 104
14.2. Data Elements 104
14.3. Operations 107
14.3.1. Create Operation 107
14.3.2. Read Operation 107
14.3.3. Update Operation 107

14.3.4. Delete Operation 108
15. Message Data Object 108
15.1. Object Description 108
15.2. Data Elements 109
15.3. Operations 110
15.3.1. Create Operation 110
15.3.2. Read Operation 110
15.3.3. Query Operation 110
15.3.4. Delete Operation 111
16. User Object 111
16.1. Object Description 111
16.2. Data Elements 111
16.3. Operations 112
16.3.1. Create Operation 112
16.3.2. Read Operation 113
16.3.3. Update Operation 113
16.3.4. Delete Operation 113
17. Discovery Data Object 113
17.1. Object Description 113
17.2. Data Elements 114
17.3. Operations 116
17.3.1. Read Operation 116
18. Profile Data Object 116
18.1. Object Description 116
18.2. Data Elements 117
18.3. Operations 119
18.3.1. Read Operation 119
19. IANA Considerations 119
19.1. RPP registry group 119

19.2. RPP Data Object Registry 119
19.2.1. Registration Policy 120
19.2.2. Registry Structure 120
19.2.3. Initial Registrations 120
19.3. RPP User Role Values Registry 163
19.4. RPP Domain Name Status Values Registry 164
19.4.1. Initial Registrations 164
19.5. RPP Host Status Values Registry 168
19.5.1. Initial Registrations 168
19.6. RPP Contact Status Values Registry 170
19.6.1. Initial Registrations 170
19.7. RPP Organisation Role Status Values Registry 172
19.7.1. Initial Registrations 172
19.8. RPP Organisation Status Values Registry 173
19.8.1. Initial Registrations 173
19.9. RPP User Status Values Registry 175
19.9.1. Initial Registrations 175
19.10. URN Sub-namespace for RPP (urn:ietf:params:rpp) 176
19.11. RPP Discovery registry 176
19.12. RPP Extension registry 177
19.13. RPP Profile registry 177
20. Security Considerations 178
22. References 180
22.1. Normative References 180
22.2. Informative References 182
Authors' Addresses 183

1. Introduction

The RESTful Provisioning Protocol (RPP) defines a set of data objects used to represent and manage foundational registry resources, including domain names, contacts, and hosts. This initial list is not exhaustive; additional resource and component objects MAY be defined in future revisions or introduced via IANA registration to support new features and operational needs.

In accordance with the RPP architecture [I-D.ietf-rpp-architecture], a core architectural principle is the clear distinction between the abstract data model and its concrete data representation. The data model defines the logical structure, relationships, and constraints of the objects, independent of formatting. The data representation defines how these abstract concepts are expressed in specific formats (e.g., JSON, XML, or YAML).

This document focuses on the data model of RPP objects and operations on them, including the data model of operation inputs, outputs as well as the necessary business logic of state transitions. This separation of concerns ensures the protocol maintains a stable semantic foundation that can be consistently implemented across different media types and easily adapted to new representation formats. For instance, the model defines a contact's name as a required string type, but it remains agnostic as to whether that string is ultimately encoded as a JSON property or an XML element.

1.1. Conventions and Terminology

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "NOT RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in BCP 14 [RFC2119] [RFC8174] when, and only when, they appear in all capitals, as shown here.

The following terms related to the relationship between host objects and domain objects are used as defined in Section 1.1 of [RFC5732]:

Subordinate host
: A host name object is a subordinate host of a domain when the host name consists of one or more labels followed by the name of that domain, for example, host `ns1.domain.example` is a subordinate host of domain `domain.example`.

Superordinate domain
: A domain name object is the superordinate domain of a host when the name of the host consists of one or more labels followed by the name of that domain, for example, domain `domain.example` is the superordinate domain of host `ns1.domain.example`.

Internal host
: A host whose name belongs to the namespace of the repository in which the host is being used for delegation purposes.

External host
: A host whose name does not belong to the namespace of the repository in which the host is being used for delegation purposes.

Object Authorisation
: Authorisation data related to the object beyond the default client-level authorisation accompanying the request, in order to authorise operations on the object. Typically it is a value related to or derived from an Authorisation Information Object provisioned with the object.

Data Object A top-level provisioned resource object that has an independent lifecycle and identity in the registry. Data Objects carry data elements describing the resource and define the set of operations that can be performed on them.

Component Object A reusable data structure that carries data only, with no operations of its own. Component Objects are embedded within Data Objects or other objects to avoid repetition of common data patterns.

Process Object An object that represents a long-running or multi-step operation initiated on a Data Object. Process Objects carry operation-related state and data, and may define their own operations to interact with the process. They have no independent existence - their lifecycle is bound to the owning Data Object.

Owner Data Object A Data Object which a process (represented as Process Object) was initiated upon and which owns this Process Object.

Unique Identifier The data element of an object whose value uniquely identifies an instance of that object within its applicable scope. The scope is defined by the object: for a Data Object it is typically the server, whereas for a Process Object it is the Owner Data Object. An object definition names its Unique Identifier explicitly in the object preamble. Objects that have no independent identity, such as component objects embedded within another object, have no Unique Identifier.

Input Data Element A data element that a client provides as input to a specific operation, in addition to the identifier or data elements of the object the operation targets, and that carries content the operation acts upon. An input data element is defined as part of the operation definition and is not part of the state of the object. An input data element is described using the attributes defined in Section 2.4, except Direct Access, and Visibility, which do not apply.

Parameter A data element that a client provides with an operation to control how the server performs the operation or which data it returns. Unlike an Input Data Element, a parameter does not carry content that the operation acts upon. A parameter is not part of the state of the object and is never returned as part of the object. A parameter is described using the same attributes as an Input Data Element, and its definition specifies the behaviour of the operation when the parameter is absent.

Label Generation Ruleset (LGR) A set of rules defining the valid labels for a registry, including the permitted repertoire, contextual rules, and, where applicable, variant mappings. Also historically known as IDN tables.

2. Resource Definition Principles

2.1. Primitive Data Types

RPP data elements use strict typing, meaning that each element must conform exactly to its declared primitive data type, and type violations MUST be treated as errors by implementations. The exact specifications for these types, including allowed ranges, encoding, and formatting, are determined by the representation format used (e.g., JSON, XML, CBOR). New RPP non-primitive data types based on existing primitive data types MAY be defined to support additional features.

2.1.1. String

A String is a sequence of Unicode characters. Usages MAY impose additional constraints on string values, such as maximum length or allowed character sets, based on specific data element definitions. An example of a string is `"host.example"`.

2.1.2. Integer

An Integer is a whole number, positive or negative. Usages MAY impose additional constraints on integer values, such as minimum and maximum allowable values, based on specific data element definitions. An example of an integer is `42`.

2.1.3. Boolean

A Boolean represents a logical true or false value. An example of a boolean is `true` or `false`, but it MAY be different depending on the native encoding of boolean values in the representation.

2.1.4. Decimal

A Decimal is a number providing an exact, base-10 representation of fractional values within a defined precision. Usage of this type MUST impose additional constraints on decimal values, such as precision or range, based on specific data element definitions. An example of a decimal is 3.14159.

2.1.5. Date

A Date is a full-date calendar date as described in [RFC3339], an example of a date is `2025-10-27`.

2.1.6. Timestamp

Timestamp (Date and time attribute) values MUST be represented in Universal Coordinated Time (UTC) using the Gregorian calendar in date-time form as defined in [RFC3339]. In EPP Compatibility Profile upper case "T" and "Z" characters MUST be used. An example of a timestamp is `2025-10-27T09:42:51Z`.

2.1.7. URL

A Uniform Resource Locator (URL) as defined in [RFC1738]. An example of a URL is `"https://host.example"`.

2.1.8. Binary

Raw binary data, implementations MAY choose how to encode the binary data, for example as base64 or hexadecimal string. An example of binary data encoded as base64 is `"UlBQIFNheXMgSGk="`.

2.1.9. Dictionary

Notation: Dictionary[Value Type]

A Dictionary is a collection of key-value pairs where keys are unique Strings and values are of the specified primitive data type. Usages MUST define the constraints on allowed keys and values. A Dictionary differs from an Aggregation Dictionary in that it maps String keys to primitive values rather than to Component or Resource Objects.

2.1.10. Object

An Object is a composite structure containing named properties. The set of allowed property names, their data types, and their constraints are determined by the data element definition that uses this type. Usages MUST specify the expected structure, including any required or optional properties. An Object differs from a Component Object in that it is defined inline as part of a data element rather than being a standalone reusable definition registered separately.

2.1.11. Fully Qualified Domain Name (FQDN)

A String that represents a Fully Qualified Domain Name (FQDN) that conforms to the description included in section 2 of [RFC9499] and includes the following additional restrictions:

- The trailing dot required when these names are stored in a DNS zone is implicit and MUST NOT be used in RPP, with the exception of the `name` element of the DNS Resource Record Object.
- Internationalized domain names MUST be represented in ASCII Compatible Encoding (ACE) using the A-label, defined in Section 2.3.2.1 of [RFC5890]
- Case-insensitive comparison MUST be used when comparing two FQDNs.
- All representations generated by the server are to be in lowercase.

2.2. Data Element Abstraction

Each data object is composed of logical data elements. A data element is a logical unit of information identified by a stable name, independent of its representation in any given media type. The definition for each element specifies its logical name, purpose, cardinality, data type, and constraints. The data type of a data element may also be a reference to another data object, using the target object's stable name.

2.3. Extensibility

The RPP data model is designed to be extensible. Extensions MAY introduce new data objects, add new data elements or associations to existing objects, define new processes on existing or new data objects, and define new operations or extend the inputs and outputs of existing operations.

2.3.1. Standardised and Private Extensions

RPP distinguishes between standardised and private extensions:

- Standardised extensions MUST be registered with IANA, as described in the IANA Considerations section of this document and in [I-D.ietf-rpp-architecture]. Registration ensures global uniqueness of extension identifiers and promotes interoperability across implementations.

- Private (non-standardised) extensions MAY be defined for use within specific implementations or organisations. Private extensions are not required to be registered with IANA, but MUST still use unique identifiers that are unlikely to conflict with standardised extensions or other private extensions. The use of reverse domain notation as a prefix (e.g., `org.example.rpp.myElement`) is RECOMMENDED for private extension identifiers to avoid naming collisions.

2.3.2. Extension Points

The following aspects of the data model are extensible:

- New data objects: Additional resource or component objects MAY be defined by extensions and registered with IANA.
- New data elements: Extensions MAY add new data elements to existing data objects. Such elements follow the same Data Element Semantics as core data elements.
- New associations: Extensions MAY introduce new associations between existing or new objects.
- New operations: Extensions MAY define entirely new operations on existing or new data objects.
- New processes: Extensions MAY define new Process Objects and MAY declare that existing or new Data Objects support them. A new Process Object MUST extend the Base Process Object, either directly or through another Process Object.
- Extended operation inputs and outputs: Extensions MAY add new data elements to the inputs or outputs of existing operations.

When an extension adds data elements, input data elements, or parameters to a core object or operation, these additions MUST NOT alter the semantics or constraints of existing core data elements.

2.4. Data Element Semantics

The definition of each data element within an object consists of the following attributes:

- Name: A human-readable name for the data element.
- Identifier: A machine-readable, unique identifier for the element, using camelCase notation.
- Cardinality: Specifies the number of times an element may appear. The notation is as follows:
  - `1` for exactly one
  - `0-1` for zero or one
  - `0+` for zero or more
  - `1+` for one or more
  - and `X-Y` for at least X and at most Y, where X and Y are non-negative integers, X is less than Y, and Y is greater than 1, e.g. `1-2` for one or two
- Data Type: Defines the element's data structure, which can be a primitive type, a Common Data Type or a reference to another component object.

- Description: Explains the purpose of the data element and any other relevant information.
- Constraints: Provides specific validation rules or limitations on top of the data type itself, such as value ranges.
- Mutability: Defines the lifecycle of the data element's value. It MUST be one of the following:
  - create-only: The element's value is provided during the object's creation and cannot be modified thereafter. After object's creation the element behaves like read-only element.
  - read-only: The element's value is managed by the server. It cannot be set or modified directly by the client, though it may change as a result of server-side operations.
  - read-write: The element's value can be set and modified by the client.
- Direct Access: An optional flag indicating whether the associated object or objects are additionally exposed as an addressable sub-resource of the containing object. When set to `true`, the associated object or objects are exposed as a sub-resource that can be addressed independently of the containing object, in addition to their inline representation within it. When absent or set to `false`, the element is accessed only inline. The default value is `false`.

A data element is required when the lower bound of its cardinality is greater than zero (`1`, `1+`, or `X-Y` with X greater than 0); otherwise it is optional.

The Direct Access flag is applicable only to a data element whose Data Type is an association (Aggregation, Composition, Labelled Aggregation, Dictionary Aggregation, Labelled Composition, or Dictionary Composition) with a Data Object, Component Object, or Process Object, of any cardinality. Where such a data element has a cardinality greater than `1`, the associated object type MUST define a Unique Identifier, so that an individual associated object can be addressed unambiguously among the collection. * Visibility: An optional attribute indicating whether the data element is included in operation output by default. It MUST be one of the following: * default: The element is included in the output of any operation returning the containing object, subject to authorisation and server policy. This value applies when the attribute is absent. * onDemand: The element is included in the output only when the client explicitly requests it using the Include Elements parameter (see Section 2.7.3), subject to authorisation and server policy.

The Visibility attribute affects operation output only. It does not affect whether a client can provide the element as input.

2.5. Inheritance

A Data Object, Component Object or Process Object (the "derived object") MAY inherit the data elements and operations of another object (the "base object"). Inheritance is denoted in the preamble of the derived object's definition using the `Extends` keyword, whose value is the name of the base object, for example:

```
* Name: Domain Create Process Object
* Identifier: domainCreateProcess
* Extends: Create Process Object
```

The following rules apply:

- Single inheritance: an object MUST extend at most one base object.
- Transitivity: the base object MAY itself extend another object. Inheritance is transitive, so a derived object inherits the data elements and operations of every ancestor in its chain. An object MUST NOT directly or indirectly extend itself.
- Inherited content: a derived object has all data elements of its base object, including their cardinality, mutability, data type and constraints, and supports all operations of its base object. Inherited data elements and operations are not repeated in the derived object's definition.
- Additions: a derived object MAY define additional data elements and operations. It MUST NOT define a data element whose identifier is already defined by any of its ancestors. A derived object MAY modify the inherited operation input and output data as needed.
- Polymorphism: where a data element or operation refers to a base object, any object that directly or indirectly extends the base object MAY be used in its place.
- Abstract objects: an object MAY be declared abstract using the `Abstract` keyword in the preamble of its definition, with the value `true`. When the `Abstract` keyword is absent or set to `false`, the object is not abstract. An abstract object serves only as a base object for its derived objects. Wherever a data element or operation refers to an abstract object, an instance of a non-abstract object that directly or indirectly extends it is used in its place. A client MUST NOT provide, and a server MUST NOT return, an instance whose own type is an abstract object. The `Abstract` keyword is not inherited.

2.6. Reserved Property Names

Data object definitions MUST NOT define properties whose names begin with the `@` character. Names beginning with `@` are reserved for use by representation specifications. This prohibition applies to all data object types: resource objects, component objects, and process objects.

2.7. Operations

For each data object a set of possible operations is defined together with their respective input and output data.

2.7.1. Authorisation

For each operation authorisation requirements and operation behaviour is specified.

Wherever "Object Authorisation" is mentioned, it means that an operation MAY accept or require additional authorisation data related to the object beyond default client-level authorisation, or that an operation MAY result in different processing or response if such authorisation is provided.

2.7.2. Operation Input and Output

The input and output of an operation are declared by the `Input` and `Output` attributes of the operation definition. The value of each attribute is either a literal `(None)` or a list of items, each of one of the following kinds:

- Reference: a reference to the context of operation or process. It can contain either a literal value `This Object` for the object that defines the operation, or literal value `Owner Data Object` for the Owner Data Object of a Process Object.
- Object: The name of the object type in the input or output. It can contain either the type or a literal value `This Object` for the object type that defines the operation, or literal value `Owner Data Object` for the type of Owner Data Object of a Process Object. It specifies an additional attribute:
  - Cardinality: [OPTIONAL] the number of representations, in the notation defined in Section 2.4. The default value is `1`.
- Input Data Elements: the input data elements of the operation, listed in the same form as the Data Elements of an object: each entry is the Name of an input data element, followed by its attributes as defined in Section 2.4, except Mutability, Direct Access and Visibility. It is allowed in Input only, at most once.
- Parameters: the parameters of the operation, listed in the same form as Input Data Elements. It is allowed in Input only, at most once.

For example:

```
* Input:
  * Reference: Owner Data Object
  * Object: Transfer Process Object
* Output:
  * Object: Transfer Process Object
```

2.7.3. Include Elements

The `includeElements` parameter is defined for every operation whose output contains one or more objects, including, but not limited to, the Uniform interface operations and the operations of Process Objects. It allows a client to request specific data elements to be included in the operation output in addition to those included by default. Individual operation definitions do not repeat it.

- Include Elements
  - Identifier: includeElements
  - Cardinality: 0-1
  - Data Type: String
  - Description: A comma-separated list of Identifiers of data elements that the client requests to be included in the operation output.

    - Constraints: Each list item is the Identifier of a top-level data element of the object or objects returned by the operation. Data elements nested within component objects or other associated objects cannot be listed. Identifiers are case-sensitive. The list contains no whitespace.

A listed Identifier refers to the data element as a whole. For a data element with a cardinality of `0+`, `1+`, or `X-Y`, it refers to the entire collection of values, and individual values cannot be selected. For a data element whose Data Type is a component object or an association, it refers to the complete associated object or objects.

The server processes this data element as follows:

- If `includeElements` is absent, the server MUST NOT include any data element with `onDemand` Visibility in the output.
- If a listed Identifier is not the Identifier of a top-level data element defined for the returned object, including data elements defined by extensions not supported by the server, the server MUST reject the request with an error.
- For each listed Identifier, the server MUST include the referenced data element in the output if it is present on the object and the client is authorised to view it. Otherwise the element is omitted without an error.

Listing a data element that is already included in the output by default has no effect. The parameter thereby remains applicable where an operation or a representation defines an output that omits some elements by default.

2.7.4. Uniform interface

For the Create, Read, Query, Update and Delete operations of a Data Object, the following Input and Output apply, using the notation defined in Section 2.7.2. An operation definition that declares its own Input or Output attribute replaces the corresponding default.

2.7.4.1. Create

- Input:
    - Object: This Object
- Output:
    - Object: This Object

2.7.4.2. Read

- Input:
    - Reference: This Object
- Output:
    - Object: This Object

The output Object MAY vary depending on the identity of the querying client, use of Object Authorisation information, and server policy towards unauthorised clients.

If the querying client is the sponsoring client, all available information MUST be returned, except data elements with `onDemand` Visibility not requested using `includeElements` (see Section 2.7.3).

If the querying client is not the sponsoring client but the client provides valid Object Authorisation information, all available information SHOULD be returned, however some optional elements MAY be reserved to the sponsoring client only.

If the querying client is not the sponsoring client and the client does not provide valid Object Authorisation information, server policy determines which OPTIONAL elements are returned, if any, or whether the entire request is rejected.

2.7.4.3. Query

- Input: (None)
- Output:
  - Object: This Object
    - Cardinality: 0+

2.7.4.4. Update

- Input:
  - Reference: This Object
  - Object: This Object
- Output:
  - Object: This Object

2.7.4.5. Delete

- Input:
  - Reference: This Object
- Output:
  - Object: This Object
    - Cardinality: 0-1

2.7.5. Offline Review

RPP allows for offline review for operations that change the state of an object (Uniform Interface Create, Update, Delete, and the Renew and Transfer process operations) before the requested action is actually completed. In such situations, the response from the server MUST clearly indicate that the operation has been received, but that the requested action is pending. The state of the corresponding object MUST clearly reflect processing of the pending action, typically through a server-set pending status label (e.g. `pendingCreate`, `pendingUpdate`, `pendingDelete`, `pendingRenew`, `pendingTransfer`). The server MUST also notify the client when offline processing of the action has been completed, by sending a `review-response` message (see Section 5).

2.7.6. Operations beyond uniform interface

For all other operations both input and output have to be fully specified.

2.7.7. Transfer Operations

Transfer operations manage the change of sponsorship of a provisioned object from one client (the sponsoring client) to another (the gaining client). They are specified once in this section as the transfer model is common across all transferable resource objects. Individual object definitions reference this section and specify any object-specific extensions to the common pattern.

RPP supports two types of transfer:

- Pull Transfer: Initiated by the gaining client. The gaining client MUST provide valid Object Authorisation to initiate the request.
- Push Transfer: Initiated by the sponsoring client, who designates a gaining client in the request.

The transfer process MAY be immediate or follow a multi-step workflow depending on server policy. If the server implements immediate transfers, the Approve and Reject operations need not be supported; the server completes the transfer upon receipt of the Create operation.

The server MAY implement local policies to prevent transfers from stalling and implement a form of automated transfer escalation, approval or cancellation when such a stalled process is recognised.

All transfer operations act on or return the Transfer Process Object and are executed in the context of the Owner Data Object the operation is created upon.

If the server does not directly approve a pull transfer request and the current sponsoring client first approve the transfer request, the server MUST wait for the sponsoring client's decision before proceeding. If the sponsoring client approves, the server completes the transfer; if the sponsoring client rejects, the server cancels the transfer. The server MUST send a `transfer-request` message to the sponsoring client. For a pending push transfer, the server MUST send a `transfer-request` message to the designated gaining client that must act on it. If the server directly approves a transfer, it MUST send a `transfer-outcome` message to the client managing the object before the transfer. When a transfer is resolved after the currently managing client's decision, the server MUST send a `transfer-outcome` message to the requesting client.

2.7.7.1. Transfer Messages

The Pull Transfer Direct Approval Message Diagram illustrates the sequence of messages exchanged during a pull transfer with direct approval. The Pull Transfer Pending Approval Message Diagram illustrates the sequence of messages exchanged during a pull transfer that requires approval. The Push Transfer Message Diagram illustrates the sequence of messages exchanged during a push transfer.

For a pull transfer, the requesting client is the gaining client; for a push transfer, the requesting client is the sponsoring client. The messages in the diagrams are surrounded by square brackets to indicate message types.

Pull transfer (direct approval):

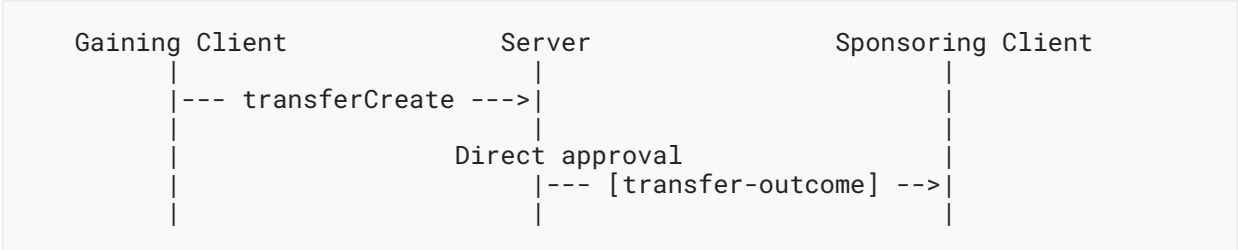


Figure 1: Pull Transfer Direct Approval Message

Pull transfer (pending approval):

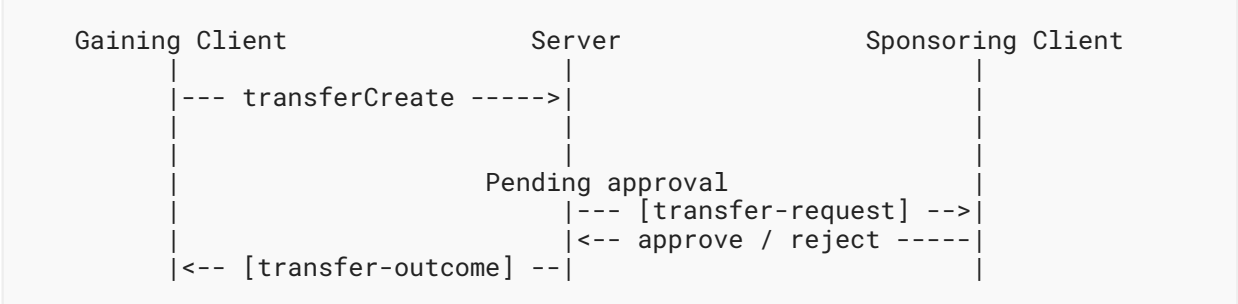


Figure 2: Pull Transfer Pending Approval Message

Push transfer:

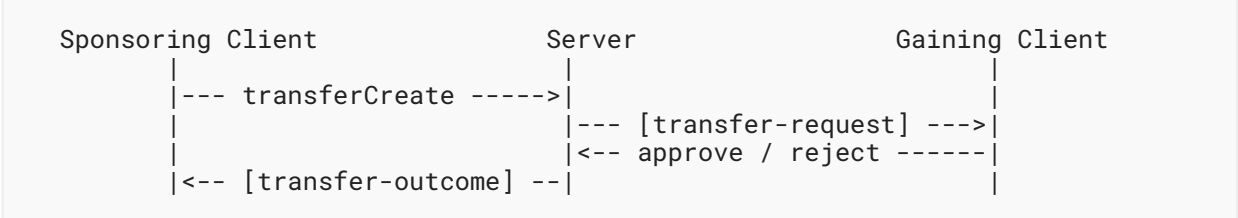


Figure 3: Push Transfer Message Diagram

> TODO: Transfer-specific error conditions (object not eligible for transfer, object pending transfer, object not pending transfer) are not defined in this document.

2.7.8. Restore Operations

Restore operations manage the recovery of an object that has entered the Redemption Grace Period (RGP). They are OPTIONAL and are only available when the RGP feature is supported by the server and MAY be supported only for a subset of Data Object types.

The RGP process MAY involve two distinct steps:

- Restore Request: (REQUIRED) Initiates the recovery of an object in the `rgpRedemptionPeriod` state, signalling to the registry the intent to restore. This corresponds to the Create operation on the Restore Process Object. On success, the object transitions to `rgpPendingRestore`.
- Restore Report: (OPTIONAL) Submits a report documenting the circumstances of the deletion and restoration, as required by registry policy. This corresponds to the Update operation on the Restore Process Object. On success, the object is returned to its pre-deletion status and all RGP status labels are removed.

Whether a restore report is required after a restore request is a matter of server policy. If the server does not require a restore report, the object returns to its pre-deletion status immediately upon a successful Create operation, bypassing the `rgpPendingRestore` state.

The Create operation MAY include the restore report, provided as the `restoreReport` data element, to complete both steps atomically in a single operation.

All restore operations act on or return the Restore Process Object and are executed in the context of the Owner Data Object the operation is created upon.

2.7.9. Renew Operations

Renew operations manage the validity period of a provisioned object. They are specified once in this section as the renewal model is common across all renewable resource objects. Individual object definitions reference this section and specify any object-specific extensions to the common pattern.

2.8. EPP Compatibility Profile

RPP is designed to coexist with the Extensible Provisioning Protocol (EPP), often operating in parallel against a common backend provisioning system. While RPP is not inherently constrained by all of EPP's requirements, a specific set of rules is necessary to ensure seamless interoperability in such mixed environments.

To address this, this document defines an "EPP Compatibility Profile". This profile specifies a set of additional constraints on RPP data objects and operations that a server MUST adhere to when supporting both RPP and EPP concurrently.

Throughout this document, all constraints that are part of this profile are explicitly marked with a reference to "EPP Compatibility Profile". Implementers of systems in a mixed EPP/RPP environment MUST follow these specific constraints in addition to the base RPP requirements.

3. Internationalized Domain Names (IDN)

Internationalized Domain Names (IDN) are domain names that include one or more non-ASCII characters. IDNs are represented in the Domain Name System (DNS) using ASCII Compatible Encoding (ACE), commonly known as Punycode. This allows the DNS to handle Unicode domain names while maintaining compatibility with the existing ASCII-based infrastructure.

4. Profiles

A profile is a named set of protocol features and versions that are used to define the compatibility and capabilities of RPP server implementations, allowing for better interoperability between different implementations. Using profiles helps to simplify the implementation and deployment of RPP by providing a clear and concise way for the client and server to communicate their capabilities and requirements.

A profile is identified by a unique name and may be published as a standard profile in the IANA registry for RPP profiles, to promote interoperability and standardization across implementations. Standard profiles names MUST use the RPP URN namespace defined in this document, for example: `urn:ietf:params:rpp:profile:{profile_name}`. The profile name MUST be unique within the RPP URN namespace and SHOULD be descriptive of the features and versions included in the profile.

A profile can also be defined as a private profile, which is not published in the IANA registry, but is used by specific server implementations only. A private profile can be defined by a server operator to specify the features and versions supported by their implementation. If the profile is not published in the IANA registry, then the server operator MUST ensure that the profile name is globally unique to avoid conflicts with other profiles, the use of reverse domain name notation is RECOMMENDED for private profiles to ensure uniqueness.

4.1. Inheritance

A profile can include another profile, which is referred to as "inheriting" from that profile. When a profile inherits from another profile, it means that the features and versions defined in the inherited profile are also included in the inheriting profile. This allows for the creation of more complex profiles by building upon existing profiles. For example, a profile named "example-profile" could inherit from a base profile named "base-profile", which includes a set of common features and versions. The "example-profile" could then add additional features and versions on top of the ones defined in the "baseProfile", or it can override some of the features and versions from the "baseProfile". This allows for greater flexibility and modularity in defining profiles, as well as promoting reuse of common features and versions across different profiles. However, it is important to note that the use of inheritance in profiles can also make things more complicated, as it can create dependencies between profiles and make it harder to understand the features and versions included in a profile. Therefore, it is recommended to use inheritance with caution

and to clearly document the relationships between profiles. The depth of inheritance MUST be limited to 1 to provent excessive complexity, and profile designers MUST NOT create circular dependencies between profiles, where a profile inherits from itself directly.

5. Messages

Messages are used to communicate events, requests, and responses between clients and the server. They provide a mechanism for notifying clients about changes in the state of objects, the outcome of operations, and other relevant information.

For a message reporting completion of an offline review, the server MUST send a `review-response` message to each client involved in the transaction when the requested action has been completed and the pending status has been removed.

The server can send `transfer-outcome`, `expiration-deletion`, and `auto-renewal` messages to clients affected by those events. The `service-notice` conveys operational information to all clients. Extensions should use the `Base Message Object` as the foundation for defining new message types for notifications of changes to Data Objects.

6. External Data Types

The RPP object model is designed to be extensible and interoperable with existing standards. To reduce redundancy and improve consistency, RPP data objects MAY incorporate data types and structures defined in external specifications rather than redefining equivalent types within this specification.

An external data type is any type, structure, or format that is normatively defined in a separate standards document and referenced by this specification. When an external data type is used, RPP implementations MUST conform to the semantics, constraints, and encoding rules defined in the originating specification.

6.1. Referencing External Data Types

When this document or any RPP specification references an external data type, the reference MUST follow the requirements and guidelines for references described in [RFC2026]. The reference MUST include a clear description of the specific type or structure being used. For example, if referencing the JSContact format defined in [RFC9553], the reference should specify that the `Card` data type is being used. The reference MUST use the following format: `External:<specification-identifier>:<type-name>`. For example, `External:RPP-JSContact-Profile:Card` indicates that the `Card` type from the RPP JSContact Profile specification is being used. The syntax for the reference is defined using ABNF [RFC5234] as follows:

```
identifier = "External:" TYPEID ":" TYPEID

TYPEID = ALPHA *( ALPHA / DIGIT / "-" / "_" ) ( ALPHA / DIGIT )
```

6.2. Versioning and Compatibility

External data types are versioned by the specification that defines them. The RPP specifications are pinned to a single version of an external type. If a newer version updates the defining specification, the RPP specification MUST be updated to indicate explicit support.

6.3. JSContact

This specification references the RPP JSContact Profile for RPP defined in [I-D.wullink-rpp-jscontact-profile] as an external specification for representing contact information. The JSContact `Card` data type defined in [RFC9553] MUST be used for the representation of contact information. When referring to this type the following values MUST be used in the Data Type reference identifiers:

specification-identifier: RPP-JSContact-Profile
type-name: Card
Data Type: External:RPP-JSContact-Profile:Card

7. Common Data Types

This section defines new shared data types and structures that are re-used across multiple data object definitions and are based on the existing Primitive Data Types.

7.1. Identifier

Identifiers are character strings with a specified minimum length, a specified maximum length, and a specified format outlined in Section 2.8 of [RFC5730]. Identifiers for certain object types MAY have additional constraints imposed either by server policy, object specific specifications or both.

7.2. Client Identifier

Client identifiers are character strings with a specified minimum length, a specified maximum length, and a specified format. Client identifiers use the "clIDType" client identifier syntax described in [RFC5730].

> TBC: do we need this or is it a relation with an entity/RFC8543 organisation? If registrars are modelled as first class objects (organisations), then clID is nothing else but a reference to this organisation, so maybe no need to define syntax separately on identifier level (or in other words it would be defined on this object). R8.1 in the form of -02 RPP requirements includes RFC8543.

7.3. Phone Number

Telephone number syntax is derived from structures defined in [ITU.E164.2005]. Telephone numbers described in this specification are character strings that MUST begin with a plus sign ("+", ASCII value 0x002B), followed by a country code defined in [ITU.E164.2005], followed by a dot (".", ASCII value 0x002E), followed by a sequence of digits representing the telephone number. An optional "x" (ASCII value 0x0078) separator with additional digits representing extension information can be appended to the end of the value.

8. Associations

RPP allows for different types of associations (relationships) between the objects. The association may be added between 2 objects with own independent lifecycle (UML aggregation) or in the relation when one object's existence and lifecycle is bound to the other parent/owner object (UML composition). In both cases, especially if the relation allows for cardinality higher than one on either side, the association may be assigned additional attributes, not being part of an object on either side of relation. In many cases such relation would be attributed with a single text string label, describing a role or a type of relation. Depending on the context this value might be unique, which allows using such label as a key in a dictionary.

The following generic Association Types are defined for RPP:

8.1. Aggregation

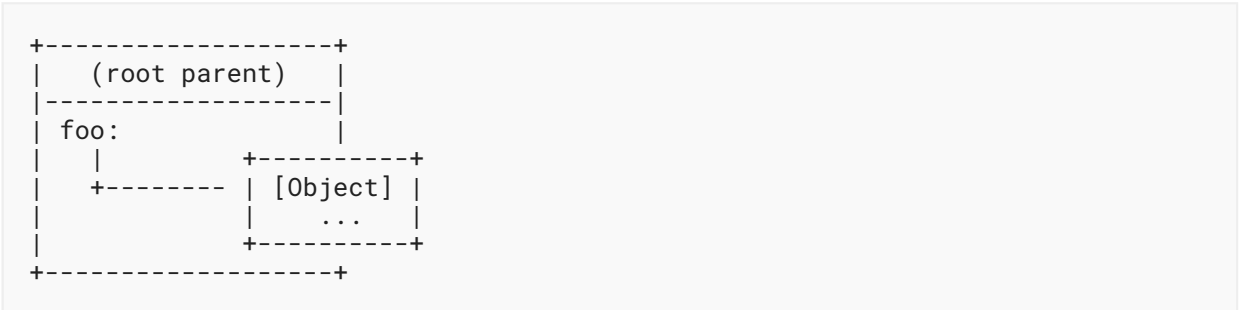
Notation: Aggregation[Type]

A relation between two independent objects.

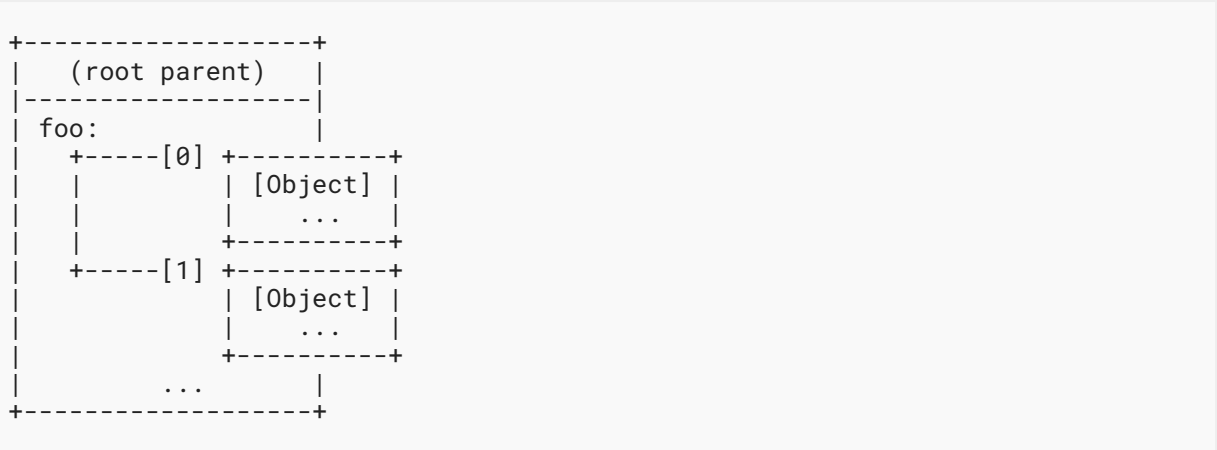
If the cardinality of the target object is more than 1, this represents an unordered array.

The target object of an aggregation MUST already exist when the association is created. The target object MAY be only partially visible to the client, or not visible at all, for example when it is managed by the server or by another client and the client only knows its identifier. Limited visibility of the target object does not prevent the client from using it in the association. This rule also applies to Labelled Aggregation and Dictionary Aggregation.

Example aggregation having cardinality 1:



Example aggregation having cardinality >1:



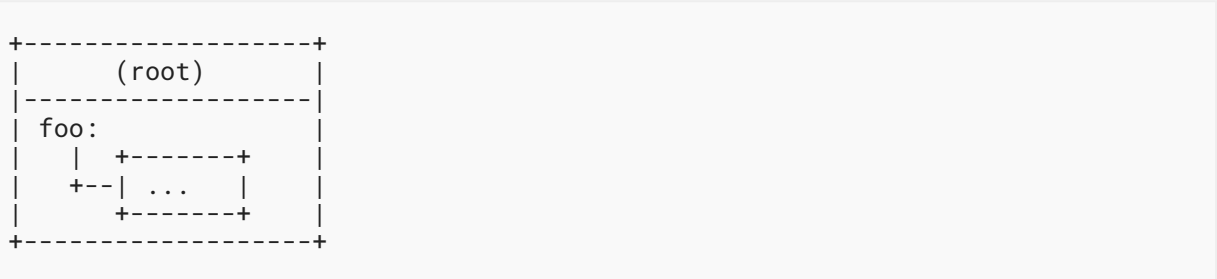
8.2. Composition

Notation: Type

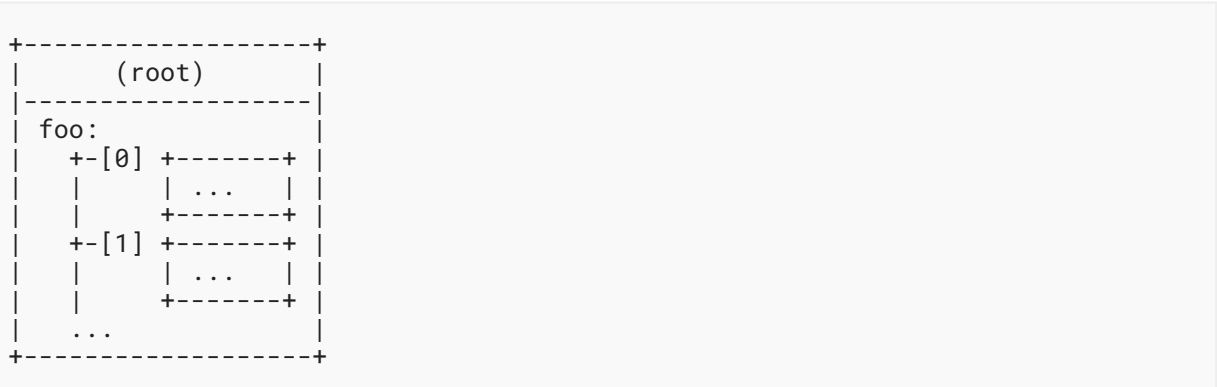
A relation between an independent parent object and 1 or more dependent child object(s).

If the cardinality of the target object is more than 1, this represents an unordered array.

Example composition having cardinality 1:



Example composition having cardinality >1:



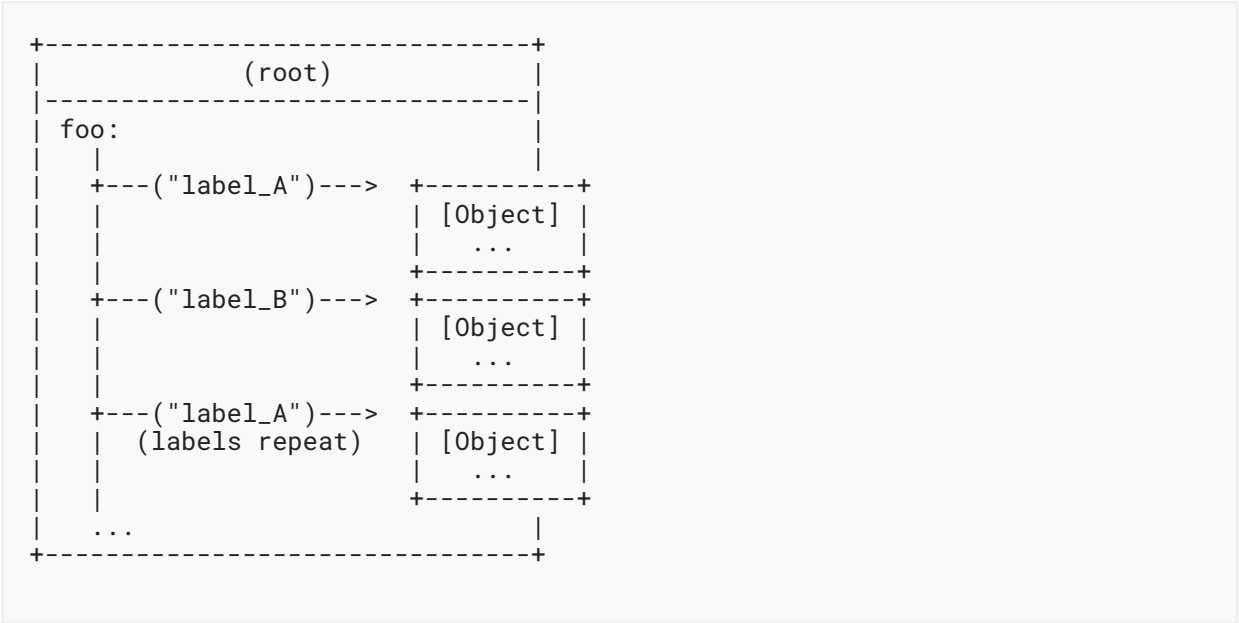
8.3. Labelled Aggregation

Notation: LabelledAggregation[Type]

A relation between two independent objects with a single text string attribute. Multiple associations with the same label are allowed and represent an unordered array.

A type defining such association MUST define Label Description with semantics of the label and Label Constraints with constraints related to the label.

Example labelled aggregation:



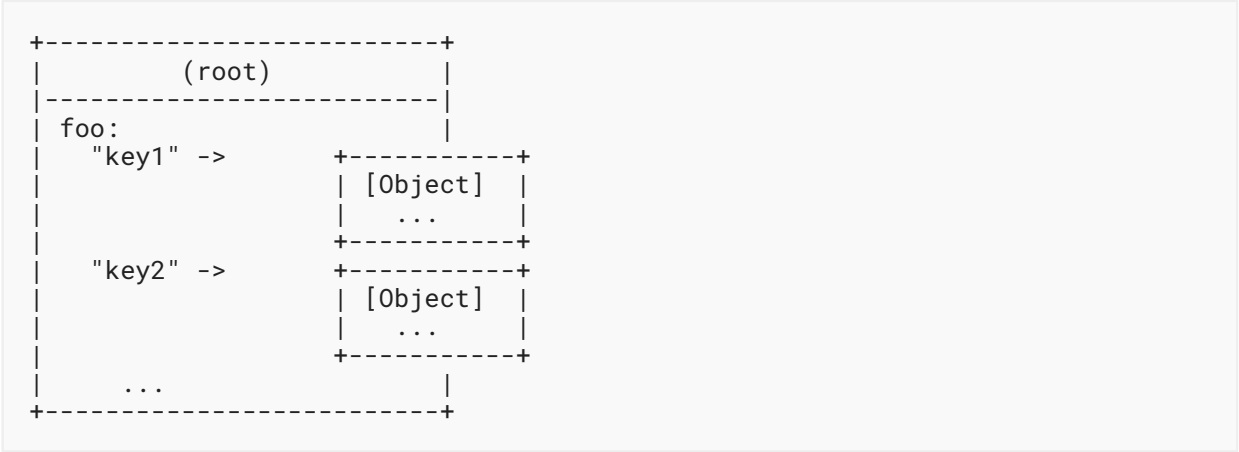
8.4. Dictionary Aggregation

Notation: DictionaryAggregation[Type]

A relation between two independent objects with a single text string attribute. Association labels MUST be unique, allowing them to be used as dictionary keys.

A type defining such association MUST define Label Description with semantics of the label and Label Constraints with constraints related to the label.

Example Dictionary Aggregation:



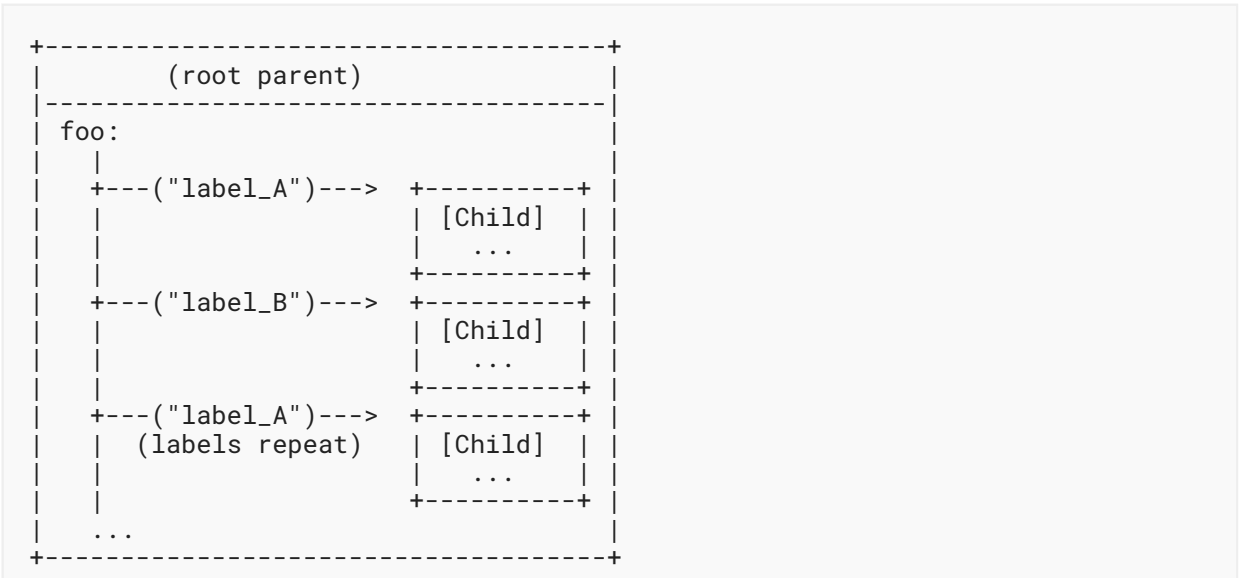
8.5. Labelled Composition

Notation: LabelledComposition[Type]

A relation between an independent parent object and a dependent child object with a single text string attribute. Multiple associations with the same label are allowed.

A type defining such association MUST define Label Description with semantics of the label and Label Constraints with constraints related to the label.

Example Labelled Composition:



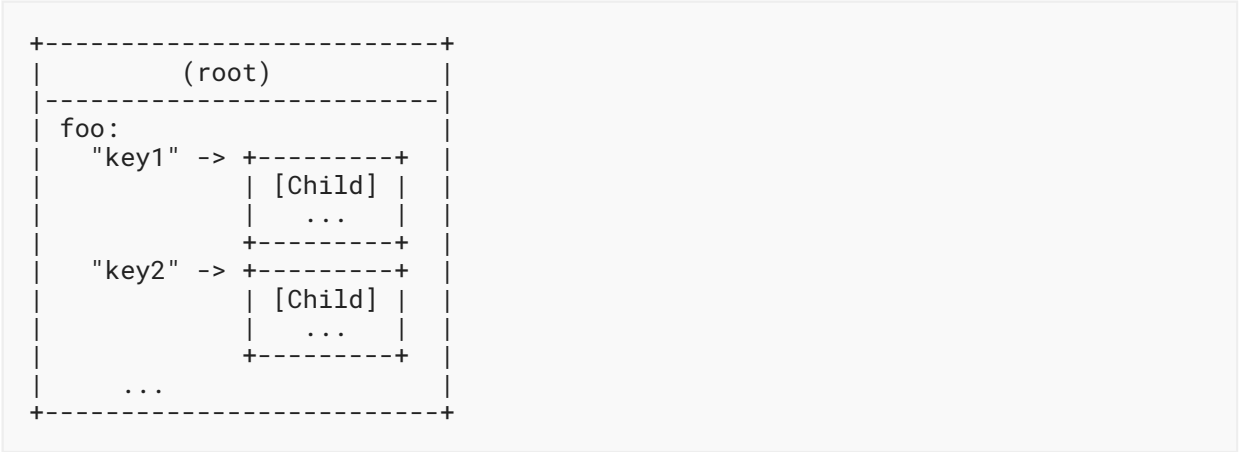
8.6. Dictionary Composition

Notation: DictionaryComposition[Type]

A relation between an independent parent object and a dependent child object with a single text string attribute. Only a single association with the same label is allowed, allowing it to be used as a dictionary key.

A type defining such association MUST define Label Description with semantics of the label and Label Constraints with constraints related to the label.

Example Dictionary Composition:



9. Component Objects

This section defines the Component Objects used in this document.

9.1. Period Object

- Name: Period Object
- Identifier: period
- Description: Represents a duration of time.
- Data Elements:
  - Value
    - Identifier: value
    - Cardinality: 1
    - Mutability: read-write
    - Data Type: Integer
    - Description: The numeric value of the period.
    - Constraints: The value MUST be from 1 to 99, inclusive.
  - Unit
    - Identifier: unit
    - Cardinality: 1
    - Mutability: read-write

        - Data Type: String
        - Description: The unit of the period.
        - Constraints: The value MUST be one of: "y" (years) or "m" (months).

9.2. Provisioning Metadata Object

- Name: Provisioning Metadata Object
- Identifier: provMetadata
- Description: Contains standard metadata about the lifecycle and ownership of a provisioned object. This metadata is common across all resource objects in the registry system.
- Data Elements:
  - Repository ID
    - Identifier: repositoryId
    - Cardinality: 0-1
    - Mutability: read-only
    - Data Type: Identifier
    - Description: A server-assigned unique identifier for the object.
    - Constraints: In EPP Compatibility Profile this data element MUST be provided.
  - Sponsoring Client ID
    - Identifier: spClientId
    - Cardinality: 1
    - Mutability: read-only
    - Data Type: Client Identifier
    - Description: The identifier of the client that is the current sponsor of the object.
    - Constraints: (None)
  - Creating Client ID
    - Identifier: crClientId
    - Cardinality: 0-1
    - Mutability: read-only
    - Data Type: Client Identifier
    - Description: The identifier of the client that created the object.
    - Constraints: (None)
  - Creation Date
    - Identifier: crDate
    - Cardinality: 0-1
    - Mutability: read-only
    - Data Type: Timestamp
    - Description: The date and time of object creation.
    - Constraints: The value is set by the server and cannot be specified by the client.

    - Updating Client ID
        - Identifier: upClientId
        - Cardinality: 0-1
        - Mutability: read-only
        - Data Type: Client Identifier
        - Description: The identifier of the client that last updated the object.
        - Constraints: This element MUST NOT be present if the object has never been modified.
    - Update Date
        - Identifier: upDate
        - Cardinality: 0-1
        - Mutability: read-only
        - Data Type: Timestamp
        - Description: The date and time of the most recent object modification.
        - Constraints: This element MUST NOT be present if the object has never been modified.
    - Transfer Date
        - Identifier: trDate
        - Cardinality: 0-1
        - Mutability: read-only
        - Data Type: Timestamp
        - Description: The date and time of the most recent successful object transfer.
        - Constraints: This element MUST NOT be provided if the object has never been transferred.

9.3. Domain Name Status Object

- Name: Domain Name Status Object
- Identifier: domainStatus
- Description: Represents one of the status values associated with a Domain Name Data Object
- Data Elements:
    - Label
        - Identifier: label
        - Cardinality: 1
        - Mutability: create-only
        - Data Type: String
        - Description: machine-readable enum label of a status
        - Constraints:
            - Only status values defined in Domain Status Values and Redemption Grace Period Status Values are allowed.

- Reason
  - Identifier: reason
  - Cardinality: 0-1
  - Mutability: create-only
  - Data Type: String
  - Description: a human-readable text that describes the rationale for the status applied to the object.
  - Constraints: None
- Due
  - Identifier: due
  - Cardinality: 0-1
  - Mutability: read-write
  - Data Type: Timestamp
  - Description: a timestamp, when this status is going to be removed automatically, or changed to other status. This field can be used to express lifecycle related information.
  - Constraints: servers MAY restrict possibility to set or update this value by the client.

9.3.1. Status Values

Status values that can be added or removed by a client are prefixed with "client". Corresponding status values that can be added or removed by a server are prefixed with "server". Status values that do not begin with either "client" or "server" are server-managed. Status values that begin with "rgp" are related to the Redemption Grace Period (RGP) and are also server-managed. A client MUST NOT alter status values set by the server. A server MAY alter or override status values set by a client, subject to local server policies.

The Set By column identifies the entity that adds and removes each status value: `client`, `server`, or `client, server` for a value that the server manages and that a client can also manage where server policy permits. A server MUST assign at least one status value to the object. A client is not required to provide a status value when creating or updating the object.

Status Value	Set By	Description
clientDeleteProhibited	client	Requests to delete the object MUST be rejected.
serverDeleteProhibited	server	Requests to delete the object MUST be rejected.
clientHold	client	DNS delegation information MUST NOT be published for the object.
serverHold	server	DNS delegation information MUST NOT be published for the object.

Status Value	Set By	Description
clientRenewProhibited	client	Requests to renew the object MUST be rejected.
serverRenewProhibited	server	Requests to renew the object MUST be rejected.
clientTransferProhibited	client	Requests to transfer the object MUST be rejected.
serverTransferProhibited	server	Requests to transfer the object MUST be rejected.
clientUpdateProhibited	client	Requests to update the object (other than to remove this status) MUST be rejected.
serverUpdateProhibited	server	Requests to update the object (other than to remove this status) MUST be rejected.
inactive	server	Delegation information has not been associated with the object. This is the default status when a domain object is first created and there are no associated host objects for the DNS delegation. This status can also be set by the server when all host-object associations are removed.
ok	server	This is the normal status value for an object that has no pending operations or prohibitions. This value is set and removed by the server as other status values are added or removed.
pendingCreate	server	A transform command has been processed for the object, but the action has not been completed by the server. Server operators can delay action completion for a variety of reasons, such as to allow for human review or third-party action.
pendingDelete	server	A transform command has been processed for the object, but the action has not been completed by the server. Server operators can delay action completion for a variety of reasons, such as to allow for human review or third-party action.
pendingRenew	server	A transform command has been processed for the object, but the action has not been completed by the server. Server operators can delay action completion for a variety of reasons, such as to allow for human review or third-party action.

Status Value	Set By	Description
pendingTransfer	server	A transform command has been processed for the object, but the action has not been completed by the server. Server operators can delay action completion for a variety of reasons, such as to allow for human review or third-party action.
pendingUpdate	server	A transform command has been processed for the object, but the action has not been completed by the server. Server operators can delay action completion for a variety of reasons, such as to allow for human review or third-party action.

Table 1: Domain Name Status Values

9.3.2. Allowed Transitions

The following diagram describes the allowed status transitions for a domain object, excluding the "rgp" status values, which follow the separate Redemption Grace Period State Diagram below. The `clientHold`, `serverHold`, `clientDeleteProhibited`, `serverDeleteProhibited`, `clientRenewProhibited`, `serverRenewProhibited`, `clientTransferProhibited`, `serverTransferProhibited`, `clientUpdateProhibited`, and `serverUpdateProhibited` status values are additive flags that MAY be combined with the states shown below (subject to the Status Exclusions rules) and are not depicted as separate states; while present, the corresponding flag blocks the associated transition (delete, renew, transfer, or update, respectively) from being initiated.

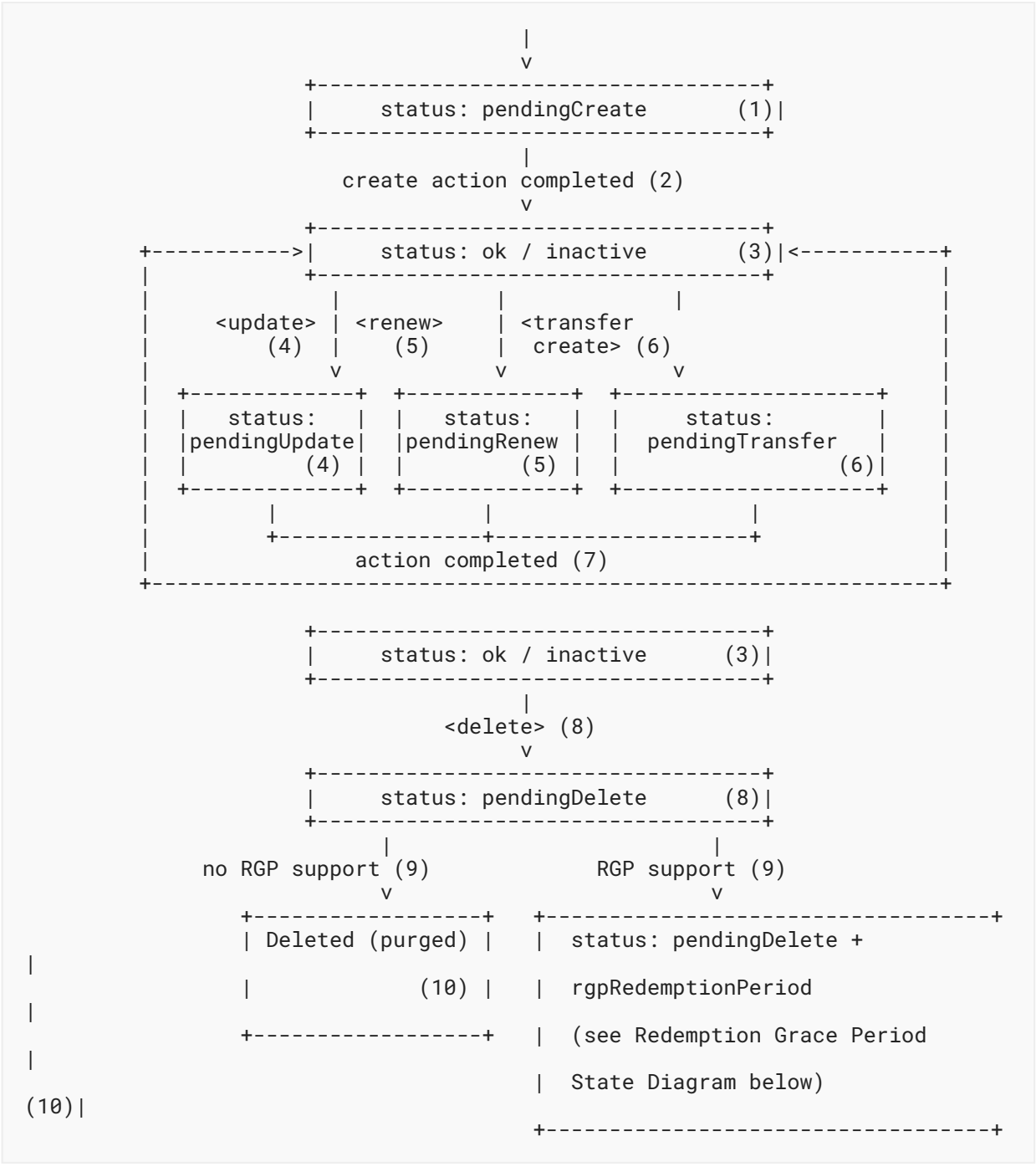


Figure 4: Domain Object State Diagram

State descriptions:

1. A create operation is received and processed. If the server defers completion, the object enters `pendingCreate` state.

2. The create action completes. The object transitions to `ok` if one or more nameservers are associated with the domain, or to `inactive` if none are associated.
3. The object is in normal operation, either as `ok` (nameservers associated) or `inactive` (no nameservers associated). The object toggles between these two sub-states as nameserver associations are added to or removed from the domain, independently of the transitions described below.
4. An update operation is received. If the server defers completion, the object enters `pendingUpdate` state.
5. A renew operation is received to extend the domain's registration period. If the server defers completion, the object enters `pendingRenew` state.
6. A transfer Create operation is received, initiating a transfer request. The object enters `pendingTransfer` state while the request awaits approval, rejection, or automated server action.
7. Once the pending update, renew, or transfer action completes, the corresponding pending status is removed and the object returns to its `ok` or `inactive` state, as determined by its current nameserver associations.
8. A delete operation is received and processed. The object enters `pendingDelete` state.
9. Whether RGP is supported for the domain determines the next transition upon completion of the pending delete action.
10. If RGP is not supported, the object is purged once the delete action completes. If RGP is supported, the object instead continues into the `rgpRedemptionPeriod` state, following the Redemption Grace Period State Diagram below.

9.3.3. Pending Status Removal

When the requested action has been completed, the pendingCreate, pendingDelete, pendingRenew, pendingTransfer, or pendingUpdate status value MUST be removed. All clients involved in the transaction MUST be notified using a service message that the action has been completed and that the status of the object has changed.

9.3.4. Status Exclusions

Status Value	MUST NOT be combined with
ok	Any other status
pendingDelete	clientDeleteProhibited, serverDeleteProhibited
pendingRenew	clientRenewProhibited, serverRenewProhibited
pendingTransfer	clientTransferProhibited, serverTransferProhibited

Status Value	MUST NOT be combined with
pendingUpdate	clientUpdateProhibited, serverUpdateProhibited
pendingCreate, pendingDelete, pendingRenew, pendingTransfer, pendingUpdate	Each other (mutually exclusive)

Table 2: Status Exclusions

Other status combinations not expressly prohibited MAY be used.

9.3.5. Redemption Grace Period

Status values that begin with "rgp" are related to the Redemption Grace Period (RGP) and are server-managed.

The Set By column identifies the entity that adds and removes each status value: `client`, `server`, or `client, server` for a value that the server manages and that a client can also manage where server policy permits.

Status Value	Set By	Description
rgpAddPeriod	server	This grace period is provided after the initial registration of a domain name. If the domain name is deleted by the registrar during this period, the registry provides a credit to the registrar for the cost of the registration.
rgpAutoRenewPeriod	server	This grace period is provided after a domain name registration period expires and is extended (renewed) automatically by the registry. If the domain name is deleted by the registrar during this period, the registry provides a credit to the registrar for the cost of the renewal.
rgpRenewPeriod	server	This grace period is provided after a domain name registration period is explicitly extended (renewed) by the registrar. If the domain name is deleted by the registrar during this period, the registry provides a credit to the registrar for the cost of the renewal.
rgpTransferPeriod	server	This grace period is provided after the successful transfer of domain name registration sponsorship from one registrar to another registrar. If the domain name is deleted by the new sponsoring registrar during this period, the registry provides a credit to the registrar for the cost of the transfer.

Status Value	Set By	Description
rgpRedemptionPeriod	server	This status value is used to describe a domain for which a delete operation has been received, but the domain has not yet been purged because an opportunity exists to restore the domain and abort the deletion process.
rgpPendingRestore	server	This status value is used to describe a domain that is in the process of being restored after being in the rgpRedemptionPeriod state.
rgpPendingDelete	server	This status value is used to describe a domain that has entered the purge processing state after completing the rgpRedemptionPeriod state. A domain in this status MUST also be in the pendingDelete status described above.

Table 3: Redemption Grace Period Status Values

9.3.5.1. Allowed Transitions

The following state diagram describes the object lifecycle when the Redemption Grace Period (RGP) feature is supported. It adapts the diagram from Section 2 of [RFC3915] to the RPP data model, using RPP status labels and operations instead of EPP command names.

In the diagram below, each state box shows the values of the `status` data element of the Domain Name Data Object. The values `ok` and `pendingDelete` are defined in Section 9.3.1, and the values beginning with "rgp" are defined in Section 9.3.5. The `<delete>` label refers to the Delete operation on the Domain Name Data Object. The `<create>` and `<update>` labels refer to the Create operation (Section 10.2.1.1) and the Update operation (Section 10.2.1.3) of the Restore Process Object.

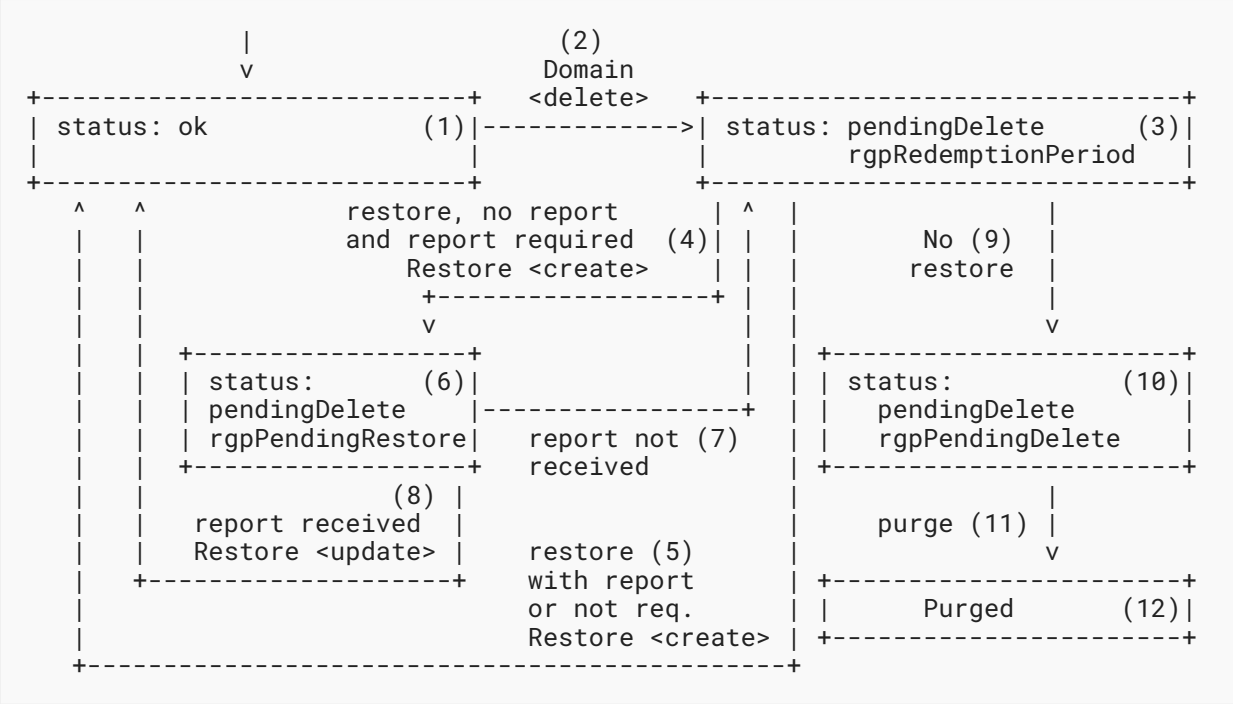


Figure 5: Redemption Grace Period State Diagram

State descriptions:

1. The object is in normal operation (`ok` or other status allowing a delete operation).
2. A delete operation is received and processed.
3. RGP begins. The object enters `pendingDelete` + `rgpRedemptionPeriod` state. The object remains here until a restore operation is requested or the redemption period elapses.
4. A restore Create operation is submitted. The registry accepts the request. Go to step 9 if the redemption period elapses before a restore is received (4a).
5. If the server does not require a restore report, the object returns to its pre-deletion status (1) immediately upon a successful Create operation. If the server requires a report but the client includes it inline in the Create operation, the server processes both atomically and the object transitions directly from `rgpRedemptionPeriod` to its pre-deletion status (1), bypassing the `rgpPendingRestore` state.
6. The object enters `pendingDelete` + `rgpPendingRestore` state. The registry awaits a restore report from the sponsoring client.
7. If no restore report is received within the registry-defined time, the object returns to `rgpRedemptionPeriod` state (step 3).
8. If a restore report is received and accepted, the object returns to its pre-deletion status and all RGP status labels are removed.
9. The redemption period elapses without a restore request being received.
10. The object enters `pendingDelete` + `rgpPendingDelete` state and awaits final purge processing.

11. The pending delete period elapses and the object is purged.
12. The object is purged and available for re-registration.

> TBD: Idea - model status object as Labelled Composition using "Label"? Con: Generic Constraints for Label will be repeated.

9.4. Host Status Object

- Name: Host Status Object
- Identifier: hostStatus
- Description: Represents one of the status values associated with a Host Data Object
- Data Elements:
  - Label
    - Identifier: label
    - Cardinality: 1
    - Mutability: create-only
    - Data Type: String
    - Description: machine-readable enum label of a status
    - Constraints:
      - Only status values defined in Status Values are allowed.
  - Reason
    - Identifier: reason
    - Cardinality: 0-1
    - Mutability: create-only
    - Data Type: String
    - Description: a human-readable text that describes the rationale for the status applied to the object.
    - Constraints: None

9.4.1. Status Values

Status values that can be added or removed by a client are prefixed with "client". Corresponding status values that can be added or removed by a server are prefixed with "server". Status values that do not begin with either "client" or "server" are server-managed. A client MUST NOT alter status values set by the server. A server MAY alter or override status values set by a client, subject to local server policies.

The Set By column identifies the entity that adds and removes each status value: `client`, `server`, or `client, server` for a value that the server manages and that a client can also manage where server policy permits. A server MUST assign at least one status value to the object. A client is not required to provide a status value when creating or updating the object.

Status Value	Set By	Description
clientDeleteProhibited	client	Requests to delete the object MUST be rejected.
serverDeleteProhibited	server	Requests to delete the object MUST be rejected.
clientUpdateProhibited	client	Requests to update the object (other than to remove this status) MUST be rejected.
serverUpdateProhibited	server	Requests to update the object (other than to remove this status) MUST be rejected.
linked	server	The host object has at least one active association with another object, such as a domain object. Servers SHOULD provide services to determine existing object associations.
ok	server	This is the normal status value for an object that has no pending operations or prohibitions. This value is set and removed by the server as other status values are added or removed.
pendingCreate	server	A transform command has been processed for the object, but the action has not been completed by the server. Server operators can delay action completion for a variety of reasons, such as to allow for human review or third-party action.
pendingDelete	server	A transform command has been processed for the object, but the action has not been completed by the server. Server operators can delay action completion for a variety of reasons, such as to allow for human review or third-party action.
pendingTransfer	server	A transform command has been processed for the object's superordinate domain object (i.e. a transfer of the domain object is pending), but the action has not been completed by the server. Server operators can delay action completion for a variety of reasons, such as to allow for human review or third-party action.

Status Value	Set By	Description
pendingUpdate	server	A transform command has been processed for the object, but the action has not been completed by the server. Server operators can delay action completion for a variety of reasons, such as to allow for human review or third-party action.

Table 4: Host Status Values

9.4.2. Allowed Transitions

The following diagram describes the allowed status transitions for a host object, based on the status values defined in Section 2.3 of [RFC5732]. The `clientDeleteProhibited`, `serverDeleteProhibited`, `clientUpdateProhibited`, and `serverUpdateProhibited` status values are additive flags that MAY be combined with the states shown below (subject to the Status Exclusions rules) and are not depicted as separate states; while present, the corresponding flag blocks the associated transition (delete or update, respectively) from being initiated. The `linked` status is set and removed by the server as associations with other objects are added or removed, independently of the transitions described below.

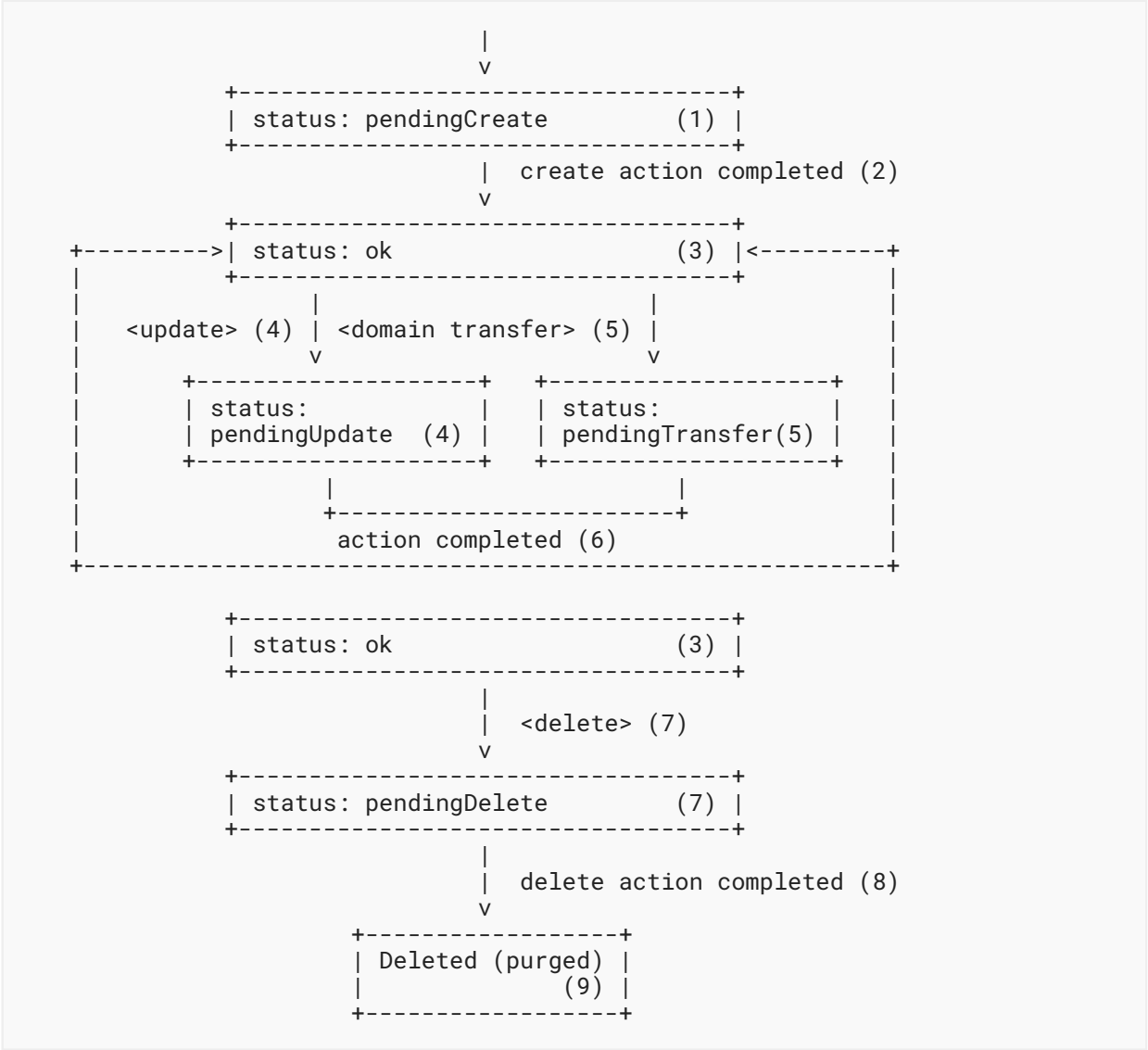


Figure 6: Host Object State Diagram

State descriptions:

1. A create operation is received and processed. If the server defers completion, the object enters `pendingCreate` state.
2. The create action completes and the `pendingCreate` status is removed. The object enters the `ok` state.
3. The object is in normal operation with the `ok` status. The `linked` status MAY accompany `ok` while the host has an active association with another object, such as a domain object.
4. An update operation is received. If the server defers completion, the object enters `pendingUpdate` state.

5. A transfer of the superordinate domain object is initiated. The server sets the `pendingTransfer` status on the subordinate host object while the domain transfer request is pending. This status is not set by a client.
6. Once the pending update or transfer action completes, the corresponding pending status is removed and the object returns to its `ok` state.
7. A delete operation is received and processed. The object enters `pendingDelete` state.
8. The pending delete action completes.
9. The object is purged.

9.4.3. Pending Status Removal

When the requested action has been completed, the pendingCreate, pendingDelete, pendingTransfer, or pendingUpdate status value MUST be removed. All clients involved in the transaction MUST be notified using a service message that the action has been completed and that the status of the object has changed.

9.4.4. Status Exclusions

Status Value	MUST NOT be combined with
ok	Any status other than `linked`
pendingDelete	clientDeleteProhibited, serverDeleteProhibited
pendingUpdate	clientUpdateProhibited, serverUpdateProhibited
pendingCreate, pendingDelete, pendingTransfer, pendingUpdate	Each other (mutually exclusive)

Table 5: Host Status Exclusions

The `linked` status MAY be combined with any status. Other status combinations not expressly prohibited MAY be used.

9.5. Contact Status Object

- Name: Contact Status Object
- Identifier: contactStatus
- Description: Represents one of the status values associated with a Contact Data Object
- Data Elements:
  - Label
    - Identifier: label
    - Cardinality: 1
    - Mutability: create-only
    - Data Type: String

        - Description: machine-readable enum label of a status
        - Constraints:
            - Only status values defined in Status Values are allowed.

    - Reason
        - Identifier: reason
        - Cardinality: 0-1
        - Mutability: create-only
        - Data Type: String
        - Description: a human-readable text that describes the rationale for the status applied to the object.
        - Constraints: None

9.5.1. Status Values

Status values that can be added or removed by a client are prefixed with "client". Corresponding status values that can be added or removed by a server are prefixed with "server". Status values that do not begin with either "client" or "server" are server-managed. A client MUST NOT alter status values set by the server. A server MAY alter or override status values set by a client, subject to local server policies.

The Set By column identifies the entity that adds and removes each status value: `client`, `server`, or `client, server` for a value that the server manages and that a client can also manage where server policy permits. A server MUST assign at least one status value to the object. A client is not required to provide a status value when creating or updating the object.

Status Value	Set By	Description
clientDeleteProhibited	client	Requests to delete the object MUST be rejected.
serverDeleteProhibited	server	Requests to delete the object MUST be rejected.
clientTransferProhibited	client	Requests to transfer the object MUST be rejected.
serverTransferProhibited	server	Requests to transfer the object MUST be rejected.
clientUpdateProhibited	client	Requests to update the object (other than to remove this status) MUST be rejected.
serverUpdateProhibited	server	Requests to update the object (other than to remove this status) MUST be rejected.
linked	server	The contact object has at least one active association with another object, such as a domain object. Servers SHOULD provide services to determine existing object associations.

Status Value	Set By	Description
ok	server	This is the normal status value for an object that has no pending operations or prohibitions. This value is set and removed by the server as other status values are added or removed.
pendingCreate	server	A transform command has been processed for the object, but the action has not been completed by the server. Server operators can delay action completion for a variety of reasons, such as to allow for human review or third-party action.
pendingDelete	server	A transform command has been processed for the object, but the action has not been completed by the server. Server operators can delay action completion for a variety of reasons, such as to allow for human review or third-party action.
pendingTransfer	server	A transform command has been processed for the object, but the action has not been completed by the server. Server operators can delay action completion for a variety of reasons, such as to allow for human review or third-party action.
pendingUpdate	server	A transform command has been processed for the object, but the action has not been completed by the server. Server operators can delay action completion for a variety of reasons, such as to allow for human review or third-party action.

Table 6: Contact Status Values

9.5.2. Allowed Transitions

The following diagram describes the allowed status transitions for a contact object, based on the status values defined in Section 2.2 of [RFC5733]. The `clientDeleteProhibited`, `serverDeleteProhibited`, `clientTransferProhibited`, `serverTransferProhibited`, `clientUpdateProhibited`, and `serverUpdateProhibited` status values are additive flags that MAY be combined with the states shown below (subject to the Status Exclusions rules) and are not depicted as separate states; while present, the corresponding flag blocks the associated transition (delete, transfer, or update, respectively) from being initiated. The `linked` status is set and removed by the server as associations with other objects are added or removed, independently of the transitions described below.

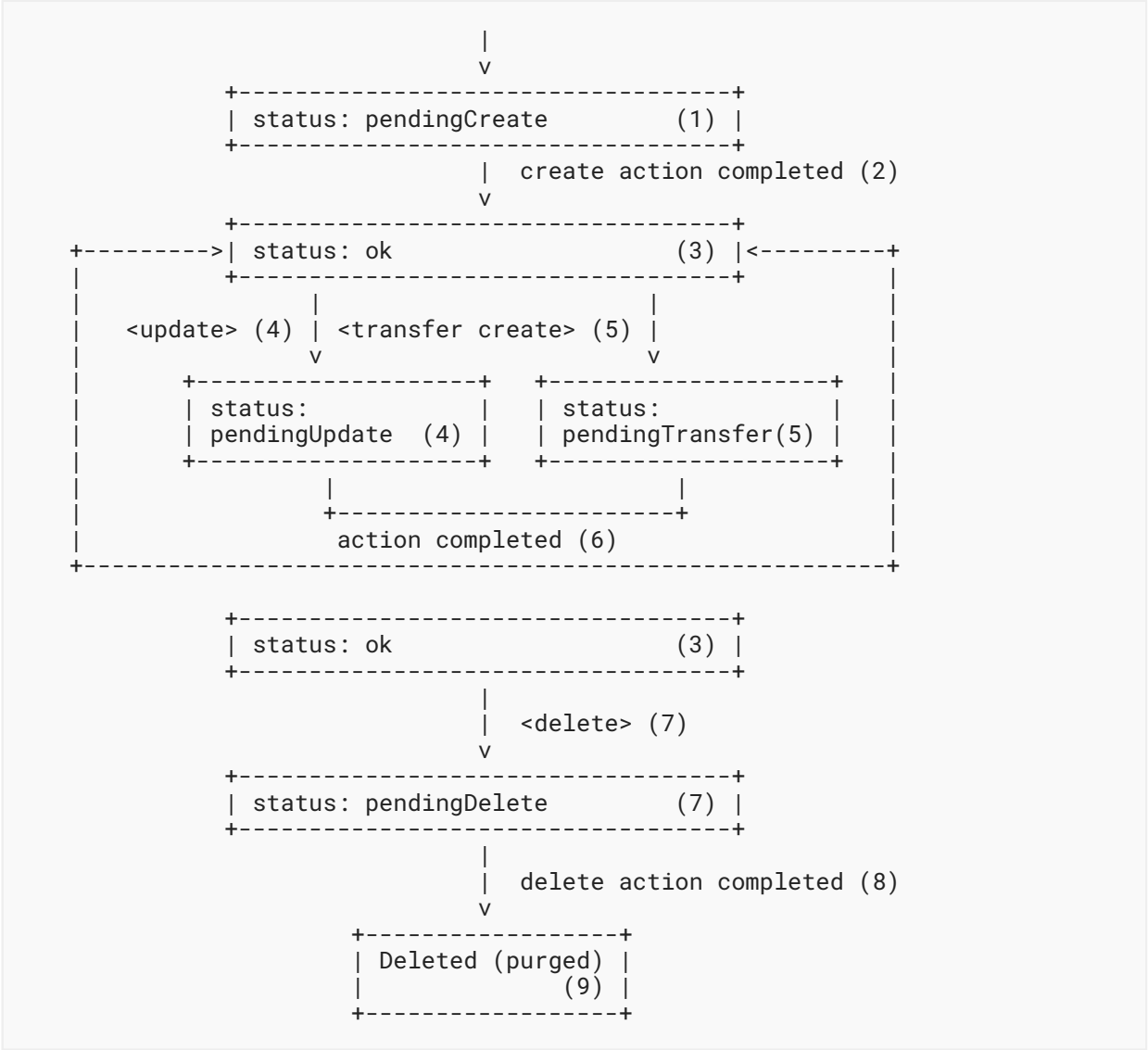


Figure 7: Contact Data Object State Diagram

State descriptions:

1. A create operation is received and processed. If the server defers completion, the object enters `pendingCreate` state.
2. The create action completes and the `pendingCreate` status is removed. The object enters the `ok` state.
3. The object is in normal operation with the `ok` status. The `linked` status MAY accompany `ok` while the contact has an active association with another object, such as a domain object.
4. An update operation is received. If the server defers completion, the object enters `pendingUpdate` state.

5. A transfer Create operation is received for the contact, initiating a transfer request. The object enters `pendingTransfer` state while the request awaits approval, rejection, cancellation, or automated server action.
6. Once the pending update or transfer action completes, the corresponding pending status is removed and the object returns to its `ok` state. A transfer that is approved changes the sponsoring client of the contact.
7. A delete operation is received and processed. The object enters `pendingDelete` state.
8. The pending delete action completes.
9. The object is purged.

9.5.3. Pending Status Removal

When the requested action has been completed, the pendingCreate, pendingDelete, pendingTransfer, or pendingUpdate status value MUST be removed. All clients involved in the transaction MUST be notified using a service message that the action has been completed and that the status of the object has changed.

9.5.4. Status Exclusions

Status Value	MUST NOT be combined with
ok	Any status other than `linked`
pendingDelete	clientDeleteProhibited, serverDeleteProhibited
pendingTransfer	clientTransferProhibited, serverTransferProhibited
pendingUpdate	clientUpdateProhibited, serverUpdateProhibited
pendingCreate, pendingDelete, pendingTransfer, pendingUpdate	Each other (mutually exclusive)

Table 7: Contact Status Exclusions

The `linked` status MAY be combined with any status. Other status combinations not expressly prohibited MAY be used.

9.6. Organisation Role Status Object

- Name: Organisation Role Status Object
- Identifier: organisationRoleStatus
- Description: Represents one of the status values associated with an Organisation Role Object. A role SHOULD have at least one associated status value.

- Data Elements:
  - Label
    - Identifier: label
    - Cardinality: 1
    - Mutability: create-only
    - Data Type: String
    - Description: machine-readable enum label of a status
    - Constraints:
      - Only status values defined in Status Values are allowed.
  - Reason
    - Identifier: reason
    - Cardinality: 0-1
    - Mutability: create-only
    - Data Type: String
    - Description: a human-readable text that describes the rationale for the status applied to the role.
    - Constraints: None

9.6.1. Status Values

Status values that can be added or removed by a client are prefixed with "client". Corresponding status values that can be added or removed by a server are prefixed with "server". Status values that do not begin with either "client" or "server" are server-managed.

The Set By column identifies the entity that adds and removes each status value: `client`, `server`, or `client, server` for a value that the server manages and that a client can also manage where server policy permits.

Status Value	Set By	Description
clientLinkProhibited	client	Requests to add new links to the role MUST be rejected.
serverLinkProhibited	server	Requests to add new links to the role MUST be rejected.
linked	server	The role has at least one active association with another object. This value is set and removed by the server and is not explicitly set by the client.
ok	server	This is the normal status value for a role that has no active prohibitions. This value is set and removed by the server as other status values are added or removed.

Table 8: Organisation Role Status Values

9.6.2. Allowed Transitions

The following diagram describes the allowed status transitions for an organisation role. The `ok` status is mutually exclusive with `clientLinkProhibited` and `serverLinkProhibited`, and the `linked` status MAY be combined with either `ok` or the link prohibition statuses (subject to the Status Exclusions rules). The `linked` status is set and removed by the server as associations with other objects are added or removed, independently of link prohibition changes, so the transitions in the top row apply equally to the bottom row. The `clientLinkProhibited` and `serverLinkProhibited` status values MAY be set at the same time.

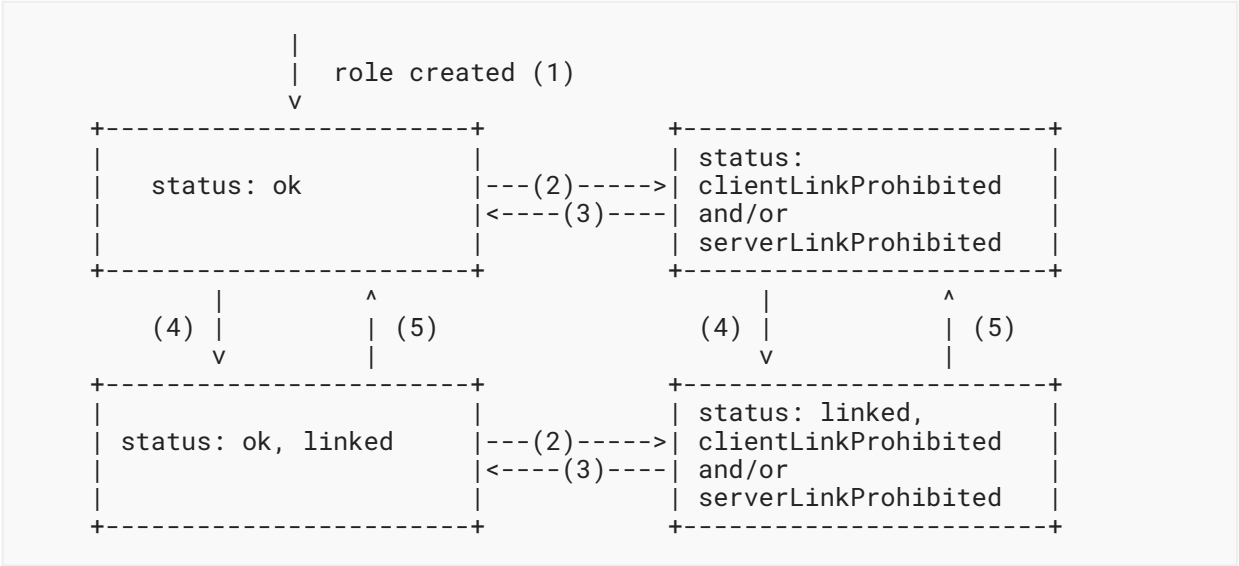


Figure 8: Organisation Role State Diagram

State descriptions:

1. A role is added to an organisation. The role starts with the `ok` status.
2. A `clientLinkProhibited` or `serverLinkProhibited` status is set, by the client or the server respectively, and the `ok` status is removed. Requests to add new links to the role MUST be rejected while a link prohibition is present.
3. The last link prohibition status is removed and the server sets the `ok` status.
4. The role gets its first active association with another object and the server sets the `linked` status.
5. The last active association of the role is removed and the server removes the `linked` status.

9.6.3. Status Exclusions

Status Value	MUST NOT be combined with
ok	Any status other than `linked`

Table 9: Organisation Role Status Exclusions

Other status combinations not expressly prohibited MAY be used.

9.7. Organisation Status Object

- Name: Organisation Status Object
- Identifier: organisationStatus
- Description: Represents one of the status values associated with an Organisation Data Object, as defined in Section 3.4 of [RFC8543]. An Organisation Data Object MUST always have at least one associated status value.
- Data Elements:
  - Label
    - Identifier: label
    - Cardinality: 1
    - Mutability: create-only
    - Data Type: String
    - Description: machine-readable enum label of a status
    - Constraints:
      - Only status values defined in Status Values are allowed.
  - Reason
    - Identifier: reason
    - Cardinality: 0-1
    - Mutability: create-only
    - Data Type: String
    - Description: a human-readable text that describes the rationale for the status applied to the object.
    - Constraints: None

9.7.1. Status Values

Status values that can be added or removed by a client are prefixed with "client". Corresponding status values that can be added or removed by a server are prefixed with "server". The `hold` and `terminated` status values are server-managed when the organisation has no parent organisation and otherwise MAY be client-managed based on server policy. Other status values that do not begin with either "client" or "server" are server-managed.

The Set By column identifies the entity that adds and removes each status value: `client`, `server`, or `client, server` for a value that the server manages and that a client can also manage where server policy permits. A server MUST assign at least one status value to the object. A client is not required to provide a status value when creating or updating the object.

Status Value	Set By	Description
ok	server	This is the normal status value for an object that has no pending operations or prohibitions. This value is set and removed by the server as other status values are added or removed.
hold	client, server	Organisation transform commands and new links MUST be rejected.
terminated	client, server	The organisation has been terminated and MUST NOT be linked. Organisation transform commands and new links MUST be rejected.
linked	server	The organisation object has at least one active association with another object. This value is not explicitly set by the client. Servers SHOULD provide services to determine existing object associations.
clientLinkProhibited	client	Requests to add new links to the organisation MUST be rejected.
serverLinkProhibited	server	Requests to add new links to the organisation MUST be rejected.
clientUpdateProhibited	client	Requests to update the object (other than to remove this status) MUST be rejected.
serverUpdateProhibited	server	Requests to update the object (other than to remove this status) MUST be rejected.
clientDeleteProhibited	client	Requests to delete the object MUST be rejected.
serverDeleteProhibited	server	Requests to delete the object MUST be rejected.
pendingCreate	server	A transform command has been processed for the object, but the action has not been completed by the server. Server operators can delay action completion for a variety of reasons, such as to allow for human review or third-party action.
pendingUpdate	server	A transform command has been processed for the object, but the action has not been completed by the server. Server operators can delay action completion for a variety of reasons, such as to allow for human review or third-party action.

Status Value	Set By	Description
pendingDelete	server	A transform command has been processed for the object, but the action has not been completed by the server. Server operators can delay action completion for a variety of reasons, such as to allow for human review or third-party action.

Table 10: Organisation Status Values

9.7.2. Allowed Transitions

The following diagram describes the allowed status transitions for an organisation object. The `linked`, `clientLinkProhibited`, `serverLinkProhibited`, `clientUpdateProhibited`, `serverUpdateProhibited`, `clientDeleteProhibited`, and `serverDeleteProhibited` status values are additive flags that MAY be combined with the states shown below (subject to the Status Exclusions rules) and are not depicted as separate states; while present, the corresponding flag blocks the associated transition (new link, update, or delete, respectively) from being initiated. The `linked` status is set and removed by the server as associations with other objects are added or removed, independently of the transitions described below.

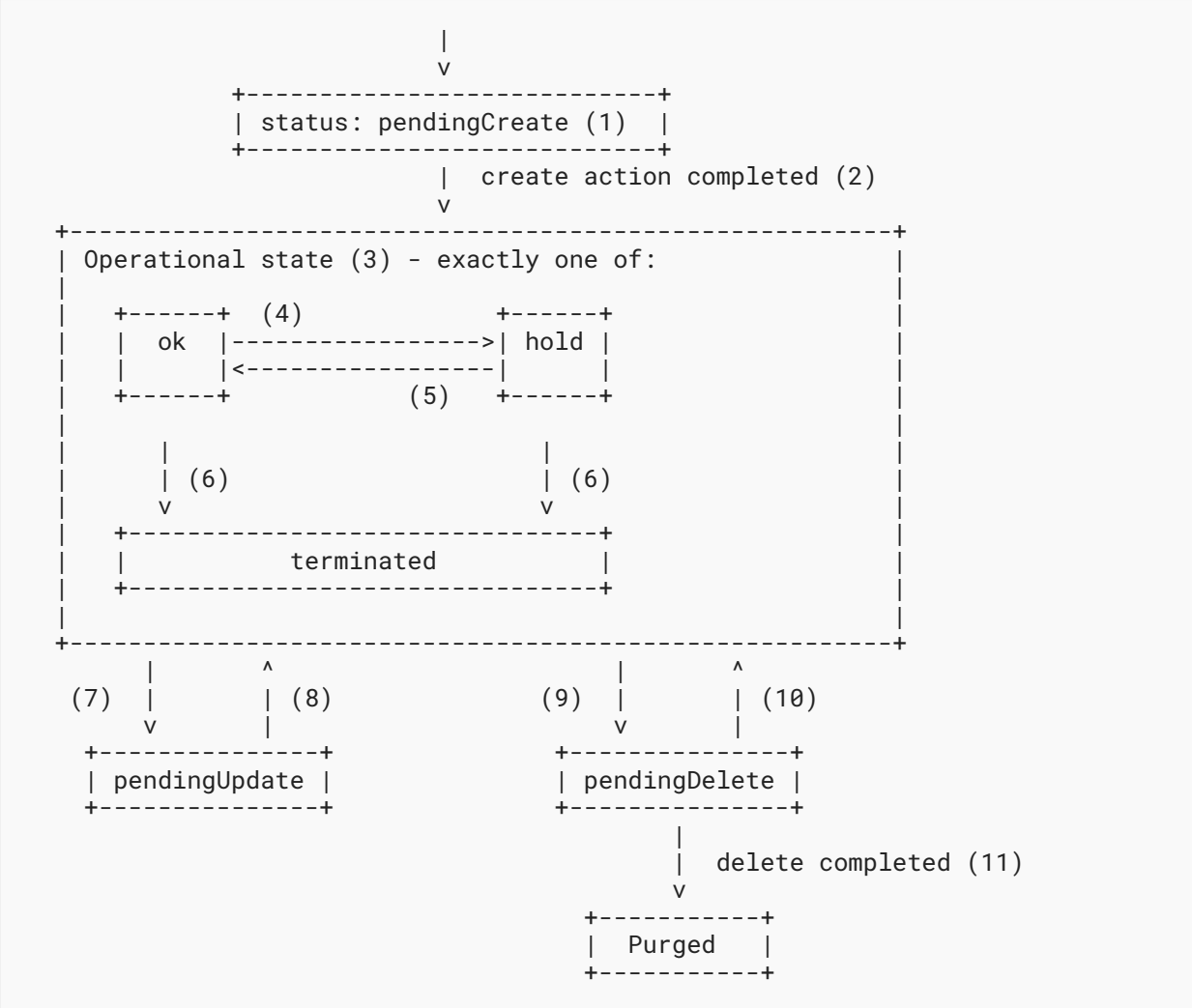


Figure 9: Organisation Object State Diagram

State descriptions:

1. A create operation is received and processed. If the server defers completion, the object enters `pendingCreate` state. If the server completes the create immediately, the object enters the `ok` state directly.
2. The create action completes and the `pendingCreate` status is removed. The object enters the operational state (3), normally `ok`.
3. The object is in exactly one of the mutually exclusive operational states `ok`, `hold`, or `terminated`. The pending states (7) and (9) are entered from, and return to, this operational state.
4. A `hold` status is set. Organisation transform operations and new links MUST be rejected while the object is in `hold`. The `hold` status is set by the server, or by the client if the organisation has a parent organisation and server policy allows it.

5. The `hold` status is removed and the object returns to `ok`, by the same parties that are allowed to set it.
6. A `terminated` status is set, from either `ok` or `hold`, by the same parties that are allowed to set `hold`. A terminated organisation MUST NOT be linked, and organisation transform operations and new links MUST be rejected. Whether and how an object can leave the `terminated` state is subject to server policy.
7. An update operation is received. If the server defers completion, the object enters `pendingUpdate` state.
8. The pending update completes, the `pendingUpdate` status is removed, and the object returns to its operational state.
9. A delete operation is received. If the server defers completion, the object enters `pendingDelete` state.
10. The pending delete is rejected by the server, the `pendingDelete` status is removed, and the object returns to its operational state.
11. The pending delete completes and the object is purged.

9.7.3. Pending Status Removal

When the requested action has been completed, the pendingCreate, pendingDelete, or pendingUpdate status value MUST be removed. All clients involved in the transaction MUST be notified using a service message that the action has been completed and that the status of the object has changed.

9.7.4. Status Exclusions

Status Value	MUST NOT be combined with
pendingCreate, ok, hold, terminated	Each other (mutually exclusive); an organisation MUST have exactly one of these statuses
ok	Any status other than `linked`
pendingDelete	clientDeleteProhibited, serverDeleteProhibited
pendingCreate, pendingDelete, pendingUpdate	Each other (mutually exclusive)

Table 11: Organisation Status Exclusions

The `linked` status MAY be combined with `clientLinkProhibited` or `serverLinkProhibited` if new links must be prohibited.

Other status combinations not expressly prohibited MAY be used.

9.8. User Status Object

- Name: User Status Object
- Identifier: userStatus

- Description: Represents the status value associated with a User Data Object. A User Data Object MUST always have exactly one associated status value.
- Data Elements:
  - Label
    - Identifier: label
    - Cardinality: 1
    - Mutability: create-only
    - Data Type: String
    - Description: machine-readable enum label of a status
    - Constraints:
      - Only status values defined in Status Values are allowed.
  - Reason
    - Identifier: reason
    - Cardinality: 0-1
    - Mutability: create-only
    - Data Type: String
    - Description: a human-readable text that describes the rationale for the status applied to the object.
    - Constraints: None

9.8.1. Status Values

The `pending` status value is server-managed. The `suspended` and `deactivated` status values are server-managed and MAY additionally be client-managed by a client with permission to manage users in the owning organisation, based on server policy. The `active` status value is set and removed by the server as other status values are added or removed.

The Set By column identifies the entity that adds and removes each status value: `client`, `server`, or `client, server` for a value that the server manages and that a client can also manage where server policy permits. A server MUST assign at least one status value to the object. A client is not required to provide a status value when creating or updating the object.

Status Value	Set By	Description
active	server	This is the normal status value for a user that is permitted to authenticate and act within the permissions of its roles.
pending	server	The user has been created, but activation has not been completed by the server. Server operators can delay activation for a variety of reasons, such as to allow for verification of the user's details or human review. Requests authenticated as the user MUST be rejected.

Status Value	Set By	Description
suspended	client, server	The user has been temporarily suspended. Requests authenticated as the user MUST be rejected.
deactivated	client, server	The user has been deactivated. Requests authenticated as the user MUST be rejected.

Table 12: User Status Values

9.8.2. Allowed Transitions

The following diagram describes the allowed status transitions for a user object.

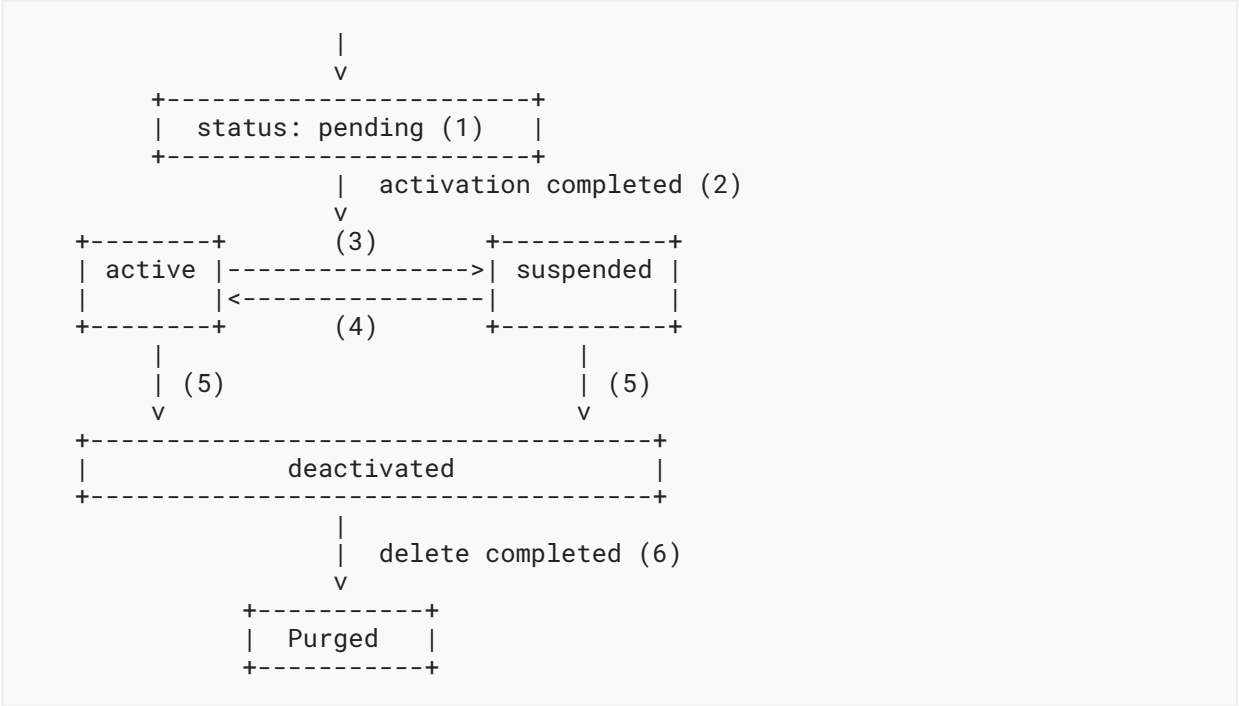


Figure 10: User Object State Diagram

State descriptions:

1. A create operation is received and processed. If the server defers activation, the object enters the `pending` state. If the server activates the user immediately, the object enters the `active` state directly.
2. Activation completes and the `pending` status is replaced by `active`.
3. A `suspended` status is set, replacing `active`.
4. The `suspended` status is removed and the object returns to `active`, by the same parties that are allowed to set it.

5. A `deactivated` status is set, from either `active` or `suspended`. Whether and how an object can leave the `deactivated` state is subject to server policy.
6. A delete operation completes and the object is purged. A delete operation can be received in any state; only the transition from `deactivated` is depicted.

9.8.3. Pending Status Removal

When activation has been completed, the `pending` status value MUST be replaced by `active`. The client that created the user MUST be notified using a service message that activation has been completed and that the status of the object has changed.

9.8.4. Status Exclusions

Status Value	MUST NOT be combined with
active, pending, suspended, deactivated	Each other (mutually exclusive); a user MUST have exactly one of these statuses

Table 13: User Status Exclusions

9.9. DNS Resource Record Object

- Name: DNS Resource Record Object
- Identifier: dnsRecord
- Description: Represents a single DNS resource record. The structure follows the top-level format of a DNS resource record as described in Section 3.2.1 of [RFC1035], adapted for use in provisioning. The RDATA field is represented as a structured object whose fields depend on the record type, following the RDATA presentation format described by the corresponding RFC defining the record type. This approach is aligned with [I-D.simmen-rpp-dns-data].
- Data Elements:
  - Name
    - Identifier: name
    - Cardinality: 1
    - Mutability: read-write
    - Data Type: String
    - Description: The owner name of the DNS entry. This MAY be the domain itself or a subordinate host name.
    - Constraints:
      - The value MUST be a syntactically valid DNS host name.
      - Absolute FQDNs (with trailing dot) and relative host names are allowed, as well as the "@" symbol representing the domain name itself.
  - Class
    - Identifier: class
    - Cardinality: 0-1
    - Mutability: read-write

- Data Type: String
- Description: The DNS resource record class.
- Constraints:
  - If present, the value MUST be chosen from Section 3.2.4 (CLASS values) of [RFC1035].
  - A client SHOULD omit this element. The server MUST assume "IN" as the default class and MAY decline other values.

- Type
  - Identifier: type
  - Cardinality: 1
  - Mutability: read-write
  - Data Type: String
  - Description: The DNS resource record type, indicating the format of the RDATA field.
  - Constraints:
    - The value MUST be a valid string representation of a resource record type as defined in [RFC1035] or other RFC describing the record type.
    - Allowed values MAY be constrained by server policies. For domain provisioning, the type would typically be constrained to the allowed parent-side entries.
    - Values MUST be converted to lower case.
    - In EPP Compatibility Profile ([RFC5732]), the following record types MUST be supported: `ns`, `a`, and `aaaa`.
    - In EPP Compatibility Profile with DNSSEC Extension [RFC5910], the following record types MUST additionally be supported: `ds` and `dnskey`.

- RDATA
  - Identifier: rdata
  - Cardinality: 1
  - Mutability: read-write
  - Data Type: Object
  - Description: The actual payload data of the DNS record. The structure of this object depends on the record type and MUST follow the RDATA presentation format described by the corresponding RFC. Property names MUST be written in camelCase. All property values MUST be represented as Strings encoding the presentation format of the value.
  - Constraints:
    - The fields within RDATA MUST match the expected structure for the given record type.
    - See Section 9.9.1 for required structures in the EPP Compatibility Profile.

9.9.1. RDATA Structures in EPP Profile

This section defines the RDATA field structures required for interoperability in the EPP Compatibility Profile.

In EPP Compatibility Profile ([RFC5732]), the following RDATA structures MUST be supported:

- For NS records ([RFC1035], Section 3.3.11): `nsdname` (the fully qualified domain name of the name server).
- For A records ([RFC1035], Section 3.4.1): `address` (the IPv4 address in dotted-decimal notation).
- For AAAA records ([RFC3596], Section 2.2): `address` (the IPv6 address in text representation as defined in [RFC5952]).

In EPP Compatibility Profile with DNSSEC Extension ([RFC5910]), the following RDATA structures MUST additionally be supported:

- For DS records ([RFC4034], Section 5): `keyTag` (key tag value), `algorithm` (algorithm number), `digestType` (digest algorithm type), and `digest` (digest value).
- For DNSKEY records ([RFC4034], Section 2): `flags` (flags field value), `protocol` (protocol field value), `algorithm` (algorithm number), and `publicKey` (encoded public key value).

> TBC: Optional keyData inside dsData (RFC 5910 Section 4.1): In the DS Data Interface, a DS record MAY optionally contain a nested keyData element used for server-side validation of the DS hash. The draft doesn't describe this pattern - a client submitting a DS record with accompanying DNSKEY for validation.

9.10. DNS Operational Controls Object

- Name: DNS Operational Controls Object
- Identifier: dnsControls
- Description: Contains operational control parameters that a client MAY use to influence server-side DNS behaviour for a set of DNS records. A server MAY ignore these values, e.g. for policy reasons. This structure is aligned with [I-D.simmen-rpp-dns-data].
- Data Elements:
  - TTL
    - Identifier: ttl
    - Cardinality: 0-1
    - Mutability: read-write
    - Data Type: Dictionary[Integer]
    - Description: Controls the caching behaviour of DNS resource records. The dictionary is keyed by lower-case record type name, with values representing the TTL in seconds for that record type.
    - Constraints:
      - The keys MUST be valid DNS resource record type names in lower case.
      - The values MUST be positive Integers.
      - The allowed value ranges MAY be constrained by server policy.
      - A server MAY ignore client-specified TTL values and apply default or policy-defined values.

- Maximum Signature Lifetime
  - Identifier: maxSigLifetime
  - Cardinality: 0-1
  - Mutability: read-write
  - Data Type: Dictionary[Integer]
  - Description: Specifies a child's preference for the maximum number of seconds after signature generation when the parent's signature on signed DNS information should expire. The dictionary is keyed by lower-case record type name, with values representing the maximum signature lifetime in seconds for that record type. The value applies to the RRSIG resource record over the signed DNS RRset. See Section 3 of [RFC4034] for information on the RRSIG resource record. This corresponds to the maxSigLife concept from [RFC5910].
  - Constraints:
    - The keys MUST be valid DNS resource record type names in lower case.
    - The values MUST be positive Integers (minimum value of 1).
    - A server MAY ignore client-specified values and apply its own default signature expiration policy.

9.11. DNS Data Object

- Name: DNS Data Object
- Identifier: dnsData
- Description: A container for DNS resource records and associated operational controls for a provisioned object. This structure groups DNS records together with control parameters that influence server-side DNS behaviour. The structure is aligned with [I-D.simmen-rpp-dns-data].
- Data Elements:
  - Records
    - Identifier: records
    - Cardinality: 0+
    - Mutability: read-write
    - Data Type: DNS Resource Record Object
    - Description: An array of DNS resource records associated with the provisioned object.
    - Constraints:
      - Allowed record types MAY be constrained by server policy.
      - In EPP Compatibility Profile with DNSSEC Extension [RFC5910], records of type DS and DNSKEY MUST be supported in addition to NS, A, and AAAA.
      - A server MUST support either DS or DNSKEY or both record types for DNSSEC provisioning. If provided with only DNSKEY, a server MUST calculate the DS record. If both record types are provided, a server MAY use the DNSKEY to validate the DS record.

- Controls
  - Identifier: controls
  - Cardinality: 0-1
  - Mutability: read-write
  - Data Type: DNS Operational Controls Object
  - Description: Operational control parameters for the DNS records.
  - Constraints: (None)

9.12. Authorisation Information Object

- Name: Authorisation Information Object
- Identifier: authInfo
- Description: Contains information used to authorise operations on a data object. It may hold different kinds of authorisation information.
- Data Elements:
  - Method
    - Identifier: method
    - Cardinality: 1
    - Mutability: create-only
    - Data Type: String
    - Description: The identifier of the RPP authorisation method.
    - Constraints:
      - The value MUST be one of the values registered at IANA as defined in [I-D.draft-wullink-rpp-core].
      - In EPP Compatibility Profile this value MUST be set to `authinfo` if standard password-based authorisation is used
  - Authorisation Information
    - Identifier: authdata
    - Cardinality: 1
    - Mutability: create-only
    - Data Type: String
    - Description: The value of the authorisation information. It might be as simple as a password string, but also more complex values like public key certificates or tokens encoded as string are possible.
    - Constraints:
      - Authorisation Information object is immutable. If the information changes (for example password is updated) a new instance MUST be created.
      - Depending on the method and server policy Authorisation Information MAY not be available for read or any other operation responding with this data element.

9.13. Disclose Object

> TODO: Model Disclose in universal (extendible) way

- Name: Disclose Object
- Identifier: disclose
- Description: TBD

9.14. Restore Report Object

- Name: Restore Report Object
- Identifier: restoreReport
- Description: Contains the redemption grace period restore report submitted by the sponsoring client as required by the RGP process ([RFC3915]). A restore report documents the state of the object before and after deletion, provides the reason for restoration, and includes mandatory client statements. This object is OPTIONAL and is only required when the RGP feature is supported and a restore report is being submitted.
- Data Elements:
  - Pre-Delete Data
    - Identifier: preData
    - Cardinality: 0-1
    - Mutability: read-write
    - Data Type: String
    - Description: A copy of the registration data that existed for the object prior to the object being deleted.
    - Constraints: None.
  - Post-Restore Data
    - Identifier: postData
    - Cardinality: 0-1
    - Mutability: read-write
    - Data Type: String
    - Description: A copy of the registration data that exists for the object at the time the restore report is submitted.
    - Constraints: None.
  - Delete Time
    - Identifier: deleteTime
    - Cardinality: 0-1
    - Mutability: read-write
    - Data Type: Timestamp

        - Description: The date and time when the object delete request was sent to the server.
        - Constraints: None.

    - Restore Time
        - Identifier: restoreTime
        - Cardinality: 0-1
        - Mutability: read-write
        - Data Type: Timestamp
        - Description: The date and time when the original restore request operation was sent to the server.
        - Constraints:
            - This element MAY be omitted when the restore report is submitted inline within the restore request in a single-step process.
            - In EPP Compatibility Profile this element MUST be present as defined in [RFC3915].

    - Restore Reason
        - Identifier: restoreReason
        - Cardinality: 0-1
        - Mutability: read-write
        - Data Type: String
        - Description: A brief explanation of the reason for restoring the object.
        - Constraints: None.

    - Statements
        - Identifier: statements
        - Cardinality: 1-2
        - Mutability: read-write
        - Data Type: String
        - Description: Client statements required by the RGP process.
        - Constraints: In EPP Compatibility Profile exactly two statements MUST be present as defined in [RFC3915].

    - Other
        - Identifier: other
        - Cardinality: 0-1
        - Mutability: read-write
        - Data Type: String
        - Description: Any additional information needed to support the statements provided by the client.
        - Constraints: None.

9.15. Organisation Role Object

- Name: Organisation Role Object
- Identifier: organisationRole
- Description: Represents a role that an organisation has within the registry ecosystem. An organisation object MUST always have at least one associated role. A single organisation MAY have multiple roles with different role types.
- Data Elements:
  - Role Identifier
    - Identifier: roleId
    - Cardinality: 0-1
    - Mutability: read-write
    - Data Type: String
    - Description: A third-party-assigned identifier for the role, such as an IANA ID for registrars.
    - Constraints: (None)
  - Role Status
    - Identifier: status
    - Cardinality: 0+
    - Mutability: read-write
    - Data Type: Organisation Role Status Object
    - Description: The status of this particular role. A role SHOULD have at least one associated status value.
    - Constraints:
      - Possible values and combinations are specified in Organisation Role Status Values.

> TBC: IANA registry for role types and statuses? must be compat with EPP

9.16. Processes Object

- Name: Processes Object
- Identifier: processes
- Description: A container grouping the Process Objects Section 10 currently or recently initiated on a Data Object. Each data element MUST be named after the identifier of a Process Object and MUST be of type Aggregation of that Process Object. The data elements below are the process types defined in this document. Extensions MUST add further data elements for the Process Objects they define.
- Data Elements:
  - Transfer Processes
    - Identifier: transferProcess

        - Cardinality: 0+
        - Mutability: read-only
        - Data Type: Aggregation[Transfer Process Object]
        - Direct Access: true
        - Description: The transfer processes initiated on the owning Data Object.
        - Constraints: (None)

    - Renew Processes
        - Identifier: renewProcess
        - Cardinality: 0+
        - Mutability: read-only
        - Data Type: Aggregation[Renew Process Object]
        - Direct Access: true
        - Description: The renew processes initiated on the owning Data Object.
        - Constraints: (None)

    - Restore Processes
        - Identifier: restoreProcess
        - Cardinality: 0+
        - Mutability: read-only
        - Data Type: Aggregation[Restore Process Object]
        - Direct Access: true
        - Description: The restore processes initiated on the owning Data Object.
        - Constraints: (None)

    - Create Processes
        - Identifier: createProcess
        - Cardinality: 0+
        - Mutability: read-only
        - Data Type: Aggregation[Create Process Object]
        - Direct Access: true
        - Description: The create processes initiated on the owning Data Object.
        - Constraints: (None)

9.17. Base Process Object

- Name: Base Process Object
- Identifier: baseProcess
- Abstract: true
- Description: Base structure for all Process Objects, containing the data elements common to every process. Each Process Object extends this object.

- Data Elements:
  - Process ID
    - Identifier: processId
    - Cardinality: 0-1
    - Mutability: read-only
    - Data Type: String
    - Description: A server-assigned identifier of the process instance, unique within the scope of the Owner Data Object.
    - Constraints: The value is set by the server and cannot be specified by the client.

9.18. Messages

The Message Components define the structure and data elements for all message objects used in the protocol. Each specific message type extends the Base Message Object to include additional data elements relevant to that message type. Any of the types defined in this section may be used for the `data` data element defined for the Message Data Object.

9.18.1. Base Message Object

- Name: Base Message Object
- Identifier: baseMessage
- Abstract: true
- Description: Base structure for all message objects, containing common data elements such as the object type and object identifier.
- Data Elements:
  - Object Type
    - Identifier: objectType
    - Cardinality: 0-1
    - Mutability: read-only
    - Data Type: String
    - Description: The object identifier of the object affected by the requested action (e.g., "domainName", "host", "contact").
    - Constraints:
      - Must only be used when the `objectId` field is present.
  - Object Identifier
    - Identifier: objectId
    - Cardinality: 0-1
    - Mutability: read-only
    - Data Type: String
    - Description: The unique identifier of the object affected by the requested operation.

        - Constraints:
            - Must be a valid unique identifier for the object type specified in the `objectType` field.
            - For both "domainName", "host" the `objectId` must be a valid fully qualified domain name (FQDN).
            - For "contact" the `objectId` must be a valid contact identifier.
            - When the message is not related to a specific object, the `objectId` field may be empty.

9.18.2. Review Response Object

The Review Response Object defines the data elements for returning the result of an offline-review. Offline-review is optional for the create, update, delete, transfer, renew and restore actions, where the server may want to perform an offline review before completing the requested action.

- Name: Review Response Object
- Identifier: reviewResponseMessage
- Extends: Base Message Object
- Description: Notification of the result of an offline-review of a requested operation.
- Data Elements:
    - Approval Result
        - Identifier: approved
        - Cardinality: 1
        - Mutability: read-only
        - Data Type: Boolean
        - Description: Whether the requested action was approved and completed.
        - Constraints: `false` means the action was denied and not performed.
    - Client Transaction ID
        - Identifier: clientTransactionId
        - Cardinality: 0-1
        - Mutability: read-only
        - Data Type: String
        - Description: The client transaction identifier from the original request, if supplied.
        - Constraints: (None)
    - Server Transaction ID
        - Identifier: serverTransactionId
        - Cardinality: 1
        - Mutability: read-only
        - Data Type: String
        - Description: The server transaction identifier returned for the original request.

        - Constraints: (None)

    - Completion Date
        - Identifier: completionDate
        - Cardinality: 1
        - Mutability: read-only
        - Data Type: Timestamp
        - Description: The date and time when review of the requested action was completed.
        - Constraints: (None)

    - Message
        - Identifier: message
        - Cardinality: 1
        - Mutability: read-only
        - Data Type: String
        - Description: Human-readable message associated with the review response.
        - Constraints: (None)

    - Process
        - Identifier: process
        - Cardinality: 0-1
        - Mutability: read-only
        - Data Type: Base Process Object
        - Description: The process whose operation was subject to the offline review.
        - Constraints: Present when the reviewed operation was performed through a Process Object (create, renew, transfer, or restore) and the server exposes that Process Object; absent otherwise, including for update and delete operations.

9.18.3. Transfer Request Message Object

- Name: Transfer Request Message Object
- Identifier: transferRequestMessage
- Extends: Base Message Object
- Description: Notification for losing client of started transfer request, where the client expected to act on the pending transfer request.
- Data Elements:
    - Transfer Status
        - Identifier: status
        - Cardinality: 1
        - Mutability: read-only
        - Data Type: String
        - Description: The current state of the transfer request.
        - Constraints: The value MUST be `pending`.

- Request Date
  - Identifier: requestDate
  - Cardinality: 1
  - Mutability: read-only
  - Data Type: Timestamp
  - Description: The date and time the transfer was requested.
  - Constraints: (None)
- Action Date
  - Identifier: actionDate
  - Cardinality: 0-1
  - Mutability: read-only
  - Data Type: Timestamp
  - Description: The deadline for responding before the server takes an automated action.
  - Constraints:
    - The action date MUST be later than the request date and is subject to server policy.
- Process
  - Identifier: process
  - Cardinality: 0-1
  - Mutability: read-only
  - Data Type: Transfer Process Object
  - Description: The transfer process the message pertains to.
  - Constraints: Absent if the server does not expose the Transfer Process Object.

9.18.4. Transfer Outcome Message Object

- Name: Transfer Outcome Message Object
- Identifier: transferOutcomeMessage
- Extends: Base Message Object
- Description: Notification for a client affected by the completion of a transfer request.
- Data Elements:
  - Transfer Status
    - Identifier: trStatus
    - Cardinality: 1
    - Mutability: read-only
    - Data Type: String
    - Description: The final state of the transfer request.
    - Constraints: The value MUST be one of `clientApproved`, `clientCancelled`, `clientRejected`, `serverApproved`, or `serverCancelled`.

- Request Date
  - Identifier: requestDate
  - Cardinality: 1
  - Mutability: read-only
  - Data Type: Timestamp
  - Description: The date and time the transfer was requested.
  - Constraints: (None)
- Action Date
  - Identifier: actionDate
  - Cardinality: 1
  - Mutability: read-only
  - Data Type: Timestamp
  - Description: The date and time the transfer request was resolved.
  - Constraints: (None)
- Expiry Date
  - Identifier: expiryDate
  - Cardinality: 0-1
  - Mutability: read-only
  - Data Type: Timestamp
  - Description: The end of the object's validity period if the operation causes a change in the validity period.
  - Constraints: (None)
- Process
  - Identifier: process
  - Cardinality: 0-1
  - Mutability: read-only
  - Data Type: Transfer Process Object
  - Description: The transfer process the message pertains to.
  - Constraints: Absent if the server does not expose the Transfer Process Object.

9.18.5. Expiration Deletion Message Object

- Name: Expiration Deletion Message Object
- Identifier: expirationDeletionMessage
- Extends: Base Message Object
- Description: Notification for the sponsoring client that an object was deleted following expiration.
- Data Elements:
  - Deletion Date
    - Identifier: deletionDate

        - Cardinality: 1
        - Mutability: read-only
        - Data Type: Timestamp
        - Description: The date and time the server deleted the object.
        - Constraints: (None)

9.18.6. Auto-Renewal Message Object

- Name: Auto-Renewal Message Object
- Identifier: autoRenewalMessage
- Extends: Base Message Object
- Description: Notification for the sponsoring client that the server automatically renewed an object.
- Data Elements:
  - Renewal Date
    - Identifier: renewalDate
    - Cardinality: 1
    - Mutability: read-only
    - Data Type: Timestamp
    - Description: The date and time the server renewed the object.
    - Constraints: (None)
  - Expiry Date
    - Identifier: expiryDate
    - Cardinality: 1
    - Mutability: read-only
    - Data Type: Timestamp
    - Description: The new expiration date of the object.
    - Constraints: (None)

9.18.7. Service Notice Message Object

The Service Notice Message Object is not linked to a specific object or process and can be used to convey general information to the client.

- Name: Service Notice Message Object
- Identifier: serviceNoticeMessage
- Extends: Base Message Object
- Description: Generic text based message containing information not covered by any other message type.
- Data Elements:
  - Message
    - Identifier: message

        - Cardinality: 1
        - Mutability: read-only
        - Data Type: String
        - Description: Human-readable service information for the receiving organisation.
        - Constraints:
            - The server MUST NOT misuse this field by using it for transmitting structured data or machine-readable information.

TODO Complete the Service Notice Message Object definition, do we need it at all?

9.19. Discovery

9.19.1. Version Object

- Name: Discovery Version Object
- Identifier: discoveryVersion
- Description: Describes a version of RPP supported by the server.
- Data Elements:
    - Version
        - Identifier: version
        - Cardinality: 1
        - Mutability: read-only
        - Data Type: String
        - Description: The RPP version, for example "1.0".
        - Constraints: The value MUST use the Semantic Versioning format.
    - Base URL
        - Identifier: baseUrl
        - Cardinality: 1
        - Mutability: read-only
        - Data Type: URL
        - Description: The base URL specific for the version, used as a prefix for all endpoint URL templates.
        - Constraints: (None)

9.19.2. Environment Object

- Name: Discovery Environment Object
- Identifier: discoveryEnvironment
- Description: Describes the environment of the RPP server.
- Data Elements:
    - Name
        - Identifier: name
        - Cardinality: 1

        - Mutability: read-only
        - Data Type: String
        - Description: The name of the RPP server, for example "rpp.example".
        - Constraints: (None)

    - Status
        - Identifier: status
        - Cardinality: 1
        - Mutability: read-only
        - Data Type: String
        - Description: The operational status of the RPP server, for example "production", "test", or "development".
        - Constraints: (None)

9.19.3. Extension Object

- Name: Extension Object
- Identifier: extension
- Description: Describes an extension supported by the server.
- Data Elements:
    - Name
        - Identifier: name
        - Cardinality: 1
        - Mutability: read-only
        - Data Type: String
        - Description: A short name for the extension, for example "registry fee extension".
        - Constraints: (None)

    - Extension ID
        - Identifier: id
        - Cardinality: 1
        - Mutability: read-only
        - Data Type: String
        - Description: A unique URN identifier for the extension, for example "urn:ietf:params:rpp:extension:registry-fee".
        - Constraints: (None)

    - Version
        - Identifier: version
        - Cardinality: 1
        - Mutability: read-only
        - Data Type: String

        - Description: The version of the extension that is supported or required for the profile, for example "1.0".
        - Constraints: (None)

    - URL
        - Identifier: url
        - Cardinality: 1
        - Mutability: read-only
        - Data Type: URL
        - Description: The location of the extension specification.
        - Constraints:
            - For standard extensions, the value MUST be the URL of the extension in the IANA registry.
            - For private extensions, the value MUST be the URL of the extension specification. The domain name used in the URL MUST resolve and the URL MUST be accessible to the client.

9.19.4. IDN LGR Object

- Name: Discovery IDN LGR Object
- Identifier: discoveryIdnLgr
- Description: Describes a Label Generation Ruleset (LGR) supported by the server for internationalized domain names.
- Data Elements:
    - TLDs
        - Identifier: tld
        - Cardinality: 1+
        - Mutability: read-only
        - Data Type: String
        - Description: The top-level domains (TLDs) to which the LGR applies.
        - Constraints: (None)

    - Name
        - Identifier: name
        - Cardinality: 1
        - Mutability: read-only
        - Data Type: String
        - Description: The IANA-registered name of the LGR.
        - Constraints: The value MUST be a valid LGR name as listed in the IANA IDN Tables registry [IDN-Tables].

    - URL
        - Identifier: url

        - Cardinality: 1
        - Mutability: read-only
        - Data Type: URL
        - Description: The location of the LGR specification in the IANA IDN Tables registry [IDN-Tables].
        - Constraints: (None)

    - Default
        - Identifier: default
        - Cardinality: 1
        - Mutability: read-only
        - Data Type: Boolean
        - Description: Indicates whether this LGR is the default for the specified TLDs.
        - Constraints: (None)

9.19.5. Notice Object

- Name: Discovery Notice Object
- Identifier: discoveryNotice
- Description: Describes a notice from the server operator.
- Data Elements:
    - Publication Time
        - Identifier: pubTime
        - Cardinality: 1
        - Mutability: read-only
        - Data Type: Timestamp
        - Description: The publication time of the notice.
        - Constraints: (None)

    - Text
        - Identifier: text
        - Cardinality: 1
        - Mutability: read-only
        - Data Type: String
        - Description: A human-readable description of the notice.
        - Constraints: (None)

10. Process Objects

This section defines the Process Objects used in this document.

Each Process Object extends the Base Process Object Section 9.17 and therefore carries its OPTIONAL Process ID data element.

10.1. Transfer Process Object

- Name: Transfer Process Object
- Identifier: transferProcess
- Unique Identifier: processId
- Extends: Base Process Object
- Description: Represents a transfer request for a provisioned object. Creating this object initiates a transfer. The object supports approve and reject as additional operations, and delete as the cancel operation. Reading the object returns the current transfer status.
- Data Elements:
  - Transfer Direction
    - Identifier: transferDir
    - Cardinality: 0-1
    - Mutability: create-only
    - Data Type: String
    - Description: Indicates whether the transfer is a "pull" or "push" transfer. Per policy, servers usually support only one model. If omitted, server policy determines the default.
    - Constraints: The value MUST be one of: "pull" (initiated by the gaining client) or "push" (initiated by the sponsoring client).
  - Gaining Client ID
    - Identifier: gainingClientId
    - Cardinality: 0-1
    - Mutability: create-only
    - Data Type: Client Identifier
    - Description: The identifier of the designated gaining client. This element is REQUIRED for push transfers and MUST NOT be provided for pull transfers.
    - Constraints: (None)
  - Transfer Status
    - Identifier: trStatus
    - Cardinality: 1
    - Mutability: read-only
    - Data Type: String
    - Description: The state of the transfer request.
    - Constraints: The value MUST be one of: "pending", "clientApproved", "clientCancelled", "clientRejected", "serverApproved", "serverCancelled".
  - Requesting Client ID
    - Identifier: reqClientId

        - Cardinality: 1
        - Mutability: read-only
        - Data Type: Client Identifier
        - Description: The identifier of the client that initiated the transfer request.
        - Constraints: (None)

    - Request Date
        - Identifier: requestDate
        - Cardinality: 1
        - Mutability: read-only
        - Data Type: Timestamp
        - Description: The date and time that the transfer was requested.
        - Constraints: (None)

    - Acting Client ID
        - Identifier: actClientId
        - Cardinality: 1
        - Mutability: read-only
        - Data Type: Client Identifier
        - Description: For a pending pull transfer, the identifier of the sponsoring client that SHOULD act upon the request. For a pending push transfer, the identifier of the designated gaining client that SHOULD act upon the request. For all other statuses, the identifier of the client that took the indicated action.
        - Constraints: (None)

    - Action Date
        - Identifier: actionDate
        - Cardinality: 1
        - Mutability: read-only
        - Data Type: Timestamp
        - Description: For a pending request, the date and time by which a response is required before an automated response action SHOULD be taken by the server. For all other statuses, the date and time when the request was completed.
        - Constraints: (None)

10.1.1. Operations

10.1.1.1. Create (Transfer Request)

- Identifier: transferCreate

The Create operation initiates a transfer by creating a Transfer Process Object associated with the provisioned object. The transfer direction and gaining client (for push transfers) are provided as `create-only` data elements of the Transfer Process Object.

- Authorisation:
  - Pull transfer:
    - The requesting client (gaining client) MUST provide valid Object Authorisation.
  - Push transfer:
    - Only the sponsoring client is authorised to initiate a push transfer.
- Input:
  - Reference: Owner Data Object
  - Object: Transfer Process Object
- Output:
  - Object: Transfer Process Object

10.1.1.2. Read (Transfer Query)

- Identifier: transferRead

The Read operation allows a client to determine the real-time status of a pending or recently completed transfer request.

- Authorisation:
  - This operation MUST be accessible to both the sponsoring client and the gaining client.
  - Server policy determines whether other clients may query transfer status and what information is returned.
- Input:
  - Reference: This Object
- Output:
  - Object: Transfer Process Object

10.1.1.3. Delete (Transfer Cancel)

- Identifier: transferDelete

The Delete operation allows the initiating client to cancel its own pending transfer request.

- Authorisation:
  - Only the initiating client (the client that originally requested the transfer).
- Input:
  - Reference: This Object

- Output:
  - Object: Transfer Process Object
    - Cardinality: 0-1

10.1.1.4. Approve

- Identifier: transferApprove

The Approve operation allows the appropriate client to accept a pending transfer request.

- Authorisation:
  - Pull transfer:
    - only the sponsoring client
  - Push transfer:
    - only the designated gaining client
- Input:
  - Reference: This Object
- Output:
  - Object: Transfer Process Object

10.1.1.5. Reject

- Identifier: transferReject

The Reject operation allows the appropriate client to decline a pending transfer request.

- Authorisation:
  - Pull transfer:
    - only the sponsoring client
  - Push transfer:
    - only the designated gaining client
- Input:
  - Reference: This Object
  - Input Data Elements:
    - Reason
      - Identifier: reason
      - Cardinality: 0-1
      - Data Type: String
      - Description: A human-readable text describing the rationale for rejecting the transfer request.
      - Constraints: In EPP Compatibility Profile this data element MUST NOT be used

- Output:
  - Object: Transfer Process Object

10.2. Restore Process Object

- Name: Restore Process Object
- Identifier: restoreProcess
- Unique Identifier: processId
- Extends: Base Process Object
- Description: Represents the current state of a restore request for an object that has entered the Redemption Grace Period (RGP).
- Data Elements:
  - Restore Status
    - Identifier: restoreStatus
    - Cardinality: 1
    - Mutability: read-only
    - Data Type: String
    - Description: The current state of the restore process.
    - Constraints: The value MUST be one of: `"pendingRestore"`, `"restored"`, `"rgpPendingDelete"`
  - Request Date
    - Identifier: requestDate
    - Cardinality: 0-1
    - Mutability: read-only
    - Data Type: Timestamp
    - Description: The date and time when the restore request was submitted.
    - Constraints: MUST NOT be present if no restore request has been submitted yet.
  - Report Date
    - Identifier: reportDate
    - Cardinality: 0-1
    - Mutability: read-only
    - Data Type: Timestamp
    - Description: The date and time when the most recent restore report was accepted by the server.
    - Constraints: MUST NOT be present if no restore report has been accepted yet.
  - Report Due Date
    - Identifier: reportDueDate
    - Cardinality: 0-1
    - Mutability: read-only

        - Data Type: Timestamp
        - Description: The date and time by which a restore report must be submitted before the object reverts to `rgpRedemptionPeriod` state. Only present when the object is in `rgpPendingRestore` state.
        - Constraints: MUST NOT be present when `restoreStatus` is not `"pendingRestore"`.

    - Restore Report
        - Identifier: restoreReport
        - Cardinality: 0-1
        - Mutability: read-write
        - Data Type: Restore Report Object
        - Visibility: onDemand
        - Description: The restore report submitted by the sponsoring client as required by the RGP process. A client MAY provide it in the Create operation, so that the restore request and the report are processed atomically, or in a later Update operation while the object is in `rgpPendingRestore` state. A report provided in an Update operation replaces any previously submitted report.
        - Constraints:
            - MUST NOT be provided if the server does not require a restore report.
            - In EPP Compatibility Profile, the restore report is write-only: the server does not return it in the output of any operation, because [RFC3915] provides no means to retrieve a submitted report.
            - In EPP Compatibility Profile, providing this element in the Create operation corresponds to `op="report"`, omitting it corresponds to `op="request"`, and providing it in the Update operation corresponds to `op="report"` as defined in [RFC3915].

10.2.1. Operations

10.2.1.1. Create (Restore Request)

- Identifier: create

The Create operation initiates the recovery of an object in the `rgpRedemptionPeriod` state by creating a Restore Process Object. The server MUST reject this operation if the owning object is not in the `rgpRedemptionPeriod` state.

If the server requires a restore report and `restoreReport` is absent, the object transitions to `rgpPendingRestore` state awaiting a subsequent Update operation. Otherwise the restore is completed immediately.

- Authorisation:
    - Only the sponsoring client is authorised to perform this operation.

- Input:
  - Reference: Owner Data Object
  - Object: Restore Process Object
  - Input Data Elements:
    - Restore Report
      - Identifier: restoreReport
      - Cardinality: 0-1
      - Data Type: Restore Report Object
      - Description: An OPTIONAL inline restore report. If provided, the server processes the request and the report atomically. If the server does not require a restore report, this element MUST NOT be present and the restore is completed immediately. If the server requires a report and this element is absent, the object transitions to `pendingRestore` state awaiting a subsequent Report operation.
      - Constraints:
      - MUST NOT be provided if the server does not require a restore report.
      - In EPP Compatibility Profile, corresponds to `op="request"` (without report) or a combined `op="request"` followed immediately by `op="report"` as defined in [RFC3915].
- Output:
  - Object: Restore Process Object

10.2.1.2. Read (Restore Query)

- Identifier: read

The Read operation allows the sponsoring client to retrieve the current state of the RGP process for an object.

- Authorisation:
  - Only the sponsoring client is authorised to perform this operation.
- Input:
  - Reference: This Object
- Output:
  - Object: Restore Process Object

In EPP Compatibility Profile this operation is not supported.

10.2.1.3. Update (Restore Report)

- Identifier: update

This operation is OPTIONAL, only for servers which support or require submission of a restore report.

The Update operation submits the restore report required by the RGP process for an object in the `rgpPendingRestore` state. Since `restoreReport` is the only read-write data element of the Restore Process Object, the client MUST provide it in this operation. A report MAY be submitted more than once if corrections are required. The server MUST reject this operation if the object is not in the `rgpPendingRestore` state.

- Authorisation:
  - Only the sponsoring client is authorised to perform this operation.
- Input:
  - Reference: This Object
  - Input Data Elements:
    - Restore Report
      - Identifier: restoreReport
      - Cardinality: 1
      - Data Type: Restore Report Object
      - Description: The restore report to be submitted as part of the RGP process.
      - Constraints:
      - In EPP Compatibility Profile, corresponds to `op="report"` as defined in [RFC3915].
- Output:
  - Object: Restore Process Object

10.3. Renew Process Object

- Name: Renew Process Object
- Identifier: renewProcess
- Unique Identifier: processId
- Extends: Base Process Object
- Description: Represents a renew request for a provisioned object. Creating this object initiates a renewal process that extends the registration period of the object. Reading this object returns the new expiry date if the renewal has been completed.
- Data Elements:
  - Expiry Date
    - Identifier: expiryDate
    - Cardinality: 0-1
    - Mutability: create-only
    - Data Type: Timestamp

        - Description: The current expiry date of the object. The server MUST validate this against the object's current `expiryDate` to prevent unintended duplicate renewals.

    - Renewal Period
        - Identifier: renewalPeriod
        - Cardinality: 0-1
        - Mutability: create-only
        - Data Type: Period Object
        - Description: The duration to be added to the object's registration period. This value is used by the server to calculate the new `expiryDate`. The default value MAY be defined by server policy. The number of units available MAY be subject to limits imposed by the server.

10.3.1. Operations

10.3.1.1. Create (Renew Request)

- Identifier: renewCreate

The renew operation extends the validity period of an existing object by creating a Renew Process Object.

- Input:
    - Reference: Owner Data Object
    - Object: Renew Process Object
- Output:
    - Object: Renew Process Object
- Authorisation:
    - Only the sponsoring client is authorised to perform this operation.

10.4. Create Process Object

- Name: Create Process Object
- Identifier: createProcess
- Unique Identifier: processId
- Extends: Base Process Object
- Description: Represents the process initiated when a resource creation operation is performed. It carries creation-specific inputs that are consumed during the creation operation and are not stored as persistent attributes of the created resource object.
- Data Elements: (None)

Beyond the data elements of the Base Process Object, the generic Create Process Object defines no additional data elements. Individual object definitions extend it with object-specific creation inputs (such as the Domain Create Process Object Section 11.4.1, which adds the registration period).

10.4.1. Operations

10.4.1.1. Create

- Identifier: create

The Create operation is invoked implicitly as a side effect of the resource object create operation and is never invoked directly. It carries the creation-specific inputs consumed during the creation of the owning resource object.

- Input:
  - Object: Create Process Object
- Output:
  - Object: Create Process Object
- Authorisation:
  - Inherited from the resource object create operation that initiates this process.

10.4.1.2. Read

- Identifier: read

The Read operation retrieves the result or status of the creation, if the server exposes the process resource.

- Input:
  - Reference: This Object
- Output:
  - Object: Create Process Object
- Authorisation:
  - Only the sponsoring client is authorised to perform this operation.

11. Domain Name Data Object

11.1. Object Description

- Name: Domain Name Data Object
- Identifier: domainName
- Unique Identifier: name

- Description: A Domain Name data object represents a domain name and contains the data required for its provisioning and management in the registry.

11.2. Data Elements

The following data elements are defined for the Domain Name Data Object.

- Name
  - Identifier: name
  - Cardinality: 1
  - Mutability: create-only
  - Data Type: Fully Qualified Domain Name
  - Description: The fully qualified name of the domain object.
  - Constraints:
    - A server MAY restrict allowable domain names to a particular top-level domain, second-level domain, or other domain for which the server is authoritative.
    - If the domain name is an internationalized domain name (IDN), the value MUST be a valid ACE encoded string representing the Unicode Name (uName).
- Unicode Name
  - Identifier: uName
  - Cardinality: 0-1
  - Mutability: create-only
  - Data Type: String
  - Description: The fully qualified name of the domain using the Unicode character set.
  - Constraints:
    - The value MUST be a fully qualified domain name in Unicode Normalization Form C (NFC) as defined in [UNICODE.NFC].
    - The server MUST validate the Unicode Name using the procedure described in section 4.2 of [RFC5891].
    - Every code point in the value MUST be classified as a candidate for inclusion by the rules defined in [RFC5892].
    - If the value contains a code point from a script with Right-to-Left directionality, the value MUST satisfy the Bidi Rule defined in [RFC5893].
    - The Unicode (U-label) form of the domain name, when converted according to the procedures defined in Section 2.3.2.4 of [RFC5890], MUST be equal to the corresponding A-label representation.
    - Any code point valid under IDNA2003 but disallowed under IDNA2008 MUST NOT be allowed in the Unicode Name (uName).
- LGR
  - Identifier: lgr

- Cardinality: 0-1
- Mutability: create-only
- Data Type: String
- Description: The name of the Label Generation Ruleset (LGR) containing the valid code points used for the Unicode Name (uName).
- Constraints:
  - MUST NOT be empty when the Unicode Name (uName) is provided.
  - MUST be a valid LGR name registered in the IANA IDN Tables registry [IDN-Tables].

- Provisioning Metadata
  - Identifier: provMetadata
  - Cardinality: 1
  - Mutability: read-only
  - Data Type: Provisioning Metadata Object
  - Description: Standard metadata about the object's lifecycle and ownership.
  - Constraints: (None)

- Status
  - Identifier: status
  - Cardinality: 1+
  - Mutability: read-write
  - Data Type: Domain Name Status Object
  - Description: The current status descriptors associated with the domain.
  - Constraints:
    - Possible combinations of Status Object Labels are specified in Section 9.3.

- Registrant
  - Identifier: registrant
  - Cardinality: 0-1
  - Mutability: read-write
  - Data Type: Aggregation[Contact Data Object]
  - Description: The contact object associated with the domain as the registrant.
  - Constraints:
    - The relation MUST correspond to a valid Contact Data Object known to the server.
    - Servers MAY restrict association of a Contact Data Object of a different sponsoring client.

> TBC: leave registrant here or move it to contacts with a type?

- Contacts
  - Identifier: contacts

- Cardinality: 0+
- Mutability: read-write
- Data Type: LabelledAggregation[Contact Data Object]
  - Label Description: The role of the associated contact.
  - Label Constraints:
    - List of supported roles is defined by server policy
    - In the EPP Compatibility Profile, the value MUST be one of: "admin", "billing", or "tech"
- Description: A collection of other contact objects associated with the domain object.
- Constraints:
  - Each associated contact MUST correspond to a valid Contact Data Object known to the server before it can be associated with the domain.
  - Maximum number of associated contacts (per role) MAY be restricted by server policy

> TBC: IANA registry for contact role label?

- Nameservers
  - Identifier: nameservers
  - Cardinality: 0+
  - Mutability: read-write
  - Data Type: Aggregation[Host Data Object]
  - Description: A collection of nameservers associated with the domain.
  - Constraints:
    - A Host Data Object MUST be known to the server before it can be associated with the domain.
    - MUST not be used when 1 or more DNS Data entries are specified through the DNS Data element.
- DNS Data
  - Identifier: dns
  - Cardinality: 0-1
  - Mutability: read-write
  - Data Type: DNS Data Object
  - Description: DNS resource records and operational controls related to the domain name. This structure follows the unified DNS data model defined in [I-D.simmen-rpp-dns-data], providing a single container for all DNS-related provisioning data including delegation, DNSSEC, and other record types.
  - Constraints:
    - The type of the record entries MAY be constrained by the server policy. Typically the values would be limited to allowed parent-side resource record types.

        - In EPP Compatibility Profile, a server MUST support NS, A, and AAAA record types for delegation, and with DNSSEC Extension [RFC5910] a server MUST additionally support DS and/or DNSKEY record types.
        - The names of DNS entries MUST be the domain name itself or subordinate to the domain name and MUST NOT be below zone cut in case of present delegation.
        - A server MAY forbid use of NS/A/AAAA record types if the server only supports host object model through Nameserver property defining those records.
        - MUST not be used when 1 or more nameservers are specified through the Nameserver element.

- Subordinate Hosts
    - Identifier: subordinateHosts
    - Cardinality: 0+
    - Mutability: read-only
    - Data Type: Aggregation[Host Data Object]
    - Description: A collection of subordinate host objects that exist under this domain.
    - Constraints: (None)

- Expiry Date
    - Identifier: expiryDate
    - Cardinality: 0-1
    - Mutability: read-only
    - Data Type: Timestamp
    - Description: The date and time identifying the end of the domain object's registration period.
    - Constraints: The value is set by the server and cannot be specified by the client.

- Authorisation Information
    - Identifier: authInfo
    - Cardinality: 0-1
    - Mutability: read-write
    - Data Type: Authorisation Information Object
    - Visibility: onDemand
    - Description: Authorisation information associated with the domain object.
    - Constraints:
        - This element MUST only be returned if the querying client is the sponsoring client or if the client supplied valid Object Authorisation with the operation.

- Processes
    - Identifier: processes
    - Cardinality: 0-1

- Mutability: read-only
- Data Type: Processes Object
- Direct Access: true
- Description: The Process Objects currently or recently initiated on the domain object.
- Constraints: (None)

11.3. Operations

11.3.1. Create Operation

- Identifier: create

The Create operation allows a client to provision a new Domain Name resource. The operation accepts as input all create-only and read-write data elements defined for the Domain Name Data Object.

- Authorisation:
  - Generally each client is authorised to create new domain objects becoming a sponsoring client. This can be however constrained by the server policy in many ways, i.e. by applying rate limiting, billing related constraints or compliance locks.
- Input:
  - Object: This Object
  - Object: Domain Create Process Object

The Create operation implicitly initiates the Domain Create Process Object Section 11.4.1, which carries the creation-specific inputs that are consumed during creation and not persisted as part of the domain object's state.

The server MUST validate the name using the procedure described in section 4.2 of [RFC5891]. If the validation of the IDN name failed because it contained a code point not available in the specified IDN table, the server MUST return an error indicating the invalid code point. If the name does not map to the provided Unicode Name (uName), the server MUST respond with an appropriate error indicating the mismatch.

11.3.2. Read Operation

- Identifier: read

The Read operation allows a client to retrieve the data elements of a Domain Name resource. The server's response MAY vary depending on client authorisation and server policy.

- Authorisation:
  - Sponsoring client:
    - Full object

- Other client:
  - Without Object Authorisation:
    - Limited object (non-confidential properties) or operation denied
  - With Object Authorisation:
    - Full object, however some properties only authorised to the sponsoring client MAY be redacted according to server policy

- Input:
  - Reference: This Object
  - Parameters:
    - Hosts Filter
      - Identifier: hostsFilter
      - Cardinality: 0-1
      - Data Type: String
      - Description: Controls which host information is returned with the object.
      - Constraints: The value MUST be one of "all", "del" (delegated), "sub" (subordinate), or "none". The default value is "all".

11.3.3. Update Operation

- Identifier: update

The Update operation allows a client to modify the read-write data elements of an existing Domain Name resource.

- Authorisation:
  - Only sponsoring client is authorised to perform this operation

At least one change to a read-write data element MUST be provided, unless the operation is extended. The registrant and the Authorisation Information MAY be removed. Contacts, nameservers, and the registrant to be added MUST be known to the server.

11.3.4. Delete Operation

- Identifier: delete

The Delete operation allows a client to remove an existing Domain Name resource. The operation targets a specific data object identified by its name.

- Authorisation:
  - Only sponsoring client is authorised to perform this operation

The server SHOULD reject a delete request if subordinate host objects are associated with the domain name.

The error response SHOULD indicate the related subordinate host objects.

11.4. Processes

The Domain Name Data Object supports the following Process Objects Section 10 unmodified:

- Renew Process
- Restore Process
  - Constraints: OPTIONAL; available only when the RGP feature is supported.

The Domain Name Data Object supports the following Process Objects extended by an object-specific definition: * Domain Create Process Section 11.4.1 * Domain Transfer Process Section 11.4.2

11.4.1. Domain Create Process Object

- Name: Domain Create Process Object
- Identifier: domainCreateProcess
- Extends: Create Process Object
- Unique Identifier: processId
- Description: The domain-specific Create Process Object Section 10.4. It is implicitly initiated by the Domain Name Data Object create operation and carries the domain creation-specific inputs, namely the requested initial registration period, that are consumed during creation and not persisted as part of the domain object's state.
- Data Elements:
  - Period
    - Identifier: period
    - Cardinality: 0-1
    - Mutability: create-only
    - Data Type: Period Object
    - Description: The initial registration period for the domain name. This value is used by the server to calculate the initial `expiryDate` of the object.
    - Constraints: The default value MAY be defined by server policy. The number of units available MAY be subject to limits imposed by the server.

11.4.1.1. Operations

11.4.1.1.1. Create

- Identifier: create

The Create operation is invoked implicitly as a side effect of the Domain Name Data Object create operation and is never invoked directly. It carries the registration period consumed during domain creation.

- Input:
  - Object: Domain Create Process Object

- Output:
  - Object: Domain Create Process Object
- Authorisation:
  - Inherited from the Domain Name Data Object create operation that initiates this process.

11.4.1.1.2. Read

- Identifier: read

The Read operation retrieves the result or status of the domain creation, if the server exposes the process resource.

- Input:
  - Reference: This Object
- Output:
  - Object: Domain Create Process Object
- Authorisation:
  - Only the sponsoring client is authorised to perform this operation.

11.4.2. Domain Transfer Process Object

- Name: Domain Transfer Process Object
- Identifier: domainTransferProcess
- Extends: Transfer Process Object
- Unique Identifier: processId
- Description: The domain-specific Transfer Process Object Section 2.7.7. It carries the domain-specific transfer inputs and outputs — the requested transfer period and the resulting expiry date — that extend the common transfer process.
- Data Elements:
  - Expiry Date
    - Identifier: expiryDate
    - Cardinality: 0-1
    - Mutability: read-only
    - Data Type: Timestamp
    - Description: The end of the domain object's registration period if the transfer caused or causes a change in the validity period.
    - Constraints: (None)
  - Transfer Period
    - Identifier: transferPeriod
    - Cardinality: 0-1
    - Mutability: create-only

        - Data Type: Period Object
        - Description: The number of units to be added to the registration period of the domain object upon successful completion of the transfer. The number of units available MAY be subject to limits imposed by the server.
        - Constraints:
            - In EPP Compatibility Profile, Transfer Period cannot be read from the Transfer Process Object

Transfer of a Domain Name Data Object MUST implicitly transfer all Host Data Objects that are subordinate to the domain object. For example, if domain object "example.com" is transferred and host object "ns1.example.com" exists, the host object MUST be transferred as part of the "example.com" transfer process. Actions that do not preserve the relationship between a domain object and its subordinate host objects MUST be disallowed.

11.4.2.1. Operations

11.4.2.1.1. Create

- Identifier: transferCreate

The Create operation initiates a transfer of the Domain Name Data Object by creating a Domain Transfer Process Object, extending the common Transfer Create Operation Section 10.1.1.1.

- Input:
    - Reference: Owner Data Object
    - Object: Domain Transfer Process Object
- Output:
    - Object: Domain Transfer Process Object
- Authorisation: as defined in Section 10.1.1.1.

12. Contact Data Object

12.1. Object Description

- Name: Contact Data Object
- Identifier: contact
- Unique Identifier: id
- Description: A Contact Data Object represents the contact information for an individual or organisation associated with other objects.

12.2. Data Elements

The following data elements are defined for the Contact Data Object.

- Handle ID
  - Identifier: id
  - Cardinality: 1
  - Mutability: create-only
  - Data Type: Identifier
  - Description: External unique identifier of the contact object.
  - Constraints:
    - This value MUST be supported to be provided by the client.
    - Servers MAY support server-side generation of this value.
- Provisioning Metadata
  - Identifier: provMetadata
  - Cardinality: 1
  - Mutability: read-only
  - Data Type: Provisioning Metadata Object
  - Description: Standard metadata about the object's lifecycle and ownership.
  - Constraints: (None)
- Status
  - Identifier: status
  - Cardinality: 1+
  - Mutability: read-write
  - Data Type: Contact Status Object
  - Description: Status descriptors associated with the contact.
  - Constraints:
    - Possible combinations of Contact Status Labels are specified in Contact Status Values.
- Contact Information
  - Identifier: contactInfo
  - Cardinality: 1
  - Mutability: read-write
  - Data Type: External:RPP-JSContact-Profile:Card
  - Description: Contains contact information.

    - Constraints:
        - In EPP Compatibility Profile, at most one internationalised and one localised form of the contact information are provided. The internationalised form MUST be represented in a subset of UTF-8 that can be represented in the 7-bit US-ASCII character set, while the localised form MAY be represented in unrestricted UTF-8.
        - In EPP Compatibility Profile, a postal address consists of up to three street lines, a city, an OPTIONAL state or province, an OPTIONAL postal code, and a country code that is a two-character identifier as specified in ISO 3166-1.
        - In EPP Compatibility Profile, telephone numbers MUST begin with a plus sign ("+"), followed by a country code defined in ITU-T E.164, followed by a dot ("."), followed by a sequence of digits, and MAY note an extension. Email addresses MUST conform to the syntax defined in [RFC5322].

- Authorisation Information
    - Identifier: authInfo
    - Cardinality: 0-1
    - Mutability: read-write
    - Data Type: Authorisation Information Object
    - Visibility: onDemand
    - Description: Authorisation information associated with the contact object.
    - Constraints: (None)

- Disclose
    - Identifier: disclose
    - Cardinality: 0-1
    - Mutability: read-write
    - Data Type: Disclose Object
    - Description: Identifies elements that require exceptional server-operator handling to allow or restrict disclosure to third parties.
    - Constraints:
        - A server MUST reject any transaction that requests disclosure practices that do not conform to the server's announced data-collection policy.

- Processes
    - Identifier: processes
    - Cardinality: 0-1
    - Mutability: read-only
    - Data Type: Processes Object
    - Direct Access: true
    - Description: The Process Objects currently or recently initiated on the contact object.
    - Constraints: (None)

> TBC: IANA registry for statuses?

12.3. Operations

12.3.1. Create Operation

- Identifier: create

The Create operation allows a client to provision a new Contact resource. The operation accepts as input all create-only and read-write data elements defined for the Contact Data Object.

- Authorisation:
  - Generally each client is authorised to create new contact objects becoming a sponsoring client. This can be however constrained by the server policy, e.g. by applying rate limiting or compliance locks.

In EPP Compatibility Profile, the following data elements MUST be provided:

- Handle ID (`id`)
- At least one Contact Information entry (`contactInfo`) containing a Name (`name`) and an Address (`addr`) with City (`city`) and Country Code (`cc`)
- E-mail (`email`)
- Authorisation Information (`authInfo`)

12.3.2. Read Operation

- Identifier: read

The Read operation allows a client to retrieve the data elements of a Contact resource. The server's response MAY vary depending on client authorisation and server policy.

- Authorisation:
  - Sponsoring client:
    - Full object
  - Other client:
    - Without Object Authorisation:
      - Limited object (non-confidential properties) or operation denied
    - With Object Authorisation:
      - Full object, however some properties only authorised to the sponsoring client MAY be redacted according to server policy

Authorisation Information (`authInfo`) MUST NOT be provided in the response if the querying client is not the current sponsoring client.

When constructing the response, the server MUST respect the disclosure policies defined by the Disclose Object (`disclose`), whether set by the server operator's default data-collection policy or by the sponsoring client for the contact. Data elements marked for non-disclosure MUST NOT be included in responses to unauthorised clients.

12.3.3. Update Operation

- Identifier: update

The Update operation allows a client to modify the attributes of an existing Contact resource.

- Authorisation:
  - Only sponsoring client is authorised to perform this operation

The following aspects of the contact object MAY be modified:

- Status values that are client-manageable (prefixed with "client") MAY be added or removed.
- Contact Information, Voice Phone Number, Fax Phone Number, E-mail, Authorisation Information, and Disclose preferences MAY be changed.

At least one change MUST be provided, unless the operation is extended.

12.3.4. Delete Operation

- Identifier: delete

The Delete operation allows a client to remove an existing Contact resource. The operation targets a specific data object identified by its Handle ID.

- Authorisation:
  - Only sponsoring client is authorised to perform this operation

The server SHOULD reject a delete request if the contact object is associated with other known objects (e.g., domain names). An associated contact SHOULD NOT be deleted until associations with other known objects have been broken.

The error response SHOULD indicate the existing object associations.

12.4. Processes

The Contact Data Object supports the following Process Objects Section 10 unmodified:

- Transfer Process

The Contact Data Object supports no Process Objects extended by an object-specific definition.

13. Host Data Object

13.1. Object Description

- Name: Host Data Object
- Identifier: host
- Unique Identifier: hostName
- Description: A Host Data Object represents a name server that provides DNS services for a domain name.

13.2. Data Elements

The following data elements are defined for the Host Data Object.

- Host Name
  - Identifier: hostName
  - Cardinality: 1
  - Mutability: read-write
  - Data Type: Fully Qualified Domain Name
  - Description: Fully qualified name of a host.
  - Constraints:
    - An Internal host MUST be treated as a subordinate host when it is used as a nameserver for a domain other than its superordinate domain.
    - Actions that do not preserve the relationship between a host object and its superordinate domain object MUST be disallowed.
- Provisioning Metadata
  - Identifier: provMetadata
  - Cardinality: 1
  - Mutability: read-only
  - Data Type: Provisioning Metadata Object
  - Description: Standard metadata about the object's lifecycle and ownership.
  - Constraints: (None)
- Status
  - Identifier: status
  - Cardinality: 1+
  - Mutability: read-write
  - Data Type: Host Status Object
  - Description: The current status descriptors associated with the host.

- Constraints: Possible combinations of Host Status Labels are specified in Host Status Values

- DNS Data
  - Identifier: dns
  - Cardinality: 0-1
  - Mutability: read-write
  - Data Type: DNS Data Object
  - Description: DNS resource records and operational controls related to the host. This structure follows the unified DNS data model defined in [I-D.simmen-rpp-dns-data].
  - Constraints:
    - The names of DNS entries MUST be the host name itself or subordinate to the host name.
    - In EPP Compatibility Profile the record entries MUST be limited to A and AAAA entries for IPv4 and IPv6 glue records respectively.
    - In EPP Compatibility Profile, IPv4 addresses MUST conform to [RFC0791] and IPv6 addresses MUST conform to [RFC4291]. A server MAY reject IP addresses that have not been allocated for public use by IANA.

- Processes
  - Identifier: processes
  - Cardinality: 0-1
  - Mutability: read-only
  - Data Type: Processes Object
  - Direct Access: true
  - Description: The Process Objects currently or recently initiated on the host object.
  - Constraints: (None)

13.3. Operations

13.3.1. Create Operation

- Identifier: create

The Create operation allows a client to provision a new Host Data Object. The operation accepts as input all create-only and read-write data elements defined for the Host Data Object.

- Authorisation:
  - Generally each client is authorised to create new host objects becoming a sponsoring client. This can be however constrained by the server policy, e.g. by applying rate limiting or compliance locks.

If the host name exists in a namespace for which the server is authoritative, then the superordinate domain of the host MUST be known to the server before the host object can be created.

In EPP Compatibility Profile, IP addresses are REQUIRED only as needed to produce DNS glue records. If the host name exists in a namespace for which the server is authoritative and is subordinate to an existing domain, IP addresses SHOULD be provided. If the host name is external to the server's namespace, IP addresses are not required by the DNS and MAY be omitted.

13.3.2. Read Operation

- Identifier: read

The Read operation allows a client to retrieve the data elements of a Host Data Object.

- Authorisation:
  - Any client is authorised to retrieve the full object. In EPP Compatibility Profile, host objects do not carry authorisation information and there is no distinction based on client identity as described in Section 3.1.2 of [RFC5732].

13.3.3. Update Operation

- Identifier: update

The Update operation allows a client to modify the attributes of an existing Host Data Object. The operation targets a specific data object identified by its host name.

- Authorisation:
  - Only sponsoring client is authorised to perform this operation

Host name changes MAY require the addition or removal of IP addresses to be accepted by the server. IP address association MAY be subject to server policies for provisioning hosts as name servers. The IP address restrictions described for the Create operation apply to updates as well.

At least one change to a data element MUST be provided, unless the operation is extended.

Host name changes can have an impact on associated objects that refer to the host object. A Host Name change SHOULD NOT require additional updates of associated objects to preserve existing associations, with one exception: changing an External host object that has associations with objects that are sponsored by a different client. Attempts to update such hosts directly MUST fail. The change can be provisioned by creating a new External host with a new name and any needed new attributes, and subsequently updating the other objects sponsored by the client.

13.3.4. Delete Operation

- Identifier: delete

The Delete operation allows a client to remove an existing Host Data Object. The operation targets a specific data object identified by its host name.

- Authorisation:
  - Only sponsoring client is authorised to perform this operation

The server SHOULD reject a delete request if the host object is associated with any other object, such as a domain name object. Deleting a host object without first breaking existing associations can cause DNS resolution failure for domain objects that refer to the deleted host object.

> TODO: consider RFC 9874 / BCP 244 for host deletions practices

The error response SHOULD indicate the related associated objects.

13.4. Processes

The Host Data Object Data Object supports the following Process Objects Section 10 unmodified:

- Transfer Process
  - Constraints: Host objects MUST NOT be transferred directly. They are transferred implicitly when their superordinate domain object is transferred.

The Host Data Object supports no Process Objects Section 10 extended by an object-specific definition.

14. Organisation Data Object

14.1. Object Description

- Name: Organisation Data Object
- Identifier: organisation
- Unique Identifier: id
- Description: An Organisation Data Object represents an entity, such as a registrar or reseller that is involved in the domain registration process. This object is based on the EPP Organisation Mapping defined in [RFC8543].

14.2. Data Elements

The following data elements are defined for the Organisation Data Object.

- Organisation ID
  - Identifier: id
  - Cardinality: 1
  - Mutability: create-only
  - Data Type: Identifier
  - Description: A server-unique identifier for the organisation object.
  - Constraints:
    - This value MUST be supported to be provided by the client.
    - Servers MAY support server-side generation of this value.

- Provisioning Metadata
  - Identifier: provMetadata
  - Cardinality: 1
  - Mutability: read-only
  - Data Type: Provisioning Metadata Object
  - Description: Standard metadata about the object's lifecycle and ownership.
  - Constraints: (None)
- Status
  - Identifier: status
  - Cardinality: 1+
  - Mutability: read-write
  - Data Type: Organisation Status Object
  - Description: The current operational status descriptors associated with the organisation. An organisation object MUST always have at least one associated status value.
  - Constraints:
    - Possible combinations of Organisation Status Labels are specified in Organisation Status Values.
    - A client MUST NOT alter server status values set by the server itself.
- Roles
  - Identifier: roles
  - Cardinality: 1+
  - Mutability: read-write
  - Data Type: DictionaryComposition[Organisation Role Object]
    - Label Description: type of role
    - Label Constraints: The value MUST be one of the role types registered in the IANA "EPP Organisation Role Values" registry as defined in Section 7.3 of [RFC8543]. Initial values include `registrar`, `reseller`, `privacyproxy`, and `dns-operator`.
  - Description: One or more roles describing the relationship the organisation has within the registry ecosystem. An organisation object MUST always have at least one associated role.
  - Constraints:
    - An organisation MAY have multiple roles with different role types.
    - Role types are registered in the IANA "EPP Organisation Role Values" registry as defined in Section 7.3 of [RFC8543]. Initial values include `registrar`, `reseller`, `privacyproxy`, and `dns-operator`.
- Parent Organisation
  - Identifier: parent
  - Cardinality: 0-1

- Mutability: read-write
- Data Type: Aggregation[Organisation Data Object]
- Description: The parent organisation in a hierarchical organisation structure (e.g., a reseller's parent registrar).
- Constraints:
  - Loops MUST be prohibited. If organisation A has organisation B as its parent, organisation B MUST NOT have organisation A as its parent, directly or transitively.

- Contact Information
  - Identifier: contactInfo
  - Cardinality: 0-1
  - Mutability: read-write
  - Data Type: External:RPP-JSContact-Profile:Card
  - Description: Contact information for the organisation.

- Contacts
  - Identifier: contacts
  - Cardinality: 0+
  - Mutability: read-write
  - Data Type: LabelledAggregation[Contact Data Object]
    - Label Description: type of contact
    - Label Constraints: The value MUST be one of the contact types registered in the IANA "EPP Organisation Contact Types" registry as defined in Section 7.4 of [RFC8543]. Initial values include `admin`, `tech`, `billing`, `abuse`, and `custom`.
  - Description: Identifiers of contact objects associated with the organisation.
  - Constraints:
    - Contact object identifiers MUST be known to the server before the contact can be associated.

> TODO: how handle the "custom" contact type?

- Users
  - Identifier: users
  - Cardinality: 0+
  - Mutability: read-write
  - Data Type: LabelledAggregation[User Data Object]
    - Label Description: RBAC role of the user
    - Label Constraints: The value MUST be one of the role types registered in the IANA "RPP User Role Values" registry.
  - Direct Access: true

    - Description: One or more RBAC roles assigned to the user.
    - Constraints:
        - Each role value MUST be a non-empty string.
        - Allowed role values MAY be constrained by server policy.

- Processes
    - Identifier: processes
    - Cardinality: 0-1
    - Mutability: read-only
    - Data Type: Processes Object
    - Direct Access: true
    - Description: The Process Objects currently or recently initiated on the organisation object.
    - Constraints: (None)

> TODO: define an IANA registry for user roles?

14.3. Operations

14.3.1. Create Operation

- Identifier: create

The Create operation allows a client to provision a new Organisation Data Object. The operation accepts as input all create-only and read-write data elements defined for the Organisation Data Object.

- Authorisation:
    - Generally only registry clients are authorised to create new organisation objects.

An organisation object MUST include at least one role on creation. The server MAY defer completing the action and return a `pendingCreate` status if human or third-party review is required.

14.3.2. Read Operation

- Identifier: read

The Read operation allows a client to retrieve the data elements of an Organisation Data Object.

- Authorisation:
    - Any client is authorised to retrieve organisation object information. The server MAY restrict the information returned based on client identity and server policy.

14.3.3. Update Operation

- Identifier: update

The Update operation allows a client to modify the attributes of an existing Organisation Data Object.

- Authorisation:
  - Generally only registry clients are authorised to update organisation objects.

The following aspects of the organisation object MAY be modified:

- Contacts MAY be added or removed.
- Roles MAY be added or removed.
- Status values that are client-manageable (prefixed with "client") MAY be added or removed.
- Parent Organisation ID and Contact Information MAY be changed.
- Users MAY be added or removed.

A client MUST NOT add, delete, or alter values for statuses managed by the server (prefixed with "server"). A server MAY add, delete, or alter status values set by a client, subject to server policy.

14.3.4. Delete Operation

- Identifier: delete

The Delete operation allows a client to remove an existing Organisation Data Object. The operation targets a specific data object identified by its Organisation ID.

- Authorisation:
  - Generally only registry clients are authorised to delete organisation objects.

An organisation object MUST NOT be deleted if it is associated with other known objects (e.g., domain names, contacts, or child organisations). The server MUST reject such a delete request and notify the client that object relationships exist.

The error response SHOULD indicate the related associated objects.

15. Message Data Object

15.1. Object Description

- Name: Message Data Object
- Identifier: message
- Unique Identifier: id
- Description: A Message Data Object represents a message which is inserted in the message queue of an organisation and retrieved by clients associated with that organisation.

15.2. Data Elements

The following data elements are defined for the Message Data Object.

- Message ID
  - Identifier: id
  - Cardinality: 1
  - Mutability: read-only
  - Data Type: Identifier
  - Description: A server-unique identifier for the message object.
  - Constraints: (None)
- Creation Date
  - Identifier: creationDate
  - Cardinality: 1
  - Mutability: read-only
  - Data Type: Timestamp
  - Description: The date and time when the message object was created and inserted into the queue.
  - Constraints: The value is set by the server.
- Status
  - Identifier: status
  - Cardinality: 1
  - Mutability: read-only
  - Data Type: String
  - Description: The current lifecycle status of the message object.
  - Constraints:
    - Possible values: `queued`, `delivered` and `removed`
- Data
  - Identifier: data
  - Cardinality: 1
  - Mutability: create-only
  - Data Type: Base Message Object
  - Description: The data related to a specific message type, such as a transfer request or a review response.
  - Constraints: (None)

15.3. Operations

15.3.1. Create Operation

- Identifier: create

The Create operation allows a server to insert a new message into the queue for a registrar. The operation accepts as input all create-only and read-write data elements defined for the Message Data Object.

- Authorisation:
  - Only the server is authorised to create new message objects.

15.3.2. Read Operation

- Identifier: read

The Read operation allows a client to retrieve the data elements of a Message Data Object.

- Authorisation:
  - Any client is authorised to retrieve message object information. The server MAY restrict the information returned based on client identity and server policy.

Organisation ID (`owner`) MUST NOT be provided in the response, since a client can only ever retrieve messages linked to its own organisation.

15.3.3. Query Operation

- Identifier: query

The Query operation allows a client to retrieve a collection of Message Data Object instances based on specified criteria.

- Authorisation:
  - Any client is authorised to retrieve message object information. The server MAY restrict the information returned based on client identity and server policy.
- Input:
  - Parameters:
    - Message Type
      - Identifier: type
      - Cardinality: 0-1
      - Data Type: String
      - Description: The type of the message to use as a filter when retrieving message objects.
      - Constraints:

        - The type value MUST be a valid identifier for an RPP message component (e.g. reviewResponse).

15.3.4. Delete Operation

- Identifier: delete

The Delete operation allows a client to remove an existing Message Data Object. The operation targets a specific data object identified by its Message ID.

- Authorisation:
  - Generally clients are authorised to delete message objects linked to the organisation the client belongs to.

16. User Object

16.1. Object Description

- Name: User Data Object
- Identifier: user
- Unique Identifier: id
- Description: Represents a user linked to an Organisation Object. Each user carries a set of RBAC roles that define the user's permissions within the context of the owning Organisation Object. The lifecycle of a User Object is bound to its Organisation Object.

16.2. Data Elements

The following data elements are defined for the User Data Object.

- User Identifier
  - Identifier: id
  - Cardinality: 1
  - Mutability: create-only
  - Data Type: Identifier
  - Description: A unique identifier for the user
  - Constraints: MUST be globally unique in the server scope
- Details
  - Identifier: details
  - Cardinality: 1
  - Mutability: read-write
  - Data Type: External:RPP-JSContact-Profile:Card
  - Description: The details of the user, such as name, email, and phone number
  - Constraints: -

- Description
  - Identifier: description
  - Cardinality: 0-1
  - Mutability: read-write
  - Data Type: String
  - Description: A description of the user
  - Constraints: -
- Status
  - Identifier: status
  - Cardinality: 1
  - Mutability: read-write
  - Data Type: User Status Object
  - Description: The current status of the user.
  - Constraints:
    - Allowed status values and transitions are specified in User Status Values.
    - A client MUST NOT alter server status values set by the server itself.
- Organisation
  - Identifier: organisation
  - Cardinality: 1
  - Mutability: read-only
  - Data Type: Aggregation[Organisation Data Object]
  - Description: A reference to the owner organisation object.
  - Constraints: -
- Processes
  - Identifier: processes
  - Cardinality: 0-1
  - Mutability: read-only
  - Data Type: Processes Object
  - Direct Access: true
  - Description: The Process Objects currently or recently initiated on the user object.
  - Constraints: (None)

> TODO: what other data elements should be included for the User Data Object?

16.3. Operations

16.3.1. Create Operation

- Identifier: create

The Create operation creates a new user, which is associated with an existing Organisation Object. The operation accepts as input all create-only and read-write data elements of the User Object.

- Authorisation:
  - Client can only create users in the context of the organisation linked to the client, and only if the client has the necessary permissions to create users in that organisation.

16.3.2. Read Operation

- Identifier: read

The Read operation retrieves the data of a specific User Object.

- Authorisation:
  - Client can only read users in the context of the organisation linked to the client, and only if the client has the necessary permissions to read users in that organisation.

16.3.3. Update Operation

- Identifier: update

The Update operation modifies the read-write data elements.

- Authorisation:
  - Client can only update users in the context of the organisation linked to the client, and only if the client has the necessary permissions to update users in that organisation.

16.3.4. Delete Operation

- Identifier: delete

The Delete operation removes a specific User Object.

- Authorisation:
  - Client can only delete users in the context of the organisation linked to the client, and only if the client has the necessary permissions to delete users in that organisation.

The server MUST reject this operation if the target User Object is the last user of the owning organisation that is permitted to manage users.

User Objects are deleted together with their owning Organisation Object.

17. Discovery Data Object

17.1. Object Description

- Name: Discovery Data Object
- Identifier: discovery
- Unique Identifier: (None)

- Description: A Discovery Data Object describes the capabilities of an RPP server, such as the supported versions, TLDs, extensions, profiles, resource collections, authentication methods and endpoints. The server provides a single Discovery Data Object, which is therefore not identified by a Unique Identifier.

17.2. Data Elements

The following data elements are defined for the Discovery Data Object.

- Versions
  - Identifier: versions
  - Cardinality: 1+
  - Mutability: read-only
  - Data Type: Discovery Version Object
  - Description: The versions of RPP supported by the server.
  - Constraints: An operator that supports more than one incompatible version MUST deploy them on distinct base URLs and announce each of them.
- Environment
  - Identifier: environment
  - Cardinality: 1
  - Mutability: read-only
  - Data Type: Discovery Environment Object
  - Description: Information about the RPP server.
  - Constraints: (None)
- TLDs
  - Identifier: tlds
  - Cardinality: 1+
  - Mutability: read-only
  - Data Type: String
  - Description: The TLDs supported by the server, for example "example" or "org".
  - Constraints: (None)
- Extensions
  - Identifier: extensions
  - Cardinality: 0+
  - Mutability: read-only
  - Data Type: Extension Object
  - Description: The extensions supported by the server.
  - Constraints: The server MUST include the version of each extension.

- Profiles
  - Identifier: profiles
  - Cardinality: 0+
  - Mutability: read-only
  - Data Type: String
  - Description: The names of the profiles supported by the server.
- Objects
  - Identifier: objects
  - Cardinality: 1+
  - Mutability: read-only
  - Data Type: String
  - Description: The list of identifiers of data objects supported by the server
  - Constraints: (None)
- Date and Time
  - Identifier: datetime
  - Cardinality: 1
  - Mutability: read-only
  - Data Type: Timestamp
  - Description: The current date and time of the server.
  - Constraints: The value MUST be expressed in UTC using the `Z` suffix, for example `2026-06-01T10:00:00Z`, and MUST NOT use a numeric UTC offset or any other time zone.
- Links
  - Identifier: links
  - Cardinality: 0+
  - Mutability: read-only
  - Data Type: LabelledAggregation[String]
    - Label Description: The relation type of the link.
    - Label Constraints: The value MUST be a link relation type registered in the IANA "Link Relation" registry.
  - Description: Links to documents related to the use of RPP with the server, such as policies and terms. Each value is the location of a linked document, labelled with its relation type.
  - Constraints:
    - Each value MUST be an absolute URL using the HTTPS scheme.
    - The document MUST be accessible to the client without authentication.

        - A server SHOULD provide a link for each of the `privacy-policy` and `terms-of-service` relation types, defined in [RFC6903].

- IDN LGRs
    - Identifier: idnLgr
    - Cardinality: 0+
    - Mutability: read-only
    - Data Type: Discovery IDN LGR Object
    - Description: The Label Generation Rulesets (LGRs) supported by the server.
    - Constraints: (None)

- Notices
    - Identifier: notices
    - Cardinality: 0+
    - Mutability: read-only
    - Data Type: Discovery Notice Object
    - Description: Notices from the server operator.
    - Constraints: (None)
    - Description: The objects allowed for provisioning operations, for example "domainName", "host" or "contact".
    - Constraints: The values MUST be valid object identifiers as defined in the RPP Data Object Registry.

17.3. Operations

17.3.1. Read Operation

- Identifier: read

The Read operation allows a client to retrieve the Discovery Data Object. It is the only operation supported for this object.

- Input: (None)
- Authorisation:
    - Any client is authorised to perform this operation. The operation MUST be available without authentication, so that a client can retrieve it before authenticating with the server.

18. Profile Data Object

18.1. Object Description

- Name: Profile Data Object
- Identifier: profile

- Unique Identifier: name
- Description: A Profile Data Object represents a named set of protocol features and versions that defines the compatibility and capabilities of RPP server implementations. A profile is published by the server, and a client can retrieve it to determine which features, objects and extensions it can use.

18.2. Data Elements

The following data elements are defined for the Profile Data Object. All data elements are managed by the server.

- Name
  - Identifier: name
  - Cardinality: 1
  - Mutability: read-only
  - Data Type: String
  - Description: A unique name that identifies the profile.
  - Constraints:
    - The name of a standard profile, which is published in the IANA registry for RPP profiles, MUST use the RPP URN namespace, for example `urn:ietf:params:rpp:profile:{profile_name}`, and MUST be unique within that namespace. It SHOULD be descriptive of the features and versions included in the profile.
    - The name of a private profile, which is not published in the IANA registry, MUST be globally unique. The use of reverse domain name notation is RECOMMENDED.
- Description
  - Identifier: description
  - Cardinality: 1
  - Mutability: read-only
  - Data Type: String
  - Description: A human-readable description of the profile and its intended use.
  - Constraints: (None)
- Version
  - Identifier: version
  - Cardinality: 1
  - Mutability: read-only
  - Data Type: String
  - Description: The version of the profile.
  - Constraints: (None)

- RPP Version
  - Identifier: rppVersion
  - Cardinality: 1
  - Mutability: read-only
  - Data Type: String
  - Description: The minimum version of the RPP protocol that is supported or required for the profile.
  - Constraints: (None)
- Objects
  - Identifier: objects
  - Cardinality: 1+
  - Mutability: read-only
  - Data Type: String
  - Description: The list of identifiers of data objects supported by the server
  - Constraints: (None)
- Extensions
  - Identifier: extensions
  - Cardinality: 0+
  - Mutability: read-only
  - Data Type: Extension Object
  - Description: The extensions, including their names and versions, that are supported or required for the profile.
  - Constraints:
    - A profile MUST NOT list the same extension name more than once.
    - The server MUST support all extensions listed in the profile.
- Base Profile
  - Identifier: baseProfile
  - Cardinality: 0-1
  - Mutability: read-only
  - Data Type: Aggregation[Profile Data Object]
  - Description: A base profile that is extended by this profile. The features and versions of the base profile are included in this profile, which MAY add to them or override them, for example the version of an extension.
  - Constraints:
    - The base profile MUST be published in the IANA registry for RPP profiles, or be made available to the client using other means.
    - The depth of inheritance MUST be limited to 1: the base profile MUST NOT itself have a base profile.

        - A profile MUST NOT be its own base profile.

- Default Profile
    - Identifier: defaultProfile
    - Cardinality: 1
    - Mutability: read-only
    - Data Type: Boolean
    - Description: The default profile that is used when no specific profile is requested by the client.
    - Constraints:
        - The default profile MUST be published in the IANA registry for RPP profiles, or be made available to the client using other means.
        - Only one profile MUST be designated as the default profile.

18.3. Operations

18.3.1. Read Operation

- Identifier: read

The Read operation allows a client to retrieve the Profile Data Object identified by its name. It is the only operation supported for this object.

- Authorisation:
    - Any authenticated client is authorised to perform this operation.

19. IANA Considerations

19.1. RPP registry group

The IANA is requested to create a new registry group for RPP, this will be used to group together all the RPP-related registries such as those for discovery URLs, extensions and profiles.

```
Name of the registry: RESTful Provisioning Protocol (RPP)
Reference: This Document
IANA Registry Reference: TODO
```

19.2. RPP Data Object Registry

Name of the registry: RPP Data Object Registry Registry group: RESTful Provisioning Protocol (RPP) Registration procedure: Specification Required

This document establishes the "RESTful Provisioning Protocol (RPP) Data Object Registry". This registry serves as a catalogue of all data objects, component objects, data elements, and operations used within RPP.

19.2.1. Registration Policy

The policy for adding new objects, data elements, or operations to this registry is "Specification Required" [RFC8126].

Standardised RPP extensions that introduce new data objects, add data elements to existing objects, or define new operations, operation input data elements, or operation parameters MUST register these additions in this registry. Each such registration MUST reference the specification that defines the extension.

Private (non-standardised) extensions are not required to register in this registry.

19.2.2. Registry Structure

The registry is organised as a collection of Object definitions. Each Object definition MUST include:

- A header containing the Object Identifier, Object Name, Object Type (Resource, Process or Component), the identifier of the object it extends (if any), a brief description, and a reference to its defining specification.
- A "Data Elements" table listing all persisted data elements associated with the object. Each entry MUST specify the element's Identifier, Name, Cardinality, Mutability, Data Type, description, and a reference to the specification that defines it.
- An "Operations" section (applicable only for Object Types Resource or Process). For each operation, the registry MUST provide:
  - The Operation's Name, a description, and a reference to the specification that defines it.
  - An "Input Data Elements" table listing all input data elements of the operation. Each entry MUST specify the input data element's Identifier, Name, Cardinality, Data Type, description, and a reference to the specification that defines it.
  - A "Parameters" table listing all parameters of the operation. Each entry MUST specify the parameter's Identifier, Name, Cardinality, Data Type, description, and a reference to the specification that defines it.

Extensions MAY add new data elements, operations, operation input data elements, or operation parameters to existing Object definitions in the registry. Each such addition MUST reference the extension specification that introduces it, allowing implementations to distinguish core protocol elements from extension-defined elements.

19.2.3. Initial Registrations

The initial contents of the RPP Data Object Registry are defined below.

Object: period

Object Name: Period Object

Object Type: Component

Description: Represents a duration of time.

Reference: [This-ID]

Data Elements

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
value	Value	1	read-write	Integer	The numeric value of the period.
unit	Unit	1	read-write	String	The unit of the period.

Table 14

Object: dnsRecord

Object Name: DNS Resource Record Object

Object Type: Component

Description: Represents a single DNS resource record.

Reference: [This-ID]

Data Elements

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
name	Name	1	read-write	String	The owner name of the DNS entry.
class	Class	0-1	read-write	String	The DNS resource record class.
type	Type	1	read-write	String	The DNS resource record type, indicating the format of the RDATA field.
rdata	RDATA	1	read-write	Object	The actual payload data of the DNS record. Structure depends on the record type.

Table 15

Object: dnsControls

Object Name: DNS Operational Controls Object

Object Type: Component

Description: Contains operational control parameters that a client MAY use to influence server-side DNS behaviour for a set of DNS records.

Reference: [This-ID]

Data Elements

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
ttl	TTL	0-1	read-write	Dictionary [Integer]	Controls the caching behaviour of DNS resource records, keyed by lower-case record type name.
maxSigLifetime	Maximum Signature Lifetime	0-1	read-write	Dictionary [Integer]	Maximum number of seconds after signature generation when the parent's signature on signed DNS data should expire, keyed by lower-case record type name.

Table 16

Object: dnsData

Object Name: DNS Data Object

Object Type: Component

Description: A container for DNS resource records and associated operational controls for a provisioned object.

Reference: [This-ID]

Data Elements

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
records	Records	0+	read-write	DNS Resource Record Object	An array of DNS resource records associated with the provisioned object.
controls	Controls	0-1	read-write	DNS Operational Controls Object	Operational control parameters for the DNS records.

Table 17

Object: authInfo

Object Name: Authorisation Information Object

Object Type: Component

Description: Contains authorisation credentials for an operation.

Reference: [This-ID]

Data Elements

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
method	Method	1	create-only	String	The identifier of the RPP authorisation method.
authdata	Authorisation Information	1	create-only	String	The value of the authorisation information. It might be as simple as a password string, but also more complex values like public key certificates or tokens encoded as string are possible.

Table 18

Object: domainStatus

Object Name: Domain Name Status Object

Object Type: Component

Description: Represents one of the status values associated with a Domain Name Data Object

Reference: [This-ID]

Data Elements

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
label	Label	1	create-only	String	machine-readable enum label of a status
reason	Reason	0-1	create-only	String	a human-readable text that describes the rationale for the status applied to the object.
due	Due	0-1	read-write	Timestamp	a timestamp, when this status is going to be removed automatically, or changed to other status. This field can be used to express lifecycle related information.

Table 19

Object: hostStatus

Object Name: Host Status Object

Object Type: Component

Description: Represents one of the status values associated with a Host Data Object

Reference: [This-ID]

Data Elements

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
label	Label	1	create-only	String	machine-readable enum label of a status
reason	Reason	0-1	create-only	String	a human-readable text that describes the rationale for the status applied to the object.

Table 20

Object: contactStatus

Object Name: Contact Status Object

Object Type: Component

Description: Represents one of the status values associated with a Contact Data Object

Reference: [This-ID]

Data Elements

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
label	Label	1	create-only	String	machine-readable enum label of a status
reason	Reason	0-1	create-only	String	a human-readable text that describes the rationale for the status applied to the object.

Table 21

Object: organisationRoleStatus

Object Name: Organisation Role Status Object

Object Type: Component

Description: Represents one of the status values associated with an Organisation Role Object. A role SHOULD have at least one associated status value.

Reference: [This-ID]

Data Elements

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
label	Label	1	create-only	String	machine-readable enum label of a status
reason	Reason	0-1	create-only	String	a human-readable text that describes the rationale for the status applied to the role.

Table 22

Object: organisationStatus

Object Name: Organisation Status Object

Object Type: Component

Description: Represents one of the status values associated with an Organisation Data Object, as defined in Section 3.4 of [RFC8543]. An Organisation Data Object MUST always have at least one associated status value.

Reference: [This-ID]

Data Elements

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
label	Label	1	create-only	String	machine-readable enum label of a status
reason	Reason	0-1	create-only	String	a human-readable text that describes the rationale for the status applied to the object.

Table 23

Object: userStatus

Object Name: User Status Object

Object Type: Component

Description: Represents the status value associated with a User Data Object. A User Data Object MUST always have exactly one associated status value.

Reference: [This-ID]

Data Elements

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
label	Label	1	create-only	String	machine-readable enum label of a status
reason	Reason	0-1	create-only	String	a human-readable text that describes the rationale for the status applied to the object.

Table 24

Object: provMetadata

Object Name: Provisioning Metadata Object

Object Type: Component

Description: Contains standard metadata about the lifecycle and ownership of a provisioned object.

Reference: [This-ID]

Data Elements

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
repositoryId	Repository ID	0-1	read-only	Identifier	A server-assigned unique identifier for the object.
spClientId	Sponsoring Client ID	1	read-only	Client Identifier	The identifier of the client that is the current sponsor of the object.
crClientId	Creating Client ID	0-1	read-only	Client Identifier	The identifier of the client that created the object.
crDate	Creation Date	0-1	read-only	Timestamp	The date and time of object creation.
upClientId	Updating Client ID	0-1	read-only	Client Identifier	The identifier of the client that last updated the object.
upDate	Update Date	0-1	read-only	Timestamp	The date and time of the most recent object modification.
trDate	Transfer Date	0-1	read-only	Timestamp	The date and time of the most recent successful object transfer.

Table 25

Object: processes

Object Name: Processes Object

Object Type: Component

Description: A container grouping the Process Objects currently or recently initiated on a Data Object, structured as a dictionary keyed by process type where each element holds an unordered array of Process Objects of that type. The set of process-type data elements is extensible; extensions MAY register additional process-type elements.

Reference: [This-ID]

Data Elements

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
transferProcess	Transfer Processes	0+	read-only	Aggregation [Transfer Process Object]	The transfer processes initiated on the owning Data Object.
renewProcess	Renew Processes	0+	read-only	Aggregation [Renew Process Object]	The renew processes initiated on the owning Data Object.
restoreProcess	Restore Processes	0+	read-only	Aggregation [Restore Process Object]	The restore processes initiated on the owning Data Object.
createProcess	Create Processes	0+	read-only	Aggregation [Create Process Object]	The create processes initiated on the owning Data Object.

Table 26

Object: baseProcess

Object Name: Base Process Object

Object Type: Component

Description: Base structure for all Process Objects, containing the data elements common to every process.

Reference: [This-ID]

Data Elements

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
processId	Process ID	0-1	read-only	String	A server-assigned identifier of the process instance, unique within the Owner Data Object.

Table 27

Object: baseMessage

Object Name: Base Message Object

Object Type: Component

Description: Base structure for all message objects, containing common data elements such as the object type and object identifier.

Reference: [This-ID]

Data Elements

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
objectType	Object Type	0-1	read-only	String	The object identifier of the object affected by the requested action.
objectId	Object Identifier	0-1	read-only	String	The unique identifier of the object affected by the requested operation.

Table 28

Object: reviewResponseMessage

Object Name: Review Response Object

Object Type: Component

Extends: baseMessage

Description: Notification of the result of an offline-review of a requested operation.

Reference: [This-ID]

Data Elements

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
approved	Approval Result	1	read-only	Boolean	Whether the requested action was approved and completed.
clientTransactionId	Client Transaction ID	0-1	read-only	String	The client transaction identifier from the original request, if supplied.
serverTransactionId	Server Transaction ID	1	read-only	String	The server transaction identifier returned for the original request.
completionDate	Completion Date	1	read-only	Timestamp	The date and time when review of the requested action was completed.
message	Message	1	read-only	String	Human-readable message associated with the review response.
process	Process	0-1	read-only	Base Process Object	The process whose operation was subject to the offline review.

Table 29

Object: transferRequestMessage

Object Name: Transfer Request Message Object

Object Type: Component

Extends: baseMessage

Description: Notification for losing client of started transfer request, where the client expected to act on the pending transfer request.

Reference: [This-ID]

Data Elements

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
status	Transfer Status	1	read-only	String	The current state of the transfer request.
requestDate	Request Date	1	read-only	Timestamp	The date and time the transfer was requested.
actionDate	Action Date	0-1	read-only	Timestamp	The deadline for responding before the server takes an automated action.
process	Process	0-1	read-only	Transfer Process Object	The transfer process the message pertains to.

Table 30

Object: transferOutcomeMessage

Object Name: Transfer Outcome Message Object

Object Type: Component

Extends: baseMessage

Description: Notification for a client affected by the completion of a transfer request.

Reference: [This-ID]

Data Elements

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
trStatus	Transfer Status	1	read-only	String	The final state of the transfer request.
requestDate	Request Date	1	read-only	Timestamp	The date and time the transfer was requested.
actionDate	Action Date	1	read-only	Timestamp	The date and time the transfer request was resolved.
expiryDate	Expiry Date	0-1	read-only	Timestamp	The end of the object's validity period if the operation causes a change in the validity period.
process	Process	0-1	read-only	Transfer Process Object	The transfer process the message pertains to.

Table 31

Object: expirationDeletionMessage

Object Name: Expiration Deletion Message Object

Object Type: Component

Extends: baseMessage

Description: Notification for the sponsoring client that an object was deleted following expiration.

Reference: [This-ID]

Data Elements

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
deletionDate	Deletion Date	1	read-only	Timestamp	The date and time the server deleted the object.

Table 32

Object: autoRenewalMessage

Object Name: Auto-Renewal Message Object

Object Type: Component

Extends: baseMessage

Description: Notification for the sponsoring client that the server automatically renewed an object.

Reference: [This-ID]

Data Elements

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
renewalDate	Renewal Date	1	read-only	Timestamp	The date and time the server renewed the object.
expiryDate	Expiry Date	1	read-only	Timestamp	The new expiration date of the object.

Table 33

Object: serviceNoticeMessage

Object Name: Service Notice Message Object

Object Type: Component

Extends: baseMessage

Description: Generic text based message containing information not covered by any other message type.

Reference: [This-ID]

Data Elements

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
message	Message	1	read-only	String	Human-readable service information for the receiving organisation.

Table 34

Object: transferProcess

Object Name: Transfer Process Object

Object Type: Process

Extends: baseProcess

Description: Represents a transfer request for a provisioned object. Creating this object initiates the transfer. Approve and Reject are additional operations; Delete corresponds to cancel.

Reference: [This-ID]

Data Elements

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
transferDir	Transfer Direction	0-1	create-only	String	The direction of the transfer ("pull" or "push"). If omitted, server policy determines default.
gainingClientId	Gaining Client ID	0-1	create-only	Client Identifier	The designated gaining client. REQUIRED for push transfers; MUST NOT be provided for pull.
trStatus	Transfer Status	1	read-only	String	The state of the transfer request.
reqClientId	Requesting Client ID	1	read-only	Client Identifier	The identifier of the client that initiated the transfer request.
requestDate	Request Date	1	read-only	Timestamp	The date and time that the transfer was requested.
actClientId	Acting Client ID	1	read-only	Client Identifier	The identifier of the client that should or did act on the request.

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
actionDate	Action Date	1	read-only	Timestamp	The response deadline (if pending) or completion date.

Table 35

Operations

Operation: Create

Operation Identifier: transferCreate

Description: Initiates a transfer of a provisioned object by creating a Transfer Process Object.

Input Data Elements: (None)

Parameters: (None)

Operation: Transfer Read

Operation Identifier: transferRead

Description: Queries the status of a transfer of a provisioned object.

Input Data Elements: (None)

Parameters: (None)

Operation: Transfer Delete

Operation Identifier: transferDelete

Description: Cancels a pending transfer of a provisioned object.

Input Data Elements: (None)

Parameters: (None)

Operation: Transfer Approve

Operation Identifier: transferApprove

Description: Approves a pending transfer of a provisioned object.

Input Data Elements: (None)

Parameters: (None)

Operation: Transfer Reject

Operation Identifier: transferReject (on Transfer Process Object)

Description: Rejects a pending transfer of a provisioned object.

Input Data Elements

Identifier	Name	Card.	Data Type	Description
reason	Reason	0-1	String	A human-readable text describing the rationale for rejection.

Table 36

Parameters: (None)

Object: renewProcess

Object Name: Renew Process Object

Object Type: Process

Extends: baseProcess

Description: Represents a renewal request for a provisioned object. Creating this object initiates the renewal.

Reference: [This-ID]

Data Elements

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
expiryDate	Expiry Date	0-1	create-only	Timestamp	The expiry date of the object after the renewal is completed.

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
renewalPeriod	Renewal Period	0-1	create-only	Period Object	The duration to be added to the object's registration period. This value is used by the server to calculate the new `expiryDate`. The default value MAY be defined by server policy. The number of units available MAY be subject to limits imposed by the server.

Table 37

Operations

Operation: Create

Operation Identifier: renewCreate

Description: Initiates a renewal of a provisioned object by creating a Renew Process Object.

Input Data Elements: (None)

Parameters: (None)

Object: createProcess

Object Name: Create Process Object

Object Type: Process

Extends: baseProcess

Description: Represents the process initiated when a resource creation operation is performed. Carries creation-specific inputs that are consumed during creation and not stored as persistent attributes of the created resource object.

Reference: [This-ID]

Data Elements: (None)

Operations

Operation: Create

Operation Identifier: create

Description: Invoked implicitly as a side effect of the resource object create operation; never invoked directly. Carries creation-specific inputs consumed during creation of the owning resource object.

Input Data Elements: (None)

Parameters: (None)

Operation: Read

Operation Identifier: read

Description: Retrieves the result or status of the creation, if the server exposes the process resource.

Input Data Elements: (None)

Parameters: (None)

Object: domainCreateProcess

Object Name: Domain Create Process Object

Object Type: Process

Extends: baseProcess

Description: The Create Process Object specific to the Domain Name Data Object. Implicitly initiated by the domain create operation; carries the requested initial registration period consumed during creation and not persisted as part of the domain object's state.

Reference: [This-ID]

Data Elements

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
period	Period	0-1	create-only	Period Object	The initial registration period for the domain name.

Table 38

Operations

Operation: Create

Operation Identifier: create

Description: Invoked implicitly as a side effect of the Domain Name Data Object create operation; never invoked directly. Carries the registration period consumed during domain creation.

Input Data Elements: (None)

Parameters: (None)

Operation: Read

Operation Identifier: read

Description: Retrieves the result or status of the domain creation, if the server exposes the process resource.

Input Data Elements: (None)

Parameters: (None)

Object: domainTransferProcess

Object Name: Domain Transfer Process Object

Object Type: Process

Description: The Transfer Process Object specific to the Domain Name Data Object. Carries the domain-specific transfer inputs and outputs, namely the requested transfer period and the resulting expiry date, that extend the common transfer process.

Reference: [This-ID]

Data Elements

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
expiryDate	Expiry Date	0-1	read-only	Timestamp	The end of the domain object's registration period if the transfer caused or causes a change in the validity period.

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
transferPeriod	Transfer Period	0-1	create-only	Period Object	The number of units to be added to the registration period of the domain object upon successful completion of the transfer. The number of units available MAY be subject to limits imposed by the server.

Table 39

Operations

Operation: Create

Operation Identifier: transferCreate

Description: Initiates a transfer of a Domain Name Data Object by creating a Domain Transfer Process Object.

Input Data Elements: (None)

Parameters: (None)

Object: disclose

Object Name: Disclose Object

Object Type: Component

Description: TBD

Reference: [This-ID]

Data Elements

Element Identifier	Element Name	Card.	Mutability	Data Type	Description

Table 40

Object: restoreProcess

Object Name: Restore Process Object

Object Type: Process

Extends: baseProcess

Description: Represents the current state of a restore request for an object that has entered the Redemption Grace Period (RGP). Returned as output of all restore operations. This object is OPTIONAL and is only used when the RGP feature is supported.

Reference: [This-ID]

Data Elements

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
restoreStatus	Restore Status	1	read-only	String	The current state of the restore process.
requestDate	Request Date	0-1	read-only	Timestamp	The date and time when the restore request was submitted. Absent if no request has been submitted.
reportDate	Report Date	0-1	read-only	Timestamp	The date and time when the most recent restore report was accepted. Absent if no report has been accepted.
reportDueDate	Report Due Date	0-1	read-only	Timestamp	The deadline for submitting a restore report before the object reverts to rgpRedemptionPeriod. Present only when status is rgpPendingRestore.
restoreReport	Restore Report	0-1	read-write	Restore Report Object	The restore report submitted as required by the RGP process.

Table 41

Operations

Operation: Restore Create

Operation Identifier: create

Description: Initiates recovery of a domain name in the rgpRedemptionPeriod state by creating a Restore Process Object. This operation is OPTIONAL and is only available when the RGP feature is supported.

Input Data Elements

Identifier	Name	Card.	Data Type	Description
restoreReport	Restore Report	0-1	Restore Report Object	An OPTIONAL inline restore report. If provided, the server processes the request and the report atomically. If the server does not require a restore report, this element MUST NOT be present and the restore is completed immediately. If the server requires a report and this element is absent, the object transitions to `pendingRestore` state awaiting a subsequent Report operation.

Table 42

Parameters: (None)

Operation: Restore Read

Operation Identifier: read (on Restore Process Object)

Description: Retrieves the current state of the RGP restore process for a domain name. This operation is OPTIONAL and is only available when the RGP feature is supported.

Input Data Elements: (None)

Parameters: (None)

Operation: Restore Update

Operation Identifier: update

Description: Submits or replaces the restore report for a domain name in the rgpPendingRestore state. This operation is OPTIONAL and is only available when the RGP feature is supported and the server requires a restore report.

Input Data Elements

Identifier	Name	Card.	Data Type	Description
restoreReport	Restore Report	1	Restore Report Object	The restore report to be submitted as part of the RGP process.

Table 43

Parameters: (None)

Object: restoreReport

Object Name: Restore Report Object

Object Type: Component

Description: Contains the redemption grace period restore report submitted by the sponsoring client as required by the RGP process. This object is OPTIONAL and is only used when the RGP feature is supported and a restore report is required by server policy.

Reference: [This-ID]

Data Elements

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
preData	Pre-Delete Data	0-1	read-write	String	A copy of the registration data that existed for the object prior to deletion.
postData	Post-Restore Data	0-1	read-write	String	A copy of the registration data that exists for the object at the time the restore report is submitted.
deleteTime	Delete Time	0-1	read-write	Timestamp	The date and time when the object delete request was sent to the server.
restoreTime	Restore Time	0-1	read-write	Timestamp	The date and time when the original restore request operation was sent to the server.

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
restoreReason	Restore Reason	0-1	read-write	String	A brief explanation of the reason for restoring the object.
statements	Statements	1-2	read-write	String	Mandatory client statements required by the RGP process.
other	Other	0-1	read-write	String	Any additional information needed to support the statements provided by the client.

Table 44

Object: domainName

Object Name: Domain Name Data Object

Object Type: Resource

Description: Represents a domain name and its associated data.

Reference: [This-ID]

Data Elements

Identifier	Name	Card.	Mutability	Data Type	Description
name	Name	1	create-only	Fully Qualified Domain Name	The fully qualified name of the domain object.
uName	Unicode Name	0-1	create-only	String	The fully qualified name of the domain using the Unicode character set.

Identifier	Name	Card.	Mutability	Data Type	Description
lgr	LGR	0-1	create-only	String	The name of the Label Generation Ruleset (LGR) containing the valid code points used for the Unicode Name (uName).
provMetadata	Provisioning Metadata	1	read-only	Provisioning Metadata Object	Standard metadata about object lifecycle and ownership.
status	Status	1+	read-write	Domain Name Status Object	The current status descriptors for the domain.
registrant	Registrant	0-1	read-write	Aggregation[Contact Data Object]	The registrant contact ID.
contacts	Contacts	0+	read-write	LabelledAggregation [Contact Data Object]	Associated contact objects.
nameservers	Nameservers	0+	read-write	Aggregation [Host Data Object]	A collection of nameservers associated with the domain.

Identifier	Name	Card.	Mutability	Data Type	Description
dns	DNS Data	0-1	read-write	DNS Data Object	DNS resource records and operational controls related to the domain name.
subordinateHosts	Subordinate Hosts	0+	read-only	Aggregation [Host Data Object]	Subordinate host names.
expiryDate	Expiry Date	0-1	read-only	Timestamp	Expiry timestamp.
authInfo	Authorisation Information	0-1	read-write	Authorisation Information Object	Authorisation information for the object.
processes	Processes	0-1	read-only	Processes Object	Process Objects initiated on the domain object.

Table 45

Operations

Operation: Create

Operation Identifier: create

Description: Provisions a new Domain Name resource.

Input Data Elements: (None)

Parameters: (None)

Operation: Read

Operation Identifier: read

Description: Retrieves the data elements of a Domain Name resource.

Input Data Elements: (None)

Parameters

Identifier	Name	Card.	Data Type	Description
hostsFilter	Hosts Filter	0-1	String	Controls which host information is returned.

Table 46

Operation: Update

Description: Modifies the read-write data elements of a Domain Name resource.

Input Data Elements: (None)

Parameters: (None)

Operation: Delete

Operation Identifier: delete

Description: Removes an existing Domain Name resource.

Input Data Elements: (None)

Parameters: (None)

Object: contact

Object Name: Contact Data Object

Object Type: Resource

Description: A Contact Data Object represents the contact information for an individual or organisation associated with other objects.

Reference: [This-ID]

Data Elements

Identifier	Name	Card.	Mutability	Data Type	Description
id	Handle ID	1	create-only	Identifier	External unique identifier of the contact object.
provMetadata	Provisioning Metadata	1	read-only	Provisioning Metadata Object	Standard metadata about the object's lifecycle and ownership.

Identifier	Name	Card.	Mutability	Data Type	Description
status	Status	1+	read-write	Contact Status Object	Status descriptors associated with the contact.
contactInfo	Contact Information	1	read-write	External:RPP-JSContact-Profile:Card	JSContact Card information.
authInfo	Authorisation Information	0-1	read-write	Authorisation Information Object	Authorisation information associated with the contact object.
disclose	Disclose	0-1	read-write	Disclose Object	Identifies elements that require exceptional server-operator handling to allow or restrict disclosure to third parties.
processes	Processes	0-1	read-only	Processes Object	Process Objects initiated on the contact object.

Table 47

Operations

Operation: Create Operation

Operation Identifier: create

Description: The Create operation allows a client to provision a new Contact resource. The operation accepts as input all create-only and read-write data elements defined for the Contact Data Object.

Input Data Elements: (None)

Parameters: (None)

Operation: Read Operation

Operation Identifier: read

Description: The Read operation allows a client to retrieve the data elements of a Contact resource. The server's response MAY vary depending on client authorisation and server policy.

Input Data Elements: (None)

Parameters: (None)

Operation: Update Operation

Operation Identifier: update

Description: The Update operation allows a client to modify the attributes of an existing Contact resource.

Input Data Elements: (None)

Parameters: (None)

Operation: Delete Operation

Operation Identifier: delete

Description: The Delete operation allows a client to remove an existing Contact resource. The operation targets a specific data object identified by its Handle ID.

Input Data Elements: (None)

Parameters: (None)

Object: host

Object Name: Host Data Object

Object Type: Resource

Description: Represents a name server that provides DNS services for a domain name.

Reference: [This-ID]

Data Elements

Identifier	Name	Card.	Mutability	Data Type	Description
hostName	Host Name	1	read-write	Fully Qualified Domain Name	Fully qualified name of a host.

Identifier	Name	Card.	Mutability	Data Type	Description
provMetadata	Provisioning Metadata	1	read-only	Provisioning Metadata Object	Standard metadata about object lifecycle and ownership.
status	Status	1+	read-write	Host Status Object	The current status descriptors for the host.
dns	DNS Data	0-1	read-write	DNS Data Object	DNS resource records and operational controls related to the host.
processes	Processes	0-1	read-only	Processes Object	Process Objects initiated on the host object.

Table 48

Operations

Operation: Create

Description: Provisions a new Host Data Object.

Input Data Elements: (None)

Parameters: (None)

Operation: Read

Description: Retrieves the data elements of a Host Data Object.

Input Data Elements: (None)

Parameters: (None)

Operation: Update

Description: Modifies the attributes of a Host Data Object.

Input Data Elements: (None)

Parameters: (None)

Operation: Delete

Description: Removes an existing Host Data Object.

Input Data Elements: (None)

Parameters: (None)

Object: organisationRole

Object Name: Organisation Role Object

Object Type: Component

Description: Represents a role that an organisation has within the registry ecosystem, as defined in Section 3.2 of [RFC8543]. An organisation object MUST always have at least one associated role.

Reference: [This-ID]

Data Elements

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
status	Role Status	0+	read-write	Organisation Role Status Object	The status of this role.
roleId	Role Identifier	0-1	read-write	String	A third-party-assigned identifier for this role, such as an IANA registrar ID.

Table 49

Object: organisation

Object Name: Organisation Data Object

Object Type: Resource

Description: Represents an entity, such as a registrar, reseller, DNS service operator, or privacy proxy, involved in the domain registration process. Based on the EPP Organisation Mapping defined in [RFC8543].

Reference: [This-ID]

Data Elements

Identifier	Name	Card.	Mutability	Data Type	Description
id	Organisation ID	1	create-only	Identifier	A server-unique identifier for the organisation object.
provMetadata	Provisioning Metadata	1	read-only	Provisioning Metadata Object	Standard metadata about the object's lifecycle and ownership.
status	Status	1+	read-write	Organisation Status Object	The current operational status descriptors for the organisation.
roles	Roles	1+	read-write	DictionaryComposition [Organisation Role Object]	One or more roles describing the organisation's relationship within the registry ecosystem.
parent	Parent Organisation	0-1	read-write	Aggregation [Organisation Data Object]	The parent organisation in a hierarchical organisation structure.
contactInfo	Contact Information	0-1	read-write	External:RPP-JSContact-Profile:Card	Contact information.

Identifier	Name	Card.	Mutability	Data Type	Description
contacts	Contacts	0+	read-write	LabelledAggregation[Contact Data Object]	Identifiers of contact objects associated with the organisation.
users	Users	0+	read-write	LabelledAggregation [User Data Object]	Identifiers of user objects associated with the organisation.
processes	Processes	0-1	read-only	Processes Object	Process Objects initiated on the organisation object.

Table 50

Operations

Operation: Create

Operation Identifier: create

Description: Provisions a new Organisation Data Object.

Input Data Elements: (None)

Parameters: (None)

Operation: Read

Operation Identifier: read

Description: Retrieves the data elements of an Organisation Data Object.

Input Data Elements: (None)

Parameters: (None)

Operation: Update

Operation Identifier: update

Description: Modifies the attributes of an existing Organisation Data Object.

Input Data Elements: (None)

Parameters: (None)

Operation: Delete

Operation Identifier: delete

Description: Removes an existing Organisation Data Object.

Input Data Elements: (None)

Parameters: (None)

Object: message

Object Name: Message Data Object

Object Type: Resource

Description: Represents a message placed into an organisation's queue by the server and retrieved by the client.

Reference: [This-ID]

Data Elements

Identifier	Name	Card.	Mutability	Data Type	Description
id	Message ID	1	read-only	Identifier	A server-unique identifier for the message object.
creationDate	Creation Date	1	read-only	Timestamp	The date and time when the message object was created and inserted into the queue.
status	Status	1	read-only	String	The current lifecycle status of the message object (`queued`, `delivered`, `removed`).

Identifier	Name	Card.	Mutability	Data Type	Description
data	Data	1	create-only	Base Message Object	The data related to a specific message, such as a transfer request or a review response.

Table 51

Operations

Operation: Create

Operation Identifier: create

Description: Inserts a new message into the queue for an organisation.

Input Data Elements: (None)

Parameters: (None)

Operation: Read

Operation Identifier: read

Description: Retrieves the data elements of one or more Message Data Objects.

Input Data Elements: (None)

Parameters: (None)

Operation: Query

Operation Identifier: query

Description: Retrieves zero or more Message Data Objects, optionally filtered by the specified parameters.

Input Data Elements: (None)

Parameters

Identifier	Name	Card.	Data Type	Description
type	Message Type	0-1	String	The type of the message to use as a filter when retrieving message objects.

Table 52

Operation: Delete

Operation Identifier: delete

Description: Removes a specific Message Data Object.

Input Data Elements: (None)

Parameters: (None)

Object: user

Object Name: User Data Object

Object Type: Resource

Description: Represents a user linked to an Organisation Data Object. Each user carries a set of RBAC roles that define the user's permissions within the context of the owning organisation. The lifecycle of a User Object is bound to its Organisation Object.

Reference: [This-ID]

Data Elements

Identifier	Name	Card.	Mutability	Data Type	Description
id	User Identifier	1	create-only	Identifier	A unique identifier for the user. MUST be globally unique in the server scope.
details	Details	1	read-write	External:RPP-JSContact-Profile:Card	The details of the user.
status	Status	1	read-write	User Status Object	The current status of the user.
processes	Processes	0-1	read-only	Processes Object	Process Objects initiated on the user object.
description	Description	0-1	read-write	String	A description of the user
organisation	Organisation	1	read-only	Aggregation [Organisation Data Object]	A reference to the owner organisation object.

Table 53

Operations

Operation: Create

Operation Identifier: create

Description: Creates a new User Object.

Input Data Elements: (None)

Parameters: (None)

Operation: Read

Operation Identifier: read

Description: Retrieves the data elements of a specific User Object.

Input Data Elements: (None)

Parameters: (None)

Operation: Update

Operation Identifier: update

Description: Modifies the read-write data elements (e.g., roles) of a specific User Object.

Input Data Elements: (None)

Parameters: (None)

Operation: Delete

Operation Identifier: delete

Description: Removes a specific User Object.

Input Data Elements: (None)

Parameters: (None)

Object: profile

Object Name: Profile Data Object

Object Type: Resource

Description: Represents a named set of protocol features and versions that defines the compatibility and capabilities of RPP server implementations.

Reference: [This-ID]

Data Elements

Identifier	Name	Card.	Mutability	Data Type	Description
name	Name	1	read-only	String	A unique name that identifies the profile.
description	Description	1	read-only	String	A human-readable description of the profile and its intended use.
version	Version	1	read-only	String	The version of the profile.
rppVersion	RPP Version	1	read-only	String	The minimum version of the RPP protocol that is supported or required for the profile.
objects	Objects	1+	read-only	String	The objects allowed for provisioning operations.
extensions	Extensions	0+	read-only	Extension Object	The extensions, including their names and versions, that are supported or required for the profile.
baseProfile	Base Profile	0-1	read-only	Aggregation [Profile Data Object]	A base profile that is extended by this profile.
defaultProfile	Default Profile	1	read-only	Boolean	Indicates whether this profile is the default profile.

Table 54

Operations

Operation: Read

Operation Identifier: read

Description: Retrieves the Profile Data Object identified by its name.

Input Data Elements: (None)

Parameters: (None)

Object: discoveryVersion

Object Name: Discovery Version Object

Object Type: Component

Description: Describes a version of RPP supported by the server.

Reference: [This-ID]

Data Elements

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
version	Version	1	read-only	String	The RPP version, for example "1.0".
baseUrl	Base URL	1	read-only	URL	The base URL specific for the version, used as a prefix for all endpoint URL templates.

Table 55

Object: discoveryEnvironment

Object Name: Discovery Environment Object

Object Type: Component

Description: Describes the environment of the RPP server.

Reference: [This-ID]

Data Elements

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
name	Name	1	read-only	String	The name of the RPP server.

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
status	Status	1	read-only	String	The operational status of the RPP server.

Table 56

Object: extension

Object Name: Extension Object

Object Type: Component

Description: Describes an extension supported by the server.

Reference: [This-ID]

Data Elements

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
name	Name	1	read-only	String	A short name for the extension.
id	Extension ID	1	read-only	String	A unique URN identifier for the extension.
version	Version	1	read-only	String	The version of the extension supported by the server.
url	URL	1	read-only	URL	The location of the extension specification.

Table 57

Object: discoveryIdnLgr

Object Name: Discovery IDN LGR Object

Object Type: Component

Description: Describes a Label Generation Ruleset (LGR) supported by the server for internationalized domain names.

Reference: [This-ID]

Data Elements

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
tld	TLDs	1+	read-only	String	The top-level domains to which the LGR applies.
name	Name	1	read-only	String	The IANA-registered name of the LGR.
url	URL	1	read-only	URL	The location of the LGR specification.
default	Default	1	read-only	Boolean	Indicates whether this LGR is the default for the specified TLDs.

Table 58

Object: discoveryNotice

Object Name: Discovery Notice Object

Object Type: Component

Description: Describes a notice from the server operator.

Reference: [This-ID]

Data Elements

Element Identifier	Element Name	Card.	Mutability	Data Type	Description
pubTime	Publication Time	1	read-only	Timestamp	The publication time of the notice.
text	Text	1	read-only	String	A human-readable description of the notice.

Table 59

Object: discovery

Object Name: Discovery Data Object

Object Type: Resource

Description: Describes the capabilities of an RPP server.

Reference: [This-ID]

Data Elements

Identifier	Name	Card.	Mutability	Data Type	Description
versions	Versions	1+	read-only	Discovery Version Object	The versions of RPP supported by the server.
environment	Environment	1	read-only	Discovery Environment Object	Information about the RPP server.
tlds	TLDs	1+	read-only	String	The TLDs supported by the server.
extensions	Extensions	0+	read-only	Extension Object	The extensions supported by the server.
profiles	Profiles	0+	read-only	String	The profiles supported by the server.
objects	Objects	1+	read-only	String	The resource collections supported by the server.
datetime	Date and Time	1	read-only	Timestamp	The current date and time of the server.

Identifier	Name	Card.	Mutability	Data Type	Description
links	Links	0+	read-only	LabelledAggregation [String]	Links to documents related to the use of RPP with the server, labelled with a relation type registered in the IANA "Link Relation Types" registry.
idnLgr	IDN LGRs	0+	read-only	Discovery IDN LGR Object	The Label Generation Rulesets supported by the server.
notices	Notices	0+	read-only	Discovery Notice Object	Notices from the server operator.

Table 60

Operations

Operation: Read

Operation Identifier: read

Description: Retrieves the Discovery Data Object.

Input Data Elements: (None)

Parameters: (None)

19.3. RPP User Role Values Registry

This document establishes the "RESTful Provisioning Protocol (RPP) User Role Values Registry". This registry serves as a catalogue of all user role values used within RPP.

```
Name of the registry: RPP User Role Values
Registry group: RESTful Provisioning Protocol (RPP)
Registration procedure: Expert Review
```

Fields to be registered:

- `name`: The name of the user role, for example "RPP example role".
- `url`: The URL for the user role specification, for example "https://www.iana.org/assignments/rpp-user-roles/rpp-example-role-1.0".
- `permission`: The permission level associated with the user role, for example "read-only", "read-write", or "admin".
- `description`: A human-readable description of the user role and its intended use.

19.4. RPP Domain Name Status Values Registry

This document establishes the "RESTful Provisioning Protocol (RPP) Domain Name Status Values Registry".

```
Name of the registry: RPP Domain Name Status Values
Registry group: RESTful Provisioning Protocol (RPP)
Registration procedure: Specification Required
```

Fields to be registered:

- `value`: The machine-readable status label, for example "clientDeleteProhibited".
- `description`: A human-readable description of the status and the conditions under which it applies.
- `set-by`: The entity that adds and removes the status value: "client", "server", or "client, server" for a value that the server manages and that a client can also manage where server policy permits.
- `reference`: A reference to the specification that defines the status value.

19.4.1. Initial Registrations

The initial registrations for the RPP Domain Name Status Values registry are the status values defined in Table 1, reproduced below with their reference.

Value	Set By	Description	Reference
clientDeleteProhibited	client	Requests to delete the object MUST be rejected.	[This-ID]
serverDeleteProhibited	server	Requests to delete the object MUST be rejected.	[This-ID]

Value	Set By	Description	Reference
clientHold	client	DNS delegation information MUST NOT be published for the object.	[This-ID]
serverHold	server	DNS delegation information MUST NOT be published for the object.	[This-ID]
clientRenewProhibited	client	Requests to renew the object MUST be rejected.	[This-ID]
serverRenewProhibited	server	Requests to renew the object MUST be rejected.	[This-ID]
clientTransferProhibited	client	Requests to transfer the object MUST be rejected.	[This-ID]
serverTransferProhibited	server	Requests to transfer the object MUST be rejected.	[This-ID]
clientUpdateProhibited	client	Requests to update the object (other than to remove this status) MUST be rejected.	[This-ID]
serverUpdateProhibited	server	Requests to update the object (other than to remove this status) MUST be rejected.	[This-ID]
inactive	server	Delegation information has not been associated with the object. This is the default status when a domain object is first created and there are no associated host objects for the DNS delegation. This status can also be set by the server when all host-object associations are removed.	[This-ID]
ok	server	This is the normal status value for an object that has no pending operations or prohibitions. This value is set and removed by the server as other status values are added or removed.	[This-ID]

Value	Set By	Description	Reference
pendingCreate	server	A transform command has been processed for the object, but the action has not been completed by the server. Server operators can delay action completion for a variety of reasons, such as to allow for human review or third-party action.	[This-ID]
pendingDelete	server	A transform command has been processed for the object, but the action has not been completed by the server. Server operators can delay action completion for a variety of reasons, such as to allow for human review or third-party action.	[This-ID]
pendingRenew	server	A transform command has been processed for the object, but the action has not been completed by the server. Server operators can delay action completion for a variety of reasons, such as to allow for human review or third-party action.	[This-ID]
pendingTransfer	server	A transform command has been processed for the object, but the action has not been completed by the server. Server operators can delay action completion for a variety of reasons, such as to allow for human review or third-party action.	[This-ID]
pendingUpdate	server	A transform command has been processed for the object, but the action has not been completed by the server. Server operators can delay action completion for a variety of reasons, such as to allow for human review or third-party action.	[This-ID]

Value	Set By	Description	Reference
rgpAddPeriod	server	This grace period is provided after the initial registration of a domain name. If the domain name is deleted by the registrar during this period, the registry provides a credit to the registrar for the cost of the registration.	[This-ID]
rgpAutoRenewPeriod	server	This grace period is provided after a domain name registration period expires and is extended (renewed) automatically by the registry. If the domain name is deleted by the registrar during this period, the registry provides a credit to the registrar for the cost of the renewal.	[This-ID]
rgpRenewPeriod	server	This grace period is provided after a domain name registration period is explicitly extended (renewed) by the registrar. If the domain name is deleted by the registrar during this period, the registry provides a credit to the registrar for the cost of the renewal.	[This-ID]
rgpTransferPeriod	server	This grace period is provided after the successful transfer of domain name registration sponsorship from one registrar to another registrar. If the domain name is deleted by the new sponsoring registrar during this period, the registry provides a credit to the registrar for the cost of the transfer.	[This-ID]
rgpRedemptionPeriod	server	This status value is used to describe a domain for which a delete operation has been received, but the domain has not yet been purged because an opportunity exists to restore the domain and abort the deletion process.	[This-ID]
rgpPendingRestore	server	This status value is used to describe a domain that is in the process of being restored after being in the rgpRedemptionPeriod state.	[This-ID]

Value	Set By	Description	Reference
rgpPendingDelete	server	This status value is used to describe a domain that has entered the purge processing state after completing the rgpRedemptionPeriod state. A domain in this status MUST also be in the pendingDelete status described above.	[This-ID]

Table 61: Initial RPP Domain Name Status Values Registrations

19.5. RPP Host Status Values Registry

This document establishes the "RESTful Provisioning Protocol (RPP) Host Status Values Registry".

```
Name of the registry: RPP Host Status Values
Registry group: RESTful Provisioning Protocol (RPP)
Registration procedure: Specification Required
```

Fields to be registered:

- `value`: The machine-readable status label, for example "clientDeleteProhibited".
- `description`: A human-readable description of the status and the conditions under which it applies.
- `set-by`: The entity that adds and removes the status value: "client", "server", or "client, server" for a value that the server manages and that a client can also manage where server policy permits.
- `reference`: A reference to the specification that defines the status value.

19.5.1. Initial Registrations

The initial registrations for the RPP Host Status Values registry are the status values defined in Table 4, reproduced below with their reference.

Value	Set By	Description	Reference
clientDeleteProhibited	client	Requests to delete the object MUST be rejected.	[This-ID]
serverDeleteProhibited	server	Requests to delete the object MUST be rejected.	[This-ID]
clientUpdateProhibited	client	Requests to update the object (other than to remove this status) MUST be rejected.	[This-ID]

Value	Set By	Description	Reference
serverUpdateProhibited	server	Requests to update the object (other than to remove this status) MUST be rejected.	[This-ID]
linked	server	The host object has at least one active association with another object, such as a domain object. Servers SHOULD provide services to determine existing object associations.	[This-ID]
ok	server	This is the normal status value for an object that has no pending operations or prohibitions. This value is set and removed by the server as other status values are added or removed.	[This-ID]
pendingCreate	server	A transform command has been processed for the object, but the action has not been completed by the server. Server operators can delay action completion for a variety of reasons, such as to allow for human review or third-party action.	[This-ID]
pendingDelete	server	A transform command has been processed for the object, but the action has not been completed by the server. Server operators can delay action completion for a variety of reasons, such as to allow for human review or third-party action.	[This-ID]
pendingTransfer	server	A transform command has been processed for the object's superordinate domain object (i.e. a transfer of the domain object is pending), but the action has not been completed by the server. Server operators can delay action completion for a variety of reasons, such as to allow for human review or third-party action.	[This-ID]

Value	Set By	Description	Reference
pendingUpdate	server	A transform command has been processed for the object, but the action has not been completed by the server. Server operators can delay action completion for a variety of reasons, such as to allow for human review or third-party action.	[This-ID]

Table 62: Initial RPP Host Status Values Registrations

19.6. RPP Contact Status Values Registry

This document establishes the "RESTful Provisioning Protocol (RPP) Contact Status Values Registry".

```
Name of the registry: RPP Contact Status Values
Registry group: RESTful Provisioning Protocol (RPP)
Registration procedure: Specification Required
```

Fields to be registered:

- `value`: The machine-readable status label, for example "clientDeleteProhibited".
- `description`: A human-readable description of the status and the conditions under which it applies.
- `set-by`: The entity that adds and removes the status value: "client", "server", or "client, server" for a value that the server manages and that a client can also manage where server policy permits.
- `reference`: A reference to the specification that defines the status value.

19.6.1. Initial Registrations

The initial registrations for the RPP Contact Status Values registry are the status values defined in Table 6, reproduced below with their reference.

Value	Set By	Description	Reference
clientDeleteProhibited	client	Requests to delete the object MUST be rejected.	[This-ID]
serverDeleteProhibited	server	Requests to delete the object MUST be rejected.	[This-ID]

Value	Set By	Description	Reference
clientTransferProhibited	client	Requests to transfer the object MUST be rejected.	[This-ID]
serverTransferProhibited	server	Requests to transfer the object MUST be rejected.	[This-ID]
clientUpdateProhibited	client	Requests to update the object (other than to remove this status) MUST be rejected.	[This-ID]
serverUpdateProhibited	server	Requests to update the object (other than to remove this status) MUST be rejected.	[This-ID]
linked	server	The contact object has at least one active association with another object, such as a domain object. Servers SHOULD provide services to determine existing object associations.	[This-ID]
ok	server	This is the normal status value for an object that has no pending operations or prohibitions. This value is set and removed by the server as other status values are added or removed.	[This-ID]
pendingCreate	server	A transform command has been processed for the object, but the action has not been completed by the server. Server operators can delay action completion for a variety of reasons, such as to allow for human review or third-party action.	[This-ID]
pendingDelete	server	A transform command has been processed for the object, but the action has not been completed by the server. Server operators can delay action completion for a variety of reasons, such as to allow for human review or third-party action.	[This-ID]

Value	Set By	Description	Reference
pendingTransfer	server	A transform command has been processed for the object, but the action has not been completed by the server. Server operators can delay action completion for a variety of reasons, such as to allow for human review or third-party action.	[This-ID]
pendingUpdate	server	A transform command has been processed for the object, but the action has not been completed by the server. Server operators can delay action completion for a variety of reasons, such as to allow for human review or third-party action.	[This-ID]

Table 63: Initial RPP Contact Status Values Registrations

19.7. RPP Organisation Role Status Values Registry

This document establishes the "RESTful Provisioning Protocol (RPP) Organisation Role Status Values Registry".

```
Name of the registry: RPP Organisation Role Status Values
Registry group: RESTful Provisioning Protocol (RPP)
Registration procedure: Specification Required
```

Fields to be registered:

- `value`: The machine-readable status label, for example "clientLinkProhibited".
- `description`: A human-readable description of the status and the conditions under which it applies.
- `set-by`: The entity that adds and removes the status value: "client", "server", or "client, server" for a value that the server manages and that a client can also manage where server policy permits.
- `reference`: A reference to the specification that defines the status value.

19.7.1. Initial Registrations

The initial registrations for the RPP Organisation Role Status Values registry are the status values defined in Table 8, reproduced below with their reference.

Value	Set By	Description	Reference
clientLinkProhibited	client	Requests to add new links to the role MUST be rejected.	[This-ID]
serverLinkProhibited	server	Requests to add new links to the role MUST be rejected.	[This-ID]
linked	server	The role has at least one active association with another object. This value is set and removed by the server and is not explicitly set by the client.	[This-ID]
ok	server	This is the normal status value for a role that has no active prohibitions. This value is set and removed by the server as other status values are added or removed.	[This-ID]

Table 64: Initial RPP Organisation Role Status Values Registrations

19.8. RPP Organisation Status Values Registry

This document establishes the "RESTful Provisioning Protocol (RPP) Organisation Status Values Registry".

```
Name of the registry: RPP Organisation Status Values
Registry group: RESTful Provisioning Protocol (RPP)
Registration procedure: Specification Required
```

Fields to be registered:

- `value`: The machine-readable status label, for example "clientDeleteProhibited".
- `description`: A human-readable description of the status and the conditions under which it applies.
- `set-by`: The entity that adds and removes the status value: "client", "server", or "client, server" for a value that the server manages and that a client can also manage where server policy permits.
- `reference`: A reference to the specification that defines the status value.

19.8.1. Initial Registrations

The initial registrations for the RPP Organisation Status Values registry are the status values defined in Table 10, reproduced below with their reference.

Value	Set By	Description	Reference
ok	server	This is the normal status value for an object that has no pending operations or prohibitions. This value is set and removed by the server as other status values are added or removed.	[This-ID]
hold	client, server	Organisation transform commands and new links MUST be rejected.	[This-ID]
terminated	client, server	The organisation has been terminated and MUST NOT be linked. Organisation transform commands and new links MUST be rejected.	[This-ID]
linked	server	The organisation object has at least one active association with another object. This value is not explicitly set by the client. Servers SHOULD provide services to determine existing object associations.	[This-ID]
clientLinkProhibited	client	Requests to add new links to the organisation MUST be rejected.	[This-ID]
serverLinkProhibited	server	Requests to add new links to the organisation MUST be rejected.	[This-ID]
clientUpdateProhibited	client	Requests to update the object (other than to remove this status) MUST be rejected.	[This-ID]
serverUpdateProhibited	server	Requests to update the object (other than to remove this status) MUST be rejected.	[This-ID]
clientDeleteProhibited	client	Requests to delete the object MUST be rejected.	[This-ID]
serverDeleteProhibited	server	Requests to delete the object MUST be rejected.	[This-ID]
pendingCreate	server	A transform command has been processed for the object, but the action has not been completed by the server. Server operators can delay action completion for a variety of reasons, such as to allow for human review or third-party action.	[This-ID]

Value	Set By	Description	Reference
pendingUpdate	server	A transform command has been processed for the object, but the action has not been completed by the server. Server operators can delay action completion for a variety of reasons, such as to allow for human review or third-party action.	[This-ID]
pendingDelete	server	A transform command has been processed for the object, but the action has not been completed by the server. Server operators can delay action completion for a variety of reasons, such as to allow for human review or third-party action.	[This-ID]

Table 65: Initial RPP Organisation Status Values Registrations

19.9. RPP User Status Values Registry

This document establishes the "RESTful Provisioning Protocol (RPP) User Status Values Registry".

```
Name of the registry: RPP User Status Values
Registry group: RESTful Provisioning Protocol (RPP)
Registration procedure: Specification Required
```

Fields to be registered:

- `value`: The machine-readable status label, for example "suspended".
- `description`: A human-readable description of the status and the conditions under which it applies.
- `set-by`: The entity that adds and removes the status value: "client", "server", or "client, server" for a value that the server manages and that a client can also manage where server policy permits.
- `reference`: A reference to the specification that defines the status value.

19.9.1. Initial Registrations

The initial registrations for the RPP User Status Values registry are the status values defined in Table 12, reproduced below with their reference.

Value	Set By	Description	Reference
active	server	This is the normal status value for a user that is permitted to authenticate and act within the permissions of its roles.	[This-ID]
pending	server	The user has been created, but activation has not been completed by the server. Server operators can delay activation for a variety of reasons, such as to allow for verification of the user's details or human review. Requests authenticated as the user MUST be rejected.	[This-ID]
suspended	client, server	The user has been temporarily suspended. Requests authenticated as the user MUST be rejected.	[This-ID]
deactivated	client, server	The user has been deactivated. Requests authenticated as the user MUST be rejected.	[This-ID]

Table 66: Initial RPP User Status Values Registrations

19.10. URN Sub-namespace for RPP (urn:ietf:params:rpp)

The IANA is requested to add the following value to the "IETF URN Sub-namespace for Registered Protocol Parameter Identifiers" registry, following the template in [RFC3553]: TODO: add filled in template, if we decide to use URN for profile identification, for example "urn:ietf:params:rpp:profile:example-profile"

```
Registry name: rpp
Specification: This Document
Repository: ?
Index value: ?
```

19.11. RPP Discovery registry

The IANA is requested to create a new registry for RPP discovery URLs, this registry will be used to register the well-known URLs for RPP discovery endpoints, used by RPP clients to discover the capabilities of a RPP server.

```
Name of the registry: RPP Discovery URLs
Registry group: RESTful Provisioning Protocol (RPP)
Registration procedure: Expert Review
```

Fields to be registered:

- `tld`: The top-level domain (TLD) for which the discovery URL is applicable, for example "example".

- `url`: The URL for the discovery endpoint, for example "https://rpp.example/.well-known/rpp.json".
- `description`: A human-readable description of the discovery URL and its intended use.

19.12. RPP Extension registry

The IANA is requested to create a new registry for RPP extensions, this registry will be used to register standardized extensions to the RPP protocol. Extensions are defined as additional features or capabilities that can be added to the base RPP protocol, for example support for additional resource types, additional operations or additional authentication methods.

```
Name of the registry: RPP Extensions
Registry group: RESTful Provisioning Protocol (RPP)
Registration procedure: Expert Review
```

Fields to be registered:

- `name`: The name of the extension, for example "RPP example extension".
- `version`: The version of the extension, for example "1.0".
- `url`: The URL for the extension specification, for example "https://www.iana.org/assignments/rpp-extensions/rpp-example-extension".
- `description`: A human-readable description of the extension and its intended use.

19.13. RPP Profile registry

The IANA is requested to create a new registry for RPP profiles, this registry will be used to register standardized profiles for the RPP protocol. Profiles are defined as specific configurations of the RPP protocol that are designed to meet the needs of specific use cases or environments, for example an EPP compatibility profile that defines a set of RPP features and behaviors that are compatible with the Extensible Provisioning Protocol (EPP) [RFC5730].

```
Name of the registry: RPP Profiles
Registry group: RESTful Provisioning Protocol (RPP)
Registration procedure: Expert Review
```

Fields to be registered:

- `name`: The name of the profile, for example "EPP compatibility profile".
- `id`: A unique URN identifier for the profile, for example "urn:ietf:params:rpp:profile:epp-compatibility".
- `version`: The version of the profile, for example "1.0".
- `url`: The URL for the profile specification, for example "https://www.iana.org/assignments/rpp-profiles/epp-compatibility-profile".
- `description`: A human-readable description of the profile and its intended use.

20. Security Considerations

> TODO: write security considerations, if any

21. Changes History

draft-ietf-rpp-data-objects -01 - -02

- add Abstract keyword to flag objects used only as base objects; Base Process Object and Base Message Object declared abstract #188
- Clarification on Processes supported and extended by a Data Object
- moved restore report from operation input to read-write element of Restore Process Object and replaced Report operation with Update
- Added User Status Object with status values, allowed transitions, and IANA registry, replacing the inline user status definition
- fixed User Object delete operation constraint that prevented any user from being deleted
- Added `X-Y` bounded cardinality notation and set cardinality of Restore Report `statements` to `1-2` (Issue #183)
- defined the terms "input data element" and "parameter", replacing "transient data element", and split IANA operation tables into "Input Data Elements" and "Parameters" (Issue #147)
- allowed extensions to define new processes
- moved process data element from Base Message Object to Review Response, Transfer Request and Transfer Outcome message objects, using specific process types where possible; registered all message objects in IANA (Issue #179)
- create-only definition clarified about reading (Issue #80)
- Moved requests for IANA registrations to here, from rpp-core document (Issue #151)
- Added Default Profile element to Profile Data Object (Issue #155)
- Added Discovery Data Object (Issue #136)
- Required the labels of the Discovery Data Object `links` element to be types registered in the IANA "Link Relations" registry (Issue #137)
- Added Profile Data Object (Issue #140)
- added Visibility data element attribute and generic includeElements parameter. Set authInfo of domain and contact to onDemand visibility (Issue #57)
- Added "Inheritance" section to describe how objects can use inheritance (Issue #134)
- Removed the urgent parameter from the update operation for Domain Object (Issue #52)
- Removed all mandatory ordering requirements for arrays, making them unordered (Issue #30)
- Added support for IDN (Internationalized Domain Names) (Issue #124)
- Added Message Data Object and new Uniform Interface "Query" operation (Issue #116)

- Created Status object per Data Object with allowed values and IANA registration. (Issue #118)
- defined a structured notation for operation input and output and applied it to the uniform interface, all process operations, the domain create operation and the discovery read operation
- removed the redefined processId data element from the Domain Transfer Process Object
- moved input data elements and parameters into the Input attribute of operations
- added a Set By column to status value tables and a set-by field to the status value registries (Issue #186)
- stated that the server assigns at least one status value on create (Issue #186)
- renamed Authorisation Information and Disclose component objects to "... Object" to match their references

draft-ietf-rpp-data-objects -00 - -01

- Added Organisation, Organisation Role and User Objects, based on RFC8543 (Issue #25)
- Added Renew Process Object and operations (Issue #76)
- reserve property names beginning with `@` for representation specifications #89
- add generic Create Process Object and Domain Create Process Object carrying the registration period #89
- add optional processId to all Process Objects #89
- add extensible Processes Object component and processes element to all Data Objects #89
- declare the unique identifier element explicitly in every object preamble #89
- add "Direct Access" flag exposing association data elements as addressable sub-resources #89
- editorial pass: fix spelling, grammar, subject-verb agreement, missing articles, cross-reference wording, and copy-paste object names
- Add External Data Types section and JSContact (Issue #83)
- restructure object processes: per-object "Processes" sections, process objects extending generic ones, add Domain Transfer Process Object, with IANA updates #119

draft-kowalik-rpp-data-objects-04 - draft-ietf-rpp-data-objects-00

- RPP working group adoption

draft-kowalik-rpp-data-objects -03 - -04

- correct `"postalInfo"` to be DictionaryComposition not DictionaryAggregation
- Added missing IANA tables for Contact, Postal Address and Postal Info. Added missing Operations. Added missing operation identifiers. Fixes #17, #18, #19, #20, #21.

draft-kowalik-rpp-data-objects -02 - -03

- add Object and Dictionary[Value Type] primitive data types
- change "Aggregation/Composition Dictionary" to "Dictionary Aggregation/Composition" (Issue #32)

- describe operations for contacts #15
- describe operations for hosts #16
- add Domain Update operation with urgent transient parameter from [RFC5910]
- add identifiers to all operations
- abstract common provisioning metadata into reusable component object
- define common RGP/restore process and Restore Process Object
- define common transfer operations and Transfer Process Object with support for pull and push transfers #23
- add domain-specific transfer operations with implicit renewal and subordinate host transfer
- add host/domain relationship terminology from RFC 5732
- restructure DNS data model aligned with draft-simmen-rpp-dns-data-01: redefine DNS Resource Record Object, add DNS Operational Controls Object, add DNS Data Object; update Domain, Host, and Nameserver objects to use it
- add DNSSEC support based on [RFC5910]: DS and DNSKEY record types with structured RDATA fields
- add NS, A, and AAAA RDATA structures and record type constraints to EPP compatibility profile for DNS Resource Record Object
- expand extensibility section with standardised/private extension mechanisms, extension points, and operation extensibility
- update IANA registration policy and registry structure to accommodate extension registrations

draft-kowalik-rpp-data-objects -01 - -02

- ontology of allowed basic datatypes #4
- add examples of associations #31

22. References

22.1. Normative References

[I-D.ietf-rpp-architecture] Kowalik, P. and M. Wullink, "RPP Architecture", Work in Progress, Internet-Draft, draft-ietf-rpp-architecture-01, 27 February 2026, <https://datatracker.ietf.org/doc/html/draft-ietf-rpp-architecture-01>.

[I-D.wullink-rpp-jscontact-profile] Wullink, M. and P. Kowalik, "JSContact Profile for the RESTful Provisioning Protocol (RPP)", Work in Progress, Internet-Draft, draft-wullink-rpp-jscontact-profile, 2026, <https://datatracker.ietf.org/doc/html/draft-wullink-rpp-jscontact-profile>.

[ITU.E164.2005] International Telecommunication Union, "The international public telecommunication numbering plan", ITU-T Recommendation E.164, February 2005.

[RFC0791] Postel, J., "Internet Protocol", STD 5, RFC 791, DOI 10.17487/RFC791, September 1981, <https://www.rfc-editor.org/info/rfc791>.

[RFC1035] Mockapetris, P., "Domain names - implementation and specification", STD 13, RFC 1035, DOI 10.17487/RFC1035, November 1987, <https://www.rfc-editor.org/info/rfc1035>.

[RFC1738] Berners-Lee, T., Masinter, L., and M. McCahill, "Uniform Resource Locators (URL)", RFC 1738, DOI 10.17487/RFC1738, December 1994, <https://www.rfc-editor.org/info/rfc1738>.

[RFC2026] Bradner, S., "The Internet Standards Process -- Revision 3", BCP 9, RFC 2026, DOI 10.17487/RFC2026, October 1996, <https://www.rfc-editor.org/info/rfc2026>.

[RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, DOI 10.17487/RFC2119, March 1997, <https://www.rfc-editor.org/info/rfc2119>.

[RFC3339] Klyne, G. and C. Newman, "Date and Time on the Internet: Timestamps", RFC 3339, DOI 10.17487/RFC3339, July 2002, <https://www.rfc-editor.org/info/rfc3339>.

[RFC3553] Mealling, M., Masinter, L., Hardie, T., and G. Klyne, "An IETF URN Sub-namespace for Registered Protocol Parameters", BCP 73, RFC 3553, DOI 10.17487/RFC3553, June 2003, <https://www.rfc-editor.org/info/rfc3553>.

[RFC3915] Hollenbeck, S., "Domain Registry Grace Period Mapping for the Extensible Provisioning Protocol (EPP)", RFC 3915, DOI 10.17487/RFC3915, October 2004, <https://www.rfc-editor.org/info/rfc3915>.

[RFC4291] Hinden, R. and S. Deering, "IP Version 6 Addressing Architecture", RFC 4291, DOI 10.17487/RFC4291, February 2006, <https://www.rfc-editor.org/info/rfc4291>.

[RFC5234] Crocker, D., Ed. and P. Overell, "Augmented BNF for Syntax Specifications: ABNF", STD 68, RFC 5234, DOI 10.17487/RFC5234, January 2008, <https://www.rfc-editor.org/info/rfc5234>.

[RFC5322] Resnick, P., Ed., "Internet Message Format", RFC 5322, DOI 10.17487/RFC5322, October 2008, <https://www.rfc-editor.org/info/rfc5322>.

[RFC5730] Hollenbeck, S., "Extensible Provisioning Protocol (EPP)", STD 69, RFC 5730, DOI 10.17487/RFC5730, August 2009, <https://www.rfc-editor.org/info/rfc5730>.

[RFC5732] Hollenbeck, S., "Extensible Provisioning Protocol (EPP) Host Mapping", STD 69, RFC 5732, DOI 10.17487/RFC5732, August 2009, <https://www.rfc-editor.org/info/rfc5732>.

[RFC5733] Hollenbeck, S., "Extensible Provisioning Protocol (EPP) Contact Mapping", STD 69, RFC 5733, DOI 10.17487/RFC5733, August 2009, <https://www.rfc-editor.org/info/rfc5733>.

[RFC5890] Klensin, J., "Internationalized Domain Names for Applications (IDNA): Definitions and Document Framework", RFC 5890, DOI 10.17487/RFC5890, August 2010, <https://www.rfc-editor.org/info/rfc5890>.

[RFC5892] Faltstrom, P., Ed., "The Unicode Code Points and Internationalized Domain Names for Applications (IDNA)", RFC 5892, DOI 10.17487/RFC5892, August 2010, <https://www.rfc-editor.org/info/rfc5892>.

[RFC5893] Alvestrand, H., Ed. and C. Karp, "Right-to-Left Scripts for Internationalized Domain Names for Applications (IDNA)", RFC 5893, DOI 10.17487/RFC5893, August 2010, <https://www.rfc-editor.org/info/rfc5893>.

[RFC5952] Kawamura, S. and M. Kawashima, "A Recommendation for IPv6 Address Text Representation", RFC 5952, DOI 10.17487/RFC5952, August 2010, <https://www.rfc-editor.org/info/rfc5952>.

[RFC6903] Snell, J., "Additional Link Relation Types", RFC 6903, DOI 10.17487/RFC6903, March 2013, <https://www.rfc-editor.org/info/rfc6903>.

[RFC8126] Cotton, M., Leiba, B., and T. Narten, "Guidelines for Writing an IANA Considerations Section in RFCs", BCP 26, RFC 8126, DOI 10.17487/RFC8126, June 2017, <https://www.rfc-editor.org/info/rfc8126>.

[RFC8174] Leiba, B., "Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words", BCP 14, RFC 8174, DOI 10.17487/RFC8174, May 2017, <https://www.rfc-editor.org/info/rfc8174>.

[RFC8543] Zhou, L., Kong, N., Yao, J., Gould, J., and G. Zhou, "Extensible Provisioning Protocol (EPP) Organization Mapping", RFC 8543, DOI 10.17487/RFC8543, March 2019, <https://www.rfc-editor.org/info/rfc8543>.

[RFC9499] Hoffman, P. and K. Fujiwara, "DNS Terminology", BCP 219, RFC 9499, DOI 10.17487/RFC9499, March 2024, <https://www.rfc-editor.org/info/rfc9499>.

[RFC9553] Stepanek, R. and M. Loffredo, "JSContact: A JSON Representation of Contact Data", RFC 9553, DOI 10.17487/RFC9553, May 2024, <https://www.rfc-editor.org/info/rfc9553>.

22.2. Informative References

[I-D.simmen-rpp-dns-data] Simmen, C. and P. Kowalik, "DNS data representation for use in RESTful Provisioning Protocol (RPP)", Work in Progress, Internet-Draft, draft-simmen-rpp-dns-data-01, 20 October 2025, <https://datatracker.ietf.org/doc/html/draft-simmen-rpp-dns-data-01>.

[RFC3596] Thomson, S., Huitema, C., Ksinant, V., and M. Souissi, "DNS Extensions to Support IP Version 6", STD 88, RFC 3596, DOI 10.17487/RFC3596, October 2003, <https://www.rfc-editor.org/info/rfc3596>.

[RFC4034] Arends, R., Austein, R., Larson, M., Massey, D., and S. Rose, "Resource Records for the DNS Security Extensions", RFC 4034, DOI 10.17487/RFC4034, March 2005, <https://www.rfc-editor.org/info/rfc4034>.

[RFC5910] Gould, J. and S. Hollenbeck, "Domain Name System (DNS) Security Extensions Mapping for the Extensible Provisioning Protocol (EPP)", RFC 5910, DOI 10.17487/RFC5910, May 2010, <https://www.rfc-editor.org/info/rfc5910>.

Authors' Addresses

Pawel Kowalik
DENIC
Email: pawel.kowalik@denic.de
URI: https://denic.de/

Maarten Wullink
SIDN Labs
Email: maarten.wullink@sidn.nl
URI: https://sidn.nl/